

CUDA Kernels: GPU & Parallel Programming from First Principles

Arpan Pathak

Foreword

Computing is entering its parallel age, and the GPU is the machine that defines it.

For sixty years, the default path to a faster program was a faster serial processor: a higher clock, a smarter pipeline, a larger cache. That path ran into a wall in the mid-2000s, when clock speeds stopped climbing and silicon stopped cooperating. The industry's answer was not surrender but a change of question - from "how fast can one core go?" to "how many cores can we set free at once?" The GPU is that answer, multiplied a hundred thousand times. A modern accelerator executes tens of trillions of floating-point operations per second, moves memory at terabytes per second, and keeps more threads in flight than there are people on Earth - all directed by code that you, an individual programmer, can write and understand completely.

The instruction set is documented, the programming model is teachable, and the performance is explained by a handful of principles - the warp, the memory hierarchy, the roofline model - that fit on a single page and govern every GPU ever built. Few subjects in computer science offer this much return per hour of study.

The Hardware Gap

A CUDA library call can hide the hardware behind it. A matrix multiply runs at 3% of peak when threads in a warp read columns instead of rows. A reduction can silently drop half of its data when threads diverge across a `__syncthreads()`. A host allocation that is pageable instead of pinned doubles transfer latency. None of these failures print an error. They produce a slow benchmark or a subtly wrong result.

This book covers the hardware model, the CUDA C++ programming model, and the C++ and Rust ecosystem around them, so those failures are diagnosable instead of mysterious.

What You Will Build

Every chapter builds toward one project: **a complete GPU image-processing pipeline** - read an image, convert it to greyscale, apply a separable Gaussian blur, run a Sobel edge detector, and write the result - implemented three times:

1. in **CUDA C++** with hand-written, fully commented kernels;
2. with the **Thrust/CUB/cuBLAS** library ecosystem;
3. in **pure Rust** with NVIDIA's experimental **CUDA-Oxide** compiler, which turns idiomatic Rust into PTX.

You will build the same kind of pipeline a camera vendor would ship: pinned-memory transfers, streamed double buffering, an occupancy-tuned kernel configuration, and reproducible benchmarks. When you have finished, you will be able to look at any CUDA kernel - including the ones inside the libraries you already use - and explain, line by line, what it does and why it is fast.

Who This Book Is For

You should read this book if:

- You can write C++ or Rust, but every GPU program you have written so far was a library call you did not fully understand.
- You have launched a kernel, seen it produce garbage, and had no idea whether the bug was in your index arithmetic, your memory layout, or your synchronisation.
- You suspect that most GPU tutorials skip the hardware model and want to understand the primitives - the warp, the streaming multiprocessor, the memory hierarchy - before touching a single CUDA API.
- You write Rust and want to know what CUDA-Oxide changes, and what it does not.
- You do not own a GPU and want to learn on the free compute that the cloud gives away (see the next section).
- You ship software whose performance budget is measured in microseconds and whose correctness budget is zero.

You do not need prior GPU experience. You need to be willing to sit with the hardware model. This book does not hand-wave the memory hierarchy. Every term is defined when it first appears; every primitive - every type, every built-in variable, every API call - is

described before it is used. Where the book refers to a number (register counts, memory bandwidths, transaction sizes), it gives you the reasoning behind it, not just the number.

You Do Not Need to Own a GPU

Every line of code in this book runs on any NVIDIA GPU with CUDA 12.x - including the free ones. If you do not own a GPU, the cloud has you covered, and most providers give away enough free compute to finish this entire book:

- **Google Colab** - free T4 GPUs in browser notebooks, zero setup; the fastest way to run your first kernel.
- **Kaggle Notebooks** - free GPU hours (roughly 30 per week, refreshed weekly), data-science friendly.
- **Google Cloud** - a new-account trial credit (about USD 300 at the time of writing) covers serious L4 and A100 sessions.
- **Microsoft Azure** - a new-account credit (about USD 200) for NC/ND-series GPU virtual machines.
- **AWS** - GPU instances (g4dn, g5, p4); the free tier is CPU-only, but the Activate and Educate programmes grant credits to startups and students.
- **Paperspace / Gradient** - GPU notebooks and cloud workstations with free and low-cost tiers.
- **Lambda, RunPod, Vast.ai** - cheap on-demand GPUs (RTX 4090 up to H100) when you outgrow the free tiers.
- **NVIDIA LaunchPad** - free, time-boxed hands-on labs on real NVIDIA hardware.
- **Modal** - serverless GPU code with recurring free compute credits (about USD 30 per month at the time of writing).

Credit amounts and session limits change frequently; check the current terms before you sign up. The repository README contains a fuller comparison table to help you choose.

The Structure

The book is organised into six parts:

Part I - Foundations of GPU Computing (Chapters 1-3) covers the mathematics of parallelism, the GPU hardware model, and the CUDA programming model. Read this part carefully; every later chapter assumes the primitives defined here.

Part II - Writing CUDA C++ Kernels (Chapters 4-6) covers memory management, synchronisation, atomics, and asynchronous execution with streams and events.

Part III - Optimisation & Advanced Patterns (Chapters 7-9) covers memory optimisation, the canonical parallel algorithms (reduction, scan, histogram), and a complete, step-by-step optimisation of matrix multiplication.

Part IV - Modern C++ & The CUDA Ecosystem (Chapters 10-12) covers RAI wrappers, templates and modern C++ idioms, the Thrust/CUB/cuBLAS libraries, and runtime compilation with NVRTC.

Part V - Rust, CUDA-Oxide & Safe GPU Programming (Chapters 13-15) covers Rust host code driving CUDA kernels, NVIDIA's experimental CUDA-Oxide compiler for writing kernels in pure Rust, and the image-processing capstone.

Part VI - The Engineering Mindset (Chapter 16) covers profiling with Nsight Compute, debugging with Compute Sanitizer, and reproducible performance engineering.

A Note on the Coding Standards

This project has a constitution. You will find it as `CODING_STANDARDS.md` in the repository root. It is not a suggestion. Every code block in this book follows it:

- Every primitive is explained before it is used.
- No magic numbers - if it is not `0`, `1`, `-1`, or a power of two required by the CUDA API, it gets a named constant.
- Every kernel is commented line by line.
- Every CUDA API call that can fail is checked.
- Every `__syncthreads()` and every atomic carries a comment stating which data it protects and why.

A Note on Hardware and Honesty

The examples in this book target the CUDA 12.x toolkit and are written against the compute capability of modern NVIDIA GPUs (Ada and Hopper architectures, compute capability 8.x and 9.0). You do not need to own this hardware: the section “You Do Not Need to Own a GPU” lists the cloud options, and the free tiers alone are enough for everything in this book. Where a feature is architecture-specific, the book says so explicitly.

This book is also direct about the tooling. NVIDIA’s CUDA-Oxide is an experimental, alpha-stage compiler; its API is evolving and its syntax may change. The chapters that cover it describe the project as it exists today, with code written in the style of its documented examples. Treat those chapters as a map of the territory, not a surveyor’s certificate.

If you find a bug in the book - in the prose or in the code - open an issue or submit a pull request. This is a living document. The GPU does not stop changing, and neither should the book.

Chapter 1 starts from the mathematics and builds the programming model from the hardware up. No GPU knowledge is assumed; the requirements are the C++ or Rust listed above and a machine that can run CUDA 12.x examples.

- *Arpan Pathak*

Chapter 1: The Mathematics of Parallelism

*“The purpose of computing is insight, not numbers.” — Richard Hamming, *Numerical Methods for Scientists and Engineers* (1962)*

This chapter defines the quantitative vocabulary used throughout the book: *speedup*, *efficiency*, *Amdahl's law*, *Gustafson-Barsis scaling*, *strong and weak scaling*, *Flynn's taxonomy*, and the *roofline model*. These concepts are the tools for deciding whether an optimisation is worth the effort and which hardware resource limits a kernel.

1.1 Latency, Throughput, and the Meaning of “Faster”

A program described as “slow” is usually slow in one of two distinct senses. On a GPU the distinction determines the correct optimisation strategy.

Latency is the elapsed time between the start of an operation and its completion. A memory access has latency. A network round trip has latency. Latency is measured in time units, typically nanoseconds or milliseconds.

Throughput is the number of operations completed per unit time. A pipeline that processes 60 frames per second has a throughput of 60 Hz. Throughput is measured in operations per second.

A CPU is engineered to **minimise latency**. It uses a small number of fast cores, each with branch prediction, out-of-order execution, and large caches that hide the latency of DRAM.

A GPU is engineered to **maximise throughput**. It contains a large number of simpler execution units. Individually they are slower than a CPU core, but collectively they complete millions of operations per clock. The GPU hides latency not by predicting what happens next but by keeping many independent threads in flight, so that execution units always have work while other threads wait.

Throughput and concurrency are related by a well-known queueing result. In a system that sustains a steady flow of work:

$$\text{throughput} = \frac{\text{concurrency}}{\text{latency}}$$

This is Little's law. To see why it matters for GPUs, suppose a memory access takes 500 cycles and the machine has no other work to issue during that window. The memory system then delivers one access per 500 cycles, regardless of clock speed. If the machine keeps 1,000 independent accesses in flight, each still taking 500 cycles, it completes 1,000 accesses every 500 cycles: a 1,000-fold increase in throughput with no change in latency. This is why GPUs use many concurrent threads. **Latency is not reduced; it is amortised over concurrency.**

This result appears throughout the book as a defining property:

Primitive - latency hiding. When an execution unit must wait for a slow operation, such as a memory access or a division, the unit would otherwise be idle. The GPU switches to another ready thread. The cost of the wait is hidden, not eliminated.

A GPU is therefore well suited to computations that can be divided into many independent pieces and poorly suited to a single sequential computation. The mathematics of that division is the subject of this chapter.

1.2 Speedup and Efficiency

Let T_1 be the execution time of a program on one processing unit (one core or one thread), and T_p its execution time on p processing units. The standard definitions are:

Speedup is the ratio of the serial time to the parallel time:

$$S(p) = \frac{T_1}{T_p}$$

A perfect speedup of p means the program completes in $1/p$ of its serial time.

Efficiency is the speedup per processing unit:

$$E(p) = \frac{S(p)}{p} = \frac{T_1}{p \cdot T_p}$$

An efficiency of 1.0 means every processing unit contributes in proportion to the ideal. An efficiency of 0.5 means half of the added hardware's potential is not converted into performance. Efficiency is the more diagnostic of the two: speedup measures the improvement over the serial run, while efficiency measures how much of the added hardware is actually used. A report of “10x speedup on 64 cores” says nothing by itself; the efficiency is $10/64 = 0.156$, meaning 84% of the hardware is idle.

Two bounds follow from the definitions. Because $T_p > 0$:

$$S(p) = \frac{T_1}{T_p} \leq p \quad \text{and} \quad 0 < E(p) \leq 1$$

Equality holds only when $T_p = T_1/p$, meaning the work divides perfectly among processors and every processor is busy for the whole execution. Load imbalance, communication, synchronisation, redundant work, memory contention, and end-of-computation idle time all increase T_p and reduce efficiency. The serial fraction introduced by Amdahl's law creates the same ceiling in another form.

A GPU may have tens of thousands of threads in flight. If the achievable efficiency is 20%, the hardware is five times larger than the useful work requires. Most optimisations in this book raise efficiency by keeping threads busy, keeping memory transactions full, and removing serialisation points.

1.3 Amdahl's Law

Gene Amdahl observed in 1967 that every program contains a serial fraction: the part that cannot be parallelised, such as initialisation, I/O, a single reduction step, or a dependency chain. Let f be the fraction of the *serial* execution time that is strictly serial. The parallelisable fraction is $(1 - f)$. If the parallel part is perfectly parallelised across p units, the best possible total time is:

$$T_p = f \cdot T_1 + \frac{(1 - f) \cdot T_1}{p}$$

The maximum speedup is therefore:

$$S(p) = \frac{T_1}{f \cdot T_1 + \frac{(1-f) \cdot T_1}{p}} = \frac{1}{f + \frac{1-f}{p}}$$

The limit as $p \rightarrow \infty$ is:

$$\lim_{p \rightarrow \infty} S(p) = \frac{1}{f}$$

The serial fraction is a hard ceiling. If 5% of a program is serial, no amount of parallelism can produce more than a 20x speedup, because the serial part still requires $0.05 \cdot T_1$ regardless of the number of processing units.

The serial fraction is easy to underestimate on a GPU. Host launch overhead, a single reduction step, and dependency chains all count. As a worked example, consider a pipeline with 10 microseconds of host overhead (serial) and a kernel that takes 100 microseconds on one GPU and scales perfectly. Here $f = 10/110 \approx 0.091$, so the maximum speedup is $1/0.091 \approx 11$. No number of GPUs can make the pipeline faster than 11x. Chapter 6 exists because hiding host overhead with streams is one way to reduce the effective serial fraction.

Amdahl's law assumes a **fixed problem size**. When the problem grows with the number of processing units, the conclusion changes. That is the subject of the next section.

1.4 Gustafson-Barsis Law

John Gustafson and Edwin Barsis argued in 1988 that users do not normally keep the problem size fixed when they acquire more hardware. They solve larger problems in the same wall-clock time. Let s be the serial fraction of the *parallel* execution time, measured when all p units are busy. The scaled speedup is:

$$S(p) = p + (1 - p) \cdot s$$

The derivation shows where the formula comes from. Let T_p be the wall-clock time on p units. Split it into a serial part $s \cdot T_p$ and a parallel part $(1 - s) \cdot T_p$. If the parallel part ran on one unit instead of p , it would take $p \cdot (1 - s) \cdot T_p$. The serial part takes the same time in either case, so the estimated single-unit time is:

$$T_1 = s \cdot T_p + p \cdot (1 - s) \cdot T_p$$

and:

$$S(p) = \frac{T_1}{T_p} = s + p \cdot (1 - s) = p + (1 - p) \cdot s$$

The term $(1 - p) \cdot s$ is negative for $p > 1$, so the speedup is always somewhat below p . The gap depends on the serial fraction. If $s = 0.01$ and $p = 1,000$, the scaled speedup is approximately $1,000 - 9.99 \approx 990$. The same serial fraction would cap Amdahl-style fixed-workload speedup at 100, because in Gustafson scaling the parallel workload also grows.

The two laws answer different questions:

- **Amdahl:** “How much faster does a fixed workload run with more units?”
- **Gustafson:** “How much larger a workload can run in the same time with more units?”

Increasing an image resolution or a matrix dimension is Gustafson scaling: the workload grows and the parallel fraction grows with it. Optimising a fixed-size kernel is Amdahl scaling. Identifying the regime determines which optimisation is meaningful.

1.5 Strong Scaling and Weak Scaling

The two regimes have standard names:

- **Strong scaling** fixes the problem size and increases the number of units. Its limit is Amdahl’s law. Strong scaling applies to latency-critical workloads whose size is fixed by the application, such as a 1080p frame that must be processed at 60 Hz.
- **Weak scaling** fixes the problem size *per unit* and increases the number of units, so the total problem grows with the hardware. Its limit is Gustafson’s law. Weak scaling applies to throughput workloads such as larger batches or larger grids.

Kernel configuration decisions are strong- or weak-scaling decisions in miniature. Using more threads per element increases parallel work per thread (weak scaling); using fewer threads that each do more work holds total work fixed (strong scaling).

1.6 Types of Parallelism

Parallelism is not a single idea. Each form maps to different hardware mechanisms:

- **Task parallelism** runs different functions concurrently on different data, for example decoding one frame while filtering another. On a GPU, task parallelism is coarse: the hardware has a small number of independent execution contexts, exposed to CUDA as streams (Chapter 6).

- **Data parallelism** runs the same function on many data elements. This is the native mode of a GPU: one kernel, millions of elements.
- **Pipeline parallelism** splits a computation into stages and has each stage process a different element at the same time. A convolution pipeline may load, compute, and store in overlapping stages. On a GPU, pipelining appears both in hardware (instruction and memory pipelines) and in software (double buffering, Chapter 6).

A GPU is a **data-parallel** machine. When the term “massively parallel” is used for GPUs, it means data parallelism. Task parallelism on a GPU is an available but secondary technique.

1.7 Flynn’s Taxonomy: SISD, SIMD, SIMT, MIMD

Michael Flynn’s 1966 taxonomy classifies computers by the number of instruction streams and data streams they operate on:

- **SISD** (single instruction, single data): a conventional scalar CPU core. One instruction stream operates on one data stream.
- **SIMD** (single instruction, multiple data): one instruction operates on a vector of data elements. SSE and AVX on x86 CPUs are examples. The compiler or programmer packs data into wide registers; a 256-bit AVX register holds eight 32-bit floats, and one instruction can add all eight at once.
- **MIMD** (multiple instruction, multiple data): each processing unit runs its own instruction stream on its own data. Multi-core CPUs and GPU streaming multiprocessors as a whole belong here.
- **SIMT** (single instruction, multiple threads): NVIDIA’s execution model, combining aspects of SIMD and MIMD. The hardware fetches one instruction per cycle for a group of threads called a **warp** (Chapter 2). Each thread has its own registers and program counter. The group executes one instruction at a time, but each lane applies it to its own data.

SIMT differs from SIMD in an important way. In SIMD, data elements are packed into a vector register, and all lanes always execute the same instruction. In SIMT, threads appear to execute independently. The hardware executes them in lockstep only when their control flow agrees. If threads in the same warp take different branches, the hardware serialises the branches (Chapter 5). SIMT therefore provides the programming convenience of MIMD (data-dependent control flow per thread) while exposing the cost of SIMD when paths diverge.

1.8 Arithmetic Intensity and the Roofline Model

The roofline model, introduced by Williams, Waterman, and Patterson in 2009, answers one question: for a given computation, is the limit set by the arithmetic units or by the memory system?

1.8.1 Arithmetic Intensity: Work per Byte

A GPU has two limiting resources with different units:

- **Arithmetic units** (FP32 cores) perform work at a maximum rate P_{peak} FLOP/s.
- **Memory system** (DRAM, L2, buses) delivers data at a maximum rate B bytes/s.

Before the arithmetic units can operate on a value, the value must arrive from memory. Memory bandwidth is a finite per-second budget. It is therefore useful to describe a kernel by the ratio of its work to its data movement:

$$I = \frac{\text{FLOPs}}{\text{Bytes}}$$

I is the **arithmetic intensity**: the number of floating-point operations performed per byte moved. It is analogous to fuel efficiency: miles per gallon.

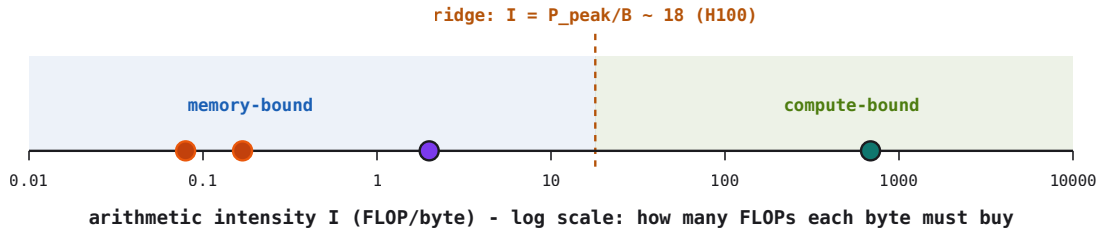
The ratio decides which resource runs out first.

- A **low-intensity** kernel performs few operations per byte. It exhausts the memory byte budget while the arithmetic units still have capacity. Such a kernel is **memory-bound**. Additional arithmetic throughput does not help because the memory system is the bottleneck.
- A **high-intensity** kernel performs many operations per byte. The arithmetic units saturate before the memory system does. Such a kernel is **compute-bound**. Additional bandwidth does not help because the arithmetic units are the bottleneck.

Data reuse raises intensity. A byte loaded and used for many operations contributes to many FLOPs. A byte loaded, used once, and discarded contributes to one FLOP.

ARITHMETIC INTENSITY: HOW MUCH WORK PER BYTE

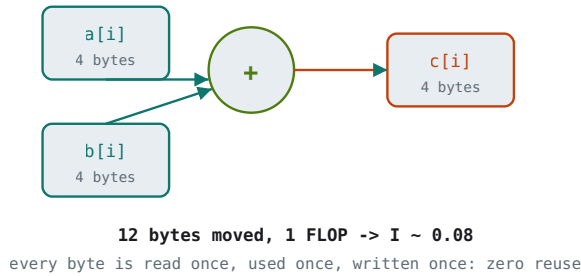
$I = \text{FLOPs} / \text{Bytes moved from memory}$ - a kernel's "fuel efficiency": the work each byte must pay for



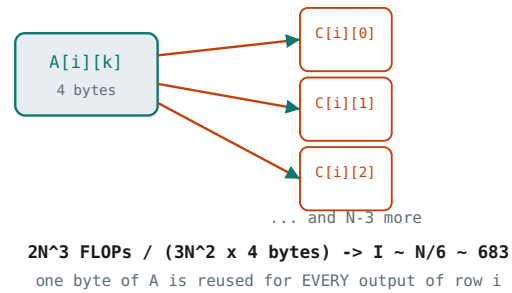
four kernels on the same machine - the only thing that changes is how many times each byte is reused:

- vector add: $I = 0.08$ FLOP/byte - 1 add for every 12 bytes (read $a[i]$, read $b[i]$, write $c[i]$)
- SAXPY: $I = 0.17$ - 2 ops (multiply + add) for the same 12 bytes - still memory-bound
- FFT (typical): $I \sim 2$ - each datum is reused across several butterfly stages
- SGEMM: $I \sim 683$ - every byte of A and B is reused $\sim N/6$ times ($N = 4096$) - compute-bound

LOW INTENSITY: EACH BYTE WORKS ONCE



HIGH INTENSITY: EACH BYTE WORKS $\sim N/6$ TIMES



The diagram shows four kernels on the same machine. A vector add moves 12 bytes (two reads and one write) for one FLOP, giving intensity $1/12 \approx 0.08$ FLOP/byte. A dense matrix multiply reuses each loaded value many times; depending on the problem and implementation, its intensity can be hundreds of FLOP/byte. The machine did not change; the degree of data reuse did.

1.8.2 The Ridge Point

Let P_{peak} be the peak floating-point throughput in FLOP/s and B the peak memory bandwidth in bytes/s. If a kernel has intensity I , the achievable performance satisfies:

$$P \leq \min(P_{\text{peak}}, I \cdot B)$$

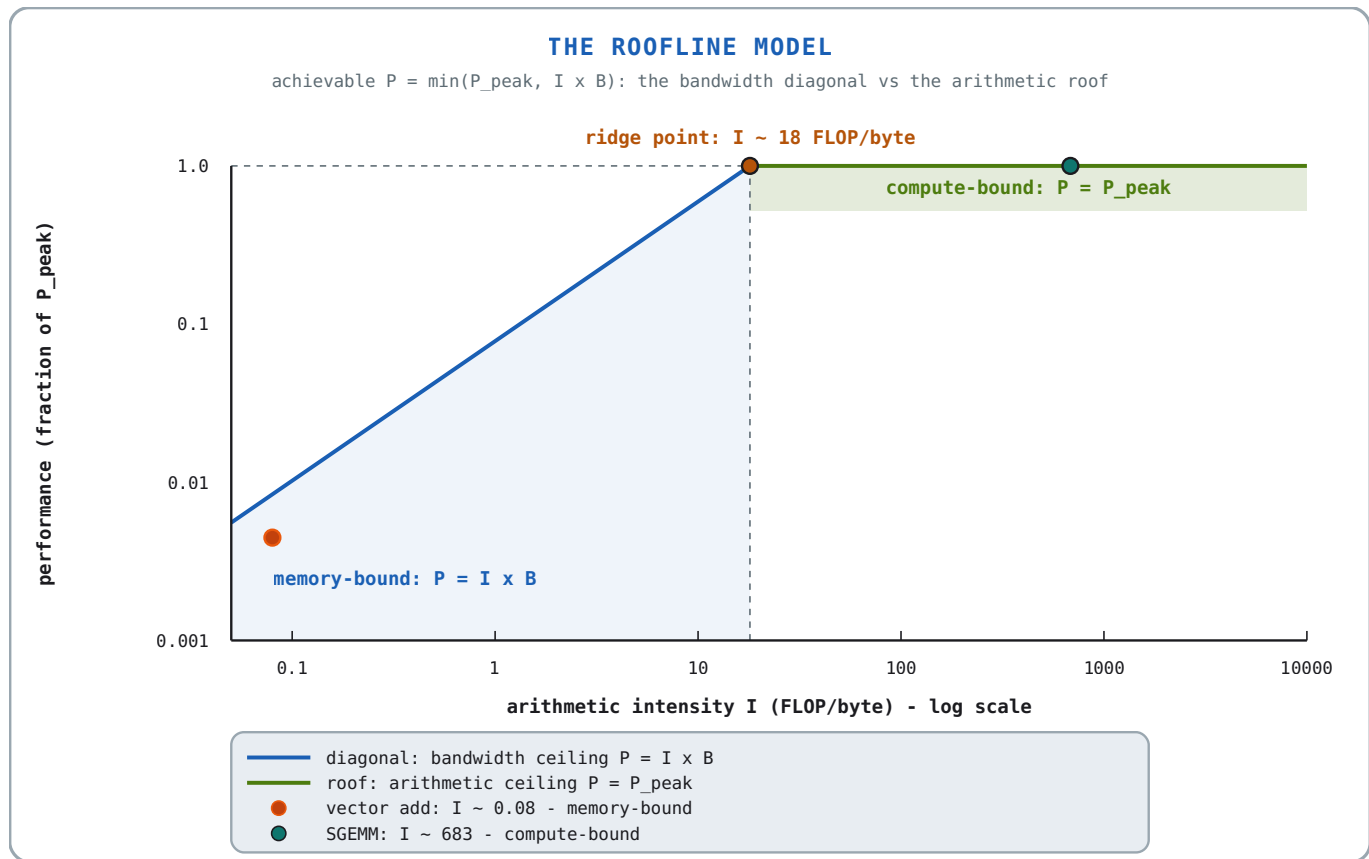
The term $I \cdot B$ follows from unit bookkeeping. A kernel performing I FLOPs per byte must receive P/I bytes/s to sustain a performance of P FLOP/s. Because the memory system can deliver at most B bytes/s:

$$\frac{P}{I} \leq B \implies P \leq I \cdot B$$

This is the **bandwidth ceiling**. The arithmetic units impose the second ceiling, P_{peak} . Both constraints hold simultaneously, so the achievable rate is their minimum.

The two ceilings meet at the **ridge point**, the intensity at which the memory system and arithmetic units are exactly balanced:

$$I_{\text{ridge}} = \frac{P_{\text{peak}}}{B}$$



Below the ridge point, performance is limited by the bandwidth diagonal ($I \cdot B$). Above it, performance is limited by the arithmetic roof (P_{peak}). The ridge point converts the machine's two raw specifications into one number that can be compared against any kernel. Hardware chapters in this book quote it for each generation (e.g., §2.1).

Worked numbers. Consider a GPU with $P_{\text{peak}} = 40$ TFLOP/s of FP32 and $B = 1$ TB/s. Its ridge point is 40 FLOP/byte. A vector add performs one addition per output element $c[i] = a[i] + b[i]$. It reads two 4-byte floats and writes one 4-byte float, moving 12 bytes:

$$I = \frac{1 \text{ FLOP}}{(2 \text{ reads} + 1 \text{ write}) \times 4 \text{ bytes}} = \frac{1}{12} \approx 0.08 \text{ FLOP/byte}$$

That is roughly 500x below the ridge point. Vector addition is memory-bound; no arithmetic optimisation helps, while bandwidth optimisation does (coalescing, §2.7; avoiding redundant reads, Chapter 7). This result motivates Chapter 7: for many real kernels, the bytes are the problem, not the arithmetic.

1.8.3 CPU-Bound, Memory-Bound, and I/O-Bound

The roofline model classifies resources inside the GPU. The same reasoning applies to whole programs on any machine, where the limiting resource may also be outside the processor. Every program is eventually limited by one of three resources:

- **The CPU:** instruction-issue and execution capacity.
- **The memory system:** DRAM bandwidth and latency.
- **An input/output device:** disk, network, PCIe bus, or GPU transfer.

A program is named after the resource that limits its total execution time.

Definition - the bound of a program. A program is *bound by resource R* if its total execution time is approximately the time it spends using (or waiting on) R . Concretely: R 's utilisation is near 100% while the other resources idle, and increasing R 's capacity alone reduces total time proportionally, while increasing any other resource's capacity changes nothing.

If the total time T is split into busy time for the CPU, memory, and I/O, the verdicts follow from one ratio:

- **CPU-bound:** $T_{\text{CPU}}/T \approx 1$. Example: SHA-256 hashing of ten million in-memory keys. The data is already in RAM, so memory and I/O idle while the CPU executes thousands of instructions per key. A faster CPU reduces the time; faster RAM or a faster disk does not.

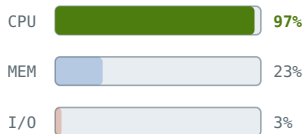
- **Memory-bound:** $T_{\text{mem}}/T \approx 1$. Example: the vector add of §1.8.2, whose intensity lies far below the ridge point. Faster memory helps; additional arithmetic throughput does not.
- **I/O-bound:** $T_{\text{io}}/T \approx 1$. Example: streaming 10 GB from disk. CPU and memory idle waiting for data. A faster disk, or asynchronous I/O (Chapter 6), helps; a faster CPU does not.

WHICH RESOURCE IS THE WALL?

a program is named after the resource whose utilisation hits ~100% - the others idle

CPU-BOUND

SHA-256 hashing of 10M
in-memory keys, no disk



CPU ~100% busy - the wall

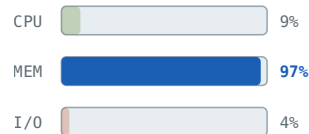
2x CPU speed -> time halves

$$T_{\text{CPU}} / T \sim 1.0$$

fix: faster CPU, offload work

MEMORY-BOUND

vector add of 10^9 floats
on the GPU ($I \sim 0.08$)



memory ~100% busy - the wall

2x bandwidth -> time halves

$$T_{\text{MEM}} / T \sim 1.0$$

fix: coalescing + reuse

I/O-BOUND

stream 10 GB from disk,
or network, into RAM



I/O ~100% busy - the wall

2x I/O rate -> time halves

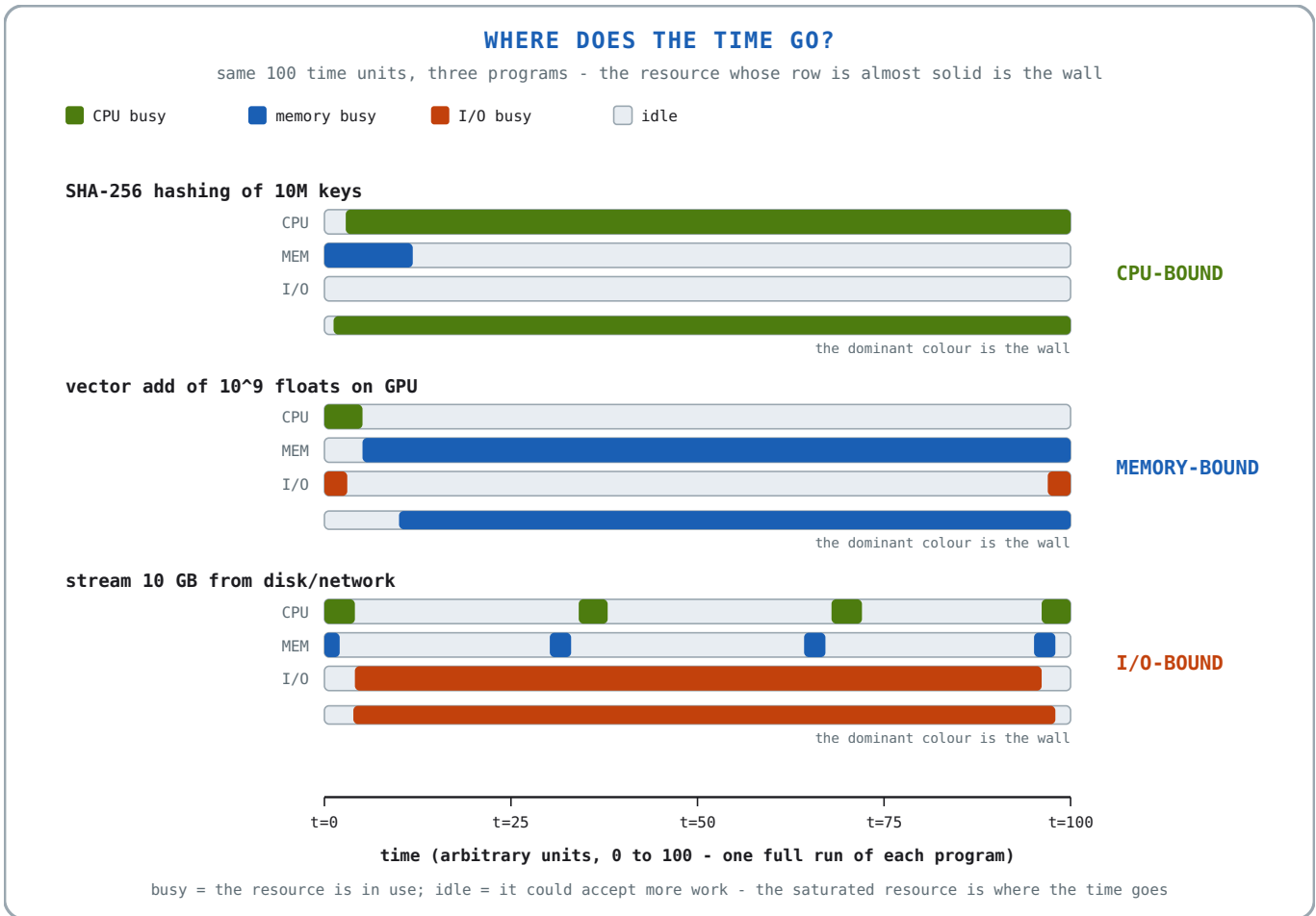
$$T_{\text{I/O}} / T \sim 1.0$$

fix: async I/O, overlap (Ch.6)

■ CPU busy ■ memory busy ■ I/O busy □ idle

diagnostic: double one resource's capacity - if total time halves, that resource is the bound

if doubling CPU, memory and I/O all change nothing, you are serial-bound (Amdahl's law, sec. 1.3) - parallelism cannot help



A practical test identifies the bound. Double the capacity of one resource and re-measure:

You double...	...and total time halves?	Then you are
CPU clock	✓	CPU-bound
Memory bandwidth	✓	Memory-bound
Disk / network / PCIe rate	✓	I/O-bound
All three	✗	Serial-bound (Amdahl, §1.3)

Each optimisation chapter in this book is an argument that a specific resource is the bottleneck. Coalescing (§2.7) targets memory-bound kernels. Register tiling (Chapter 9) targets compute-bound kernels. Streams and asynchronous transfers (Chapter 6) target I/O-bound kernels. The GPU-side term *compute-bound* corresponds to the CPU-side term *CPU-bound*: in both cases the processor is the limiting resource. Profiling (Chapter 16) exists to determine which resource is saturated before optimising.

1.9 The Cost of Synchronisation

Parallel work must occasionally rendezvous: threads must order their access to shared data or combine partial results. The programming primitives are introduced in Chapter 5, but their economics belong in this chapter.

Three costs attend any synchronisation point:

1. **Idle time.** Threads waiting at a barrier cannot use their execution units. A barrier converts available parallelism into a serial stall.
2. **Memory visibility.** For one thread to observe another thread's write, the write must become visible through the memory system. On a GPU this requires cache flushes or atomic operations and is not free.
3. **Load imbalance.** A barrier completes only when the slowest thread reaches it. If one thread does ten times the work of its neighbours, every barrier waits for that straggler.

The engineering consequence is to minimise the number of synchronisation points and keep the work between them balanced. The optimised reduction in Chapter 8 reduces the number of block-level barriers from $\log_2 N$ to a constant for this reason.

1.10 Vocabulary Summary

The terms defined in this chapter form the working vocabulary of the book:

Term	Definition	Defined
Latency	Time from start to completion of one operation	§1.1
Throughput	Operations completed per unit time	§1.1
Speedup $S(p)$	T_1/T_p	§1.2
Efficiency $E(p)$	$S(p)/p$	§1.2
Serial fraction f	The non-parallelisable fraction	§1.3
Strong scaling	Fixed problem, more units	§1.5
Weak scaling	Fixed per-unit problem, more units	§1.5
SIMT	Single instruction, multiple threads	§1.7
Arithmetic intensity I	FLOPs per byte moved	§1.8
Ridge point	P_{peak}/B	§1.8
Memory-bound	$I < I_{\text{ridge}}$, limited by bandwidth	§1.8
Compute-bound	$I > I_{\text{ridge}}$, limited by FLOPs	§1.8
CPU-bound	$T_{\text{CPU}}/T \approx 1$, limited by the processor	§1.8
I/O-bound	$T_{\text{IO}}/T \approx 1$, limited by disk/network/PCIe	§1.8

Amdahl and Gustafson in Practice

The two laws are not competing claims. They answer different questions under different assumptions.

Amdahl’s law fixes the problem size and the serial fraction, then asks how fast the same problem can run with more processors. It states that the serial part is an unremovable floor. Gustafson’s law allows the workload to grow with the hardware and asks how much work can finish in the same wall-clock time. Real users usually follow the Gustafson pattern: when a machine grows, they run larger models, higher-resolution images, or larger batches rather than rerunning the same small problem faster.

Making a fixed 1080p frame meet a 60 Hz deadline is an Amdahl regime: the work is fixed, and every microsecond of host overhead or synchronisation contributes to the serial fraction that caps speedup. Increasing batch size after adding GPUs is a Gustafson regime: the total work grows with the hardware, and the parallel fraction grows with it.

A speedup number is meaningful only with context. Any claim should state the problem size, the hardware, and whether the serial fraction was measured or assumed. Chapter 16 therefore requires recording the environment and methodology for every performance claim.

There is a second consequence of Amdahl's law that is often missed: the serial fraction is a property of the chosen *decomposition*, not an immutable property of the program. A reduction that appears serial in one formulation can become parallel with a tree. A host-side copy that appears to be overhead can be hidden with streams. The laws do not state what f is; they state what f costs. Reducing f is one of the main activities of GPU engineering and recurs throughout the book.

Common Pitfalls

- Quoting speedup without efficiency. A “64x speedup on 128 cores” is 50% efficiency; half the machine is idle.
- Forgetting that the serial fraction includes host-side overhead. Launch overhead, copies, and synchronisation are part of f , sometimes the dominant part.
- Treating a memory-bound kernel as compute-bound. If $I < I_{\text{ridge}}$, adding arithmetic throughput does not help; the memory system is the limit.
- Confusing strong and weak scaling when designing experiments. State whether the problem size is fixed or grows with the device count.

Check Your Understanding

Why is efficiency a more informative metric than speedup?

Efficiency divides speedup by the number of processing units. A vendor can report “10x speedup on 64 cores”, but efficiency is only $10/64 = 0.156$: 84% of the hardware contributes nothing beyond what the serial run already achieved. Efficiency exposes the waste that raw speedup hides.

A kernel has 2% serial time. What is the Amdahl ceiling?

The maximum speedup is $1/f = 1/0.02 = 50x$, no matter how many cores or GPUs are added. The 2% serial part alone takes 2% of the original time, so total time cannot go below that.

Vector add has intensity 0.08 FLOP/byte on a machine with ridge 40. Is it memory-bound or compute-bound?

Memory-bound. 0.08 is far below the ridge point, so the bandwidth diagonal caps performance at $I \times B$. The FP32 units are idle waiting for bytes; additional arithmetic throughput changes nothing.

Key Takeaways

- Parallelism buys throughput, not latency. The GPU hides latency by keeping many warps in flight.
- Speedup is $S(p) = T_1/T_p$; efficiency $S(p)/p$ is the informative metric.
- Amdahl's law: a serial fraction f caps fixed-workload speedup at $1/f$, regardless of unit count.
- Gustafson-Barsis: when the problem grows with the hardware, scaled speedup grows linearly.
- Arithmetic intensity $I = \text{FLOPs} / \text{bytes}$, compared with the ridge point P_{peak}/B , decides memory-bound vs compute-bound.
- A program is bound by the resource whose utilisation is near 100%: CPU, memory, or I/O. If doubling one resource halves the time, that resource is the wall.
- SIMT executes one instruction per warp; divergent control flow serialises divergent paths.

1.11 Exercises

1. A kernel has a serial fraction of $f = 0.02$. What is the maximum speedup Amdahl's law permits, regardless of hardware?
2. A vector add moves 4 bytes per element read and 4 bytes per element written, and performs 1 FLOP per element. Compute its arithmetic intensity.
3. On a machine with a ridge point of 40 FLOP/byte, is the vector add memory-bound or compute-bound? What is the maximum utilisation of peak FLOPs achievable?
4. Explain why a GPU designed for throughput executes a warp of threads in lockstep even though each thread has its own program counter.
5. A program downloads 10 GB from the network at 2 GB/s and compresses it on the CPU at 20 GB/s. Estimate the CPU utilisation. Is the program CPU-bound, memory-bound or I/O-bound? Which single change - a 2x faster CPU or a 2x faster network - speeds it up more?

Sources and Further Reading

- Gene M. Amdahl, “Validity of the Single Processor Approach to Achieving Large-Scale Computing Capabilities,” AFIPS Conference Proceedings, 1967. The paper behind Amdahl’s law.
- John L. Gustafson, “Reevaluating Amdahl’s Law,” Communications of the ACM 31(5), 1988. The paper behind Gustafson’s scaled-speedup law.
- Samuel Williams, Andrew Waterman, and David Patterson, “Roofline: An Insightful Visual Performance Model for Multicore Architectures,” Communications of the ACM 52(4), 2009. The original roofline paper.
- NVIDIA, *CUDA C++ Programming Guide*, “Compute Capabilities” appendix, for authoritative per-generation hardware limits: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- NVIDIA, *CUDA C++ Best Practices Guide*, for optimisation methodology: <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>

Chapter 2: GPU Hardware from First Principles

“The purpose of abstraction is not to be vague, but to create a new semantic level in which one can be absolutely precise.” — Edsger W. Dijkstra, “The Humble Programmer” (1972)

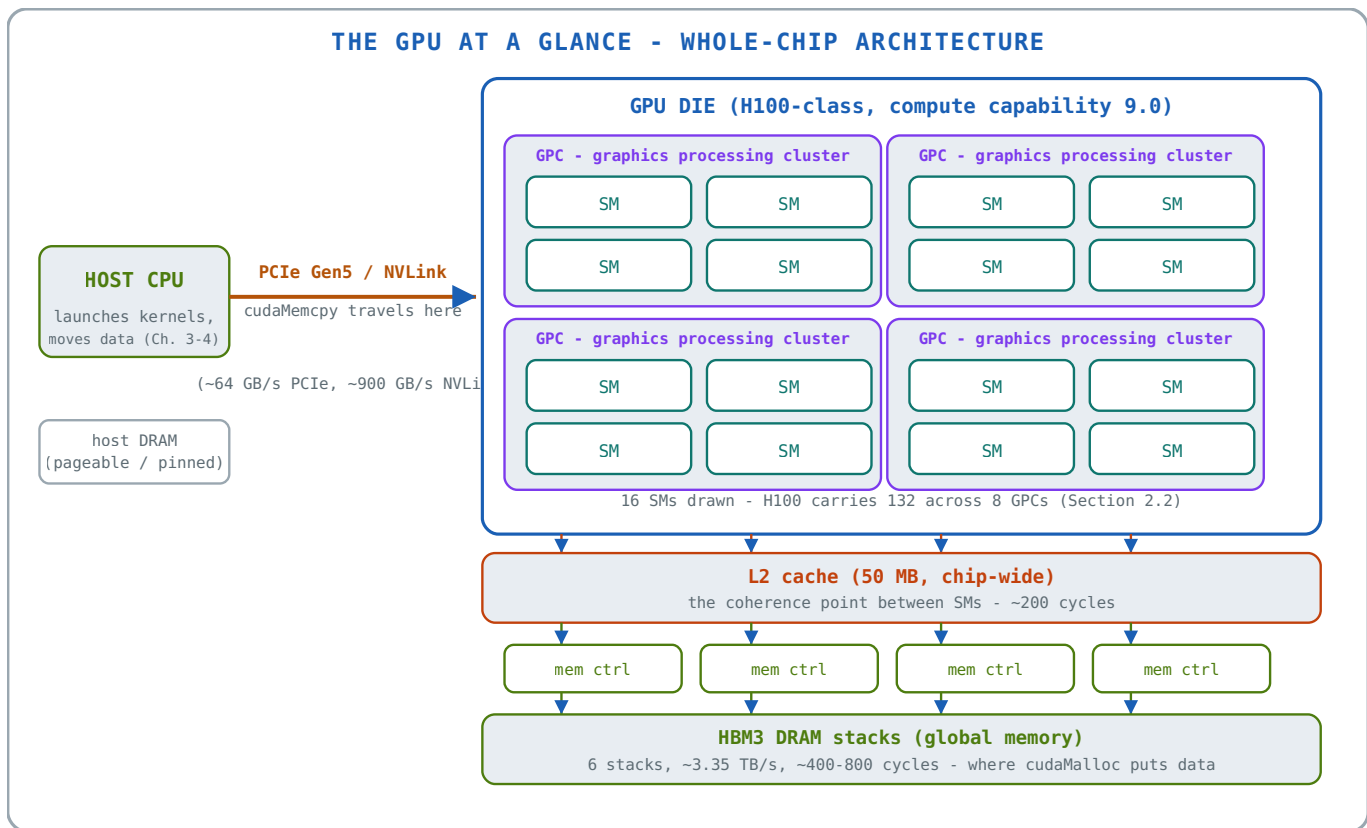
CUDA’s programming model exposes a grid of blocks, a block of threads, and a hierarchy of memories. The hardware underneath is less regular, and performance depends on its details. This chapter describes the machine: the *streaming multiprocessor*, the *warp*, the *memory hierarchy*, and the rules of *coalescing* and *occupancy*. Every term defined here is used in later chapters.

The running example is a modern NVIDIA GPU of the Hopper family (compute capability 9.0, such as the H100). Numbers differ between generations, but the structure does not.

2.1 The GPU at a Glance

A GPU consists of many identical compute clusters and a memory system. The H100 has, for example:

- **132 streaming multiprocessors (SMs)** - the compute clusters;
- **128 FP32 cores per SM** - the arithmetic units;
- **64 KB to 228 KB of shared memory per SM**, configurable;
- **64 K registers per SM**, partitioned among the threads;
- **50 MB of L2 cache**, shared by all SMs;
- **HBM3 DRAM** with a bandwidth of roughly **3.35 TB/s**.



The diagram shows the whole machine. The host CPU sits across a bus. The die is organised into *graphics processing clusters* (GPCs), each holding several SMs. Every SM funnels into one chip-wide L2; L2 feeds the memory controllers; the controllers drive the HBM3 stacks. This physical map underlies the reasoning in every later chapter.

The headline figures are design consequences rather than arbitrary specifications:

- **Many SMs.** A GPU is a throughput machine (Chapter 1, §1.1). Chip area is spent on many small compute clusters instead of a few large cores because parallelism, not single-thread speed, is the product. The 132 SMs of the H100 are what fit when each SM is deliberately small.
- **128 FP32 cores per SM.** An SM has four warp schedulers (§2.2), and each scheduler can issue one warp instruction per clock, covering 32 lanes. $128 = 4 \times 32$: one full warp per scheduler per clock without lane sharing. The number follows from the warp as the unit of execution.
- **64 K registers per SM.** Registers provide the working storage for resident warps. A deeper register file holds more warps and hides more latency (§2.9), at the cost of chip area and clock speed. The 64 K size is an engineering balance.
- **High DRAM bandwidth, high DRAM latency.** HBM3 stacks memory vertically beside the die on a silicon interposer and uses thousands of narrow channels; this is where

3.35 TB/s comes from. Every access still leaves the chip, which is why latency remains hundreds of cycles (§2.6) and why the on-chip memory hierarchy exists.

The FP32 throughput of such a chip is on the order of 60-70 TFLOP/s. With memory bandwidth of 3.35 TB/s, the roofline formula from Chapter 1 gives:

$$I_{\text{ridge}} = \frac{60 \times 10^{12} \text{ FLOP/s}}{3.35 \times 10^{12} \text{ B/s}} \approx 18 \text{ FLOP/byte}$$

A kernel below roughly 18 FLOP/byte is memory-bound on this machine. Most of the optimisation chapters explain kernels by reference to this ridge point.

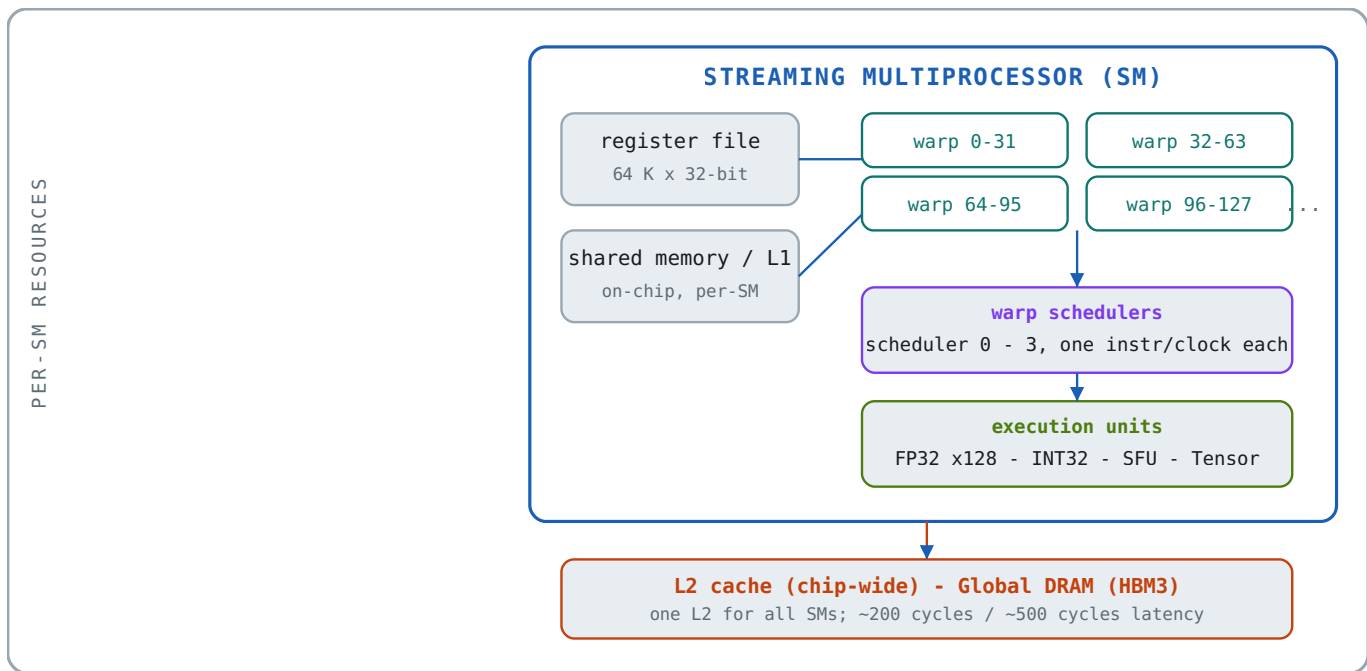
2.2 The Streaming Multiprocessor (SM)

The SM is the GPU's unit of compute. It is a small, heavily multithreaded processor, closer to a 128-lane vector machine than to a CPU core.

Each SM contains:

- **FP32 cores** (also called CUDA cores): single-precision floating-point units that can perform one FMA (fused multiply-add) per core per clock. An FMA computes $a \cdot b + c$ in one instruction; counting it as two FLOPs is why peak FLOP rates are reported as large numbers.
- **INT32 cores**: integer units. In modern architectures they share dispatch with the FP32 units but have their own register ports.
- **Tensor cores**: specialised matrix-multiply units for AI workloads. They form a separate pipeline and return in Chapter 11.
- **Special function units (SFUs)**: fast approximate implementations of transcendental functions such as `sin`, `cos`, `exp`, `log`, `1/x`, and `rsqrt`. Each SFU serves a warp at one result per clock per unit.
- **A register file** of 64 K 32-bit registers.
- **A shared memory / L1 cache** unit.
- **Four warp schedulers** (on modern SMs), each able to issue one instruction per clock to a warp.

An SM is not a multicore CPU. It does not run one instruction stream per core. It has a small number of warp schedulers, each feeding instructions to a warp of threads. Threads provide the data parallelism; the scheduler provides the control.



The diagram is a data-flow picture. Warps live in the register file, reach shared memory and arithmetic units through the schedulers, and access the rest of the chip through the L1/L2 path. The four schedulers are the control plane; the FP32/INT32/SFU/Tensor units are the data plane; shared memory and registers are the on-chip storage.

2.3 The Warp

Primitive - warp. A warp is a group of **32 consecutive threads** that are scheduled and executed together. The warp is the hardware's unit of execution, just as the thread is the programmer's unit of logic.

The hardware fetches one instruction and broadcasts it to 32 lanes. All 32 lanes execute it in the same clock, and each lane applies it to its own registers and its own data:

THE WARP: 32 THREADS, ONE INSTRUCTION, LOCKSTEP

the hardware fetches ONE instruction; all 32 lanes (threads) execute it together, each on its OWN data

warp scheduler picks one instruction

LD R4, [R2+8]

one instruction, broadcast to all 32 lanes



one instruction, fetched once - executed by 32 lanes in the SAME clock, each on its OWN data

this is SIMT (Chapter 1, 1.7): the programming convenience of many threads, the cost of SIMD

if lanes take different branches, the paths run one after another - that is divergence (Chapter 5)

The scheduler does not manage 32 threads as 32 independent items. It manages them as one warp. When it issues an instruction, every active lane executes it on whatever data that lane's thread holds.

The warp size of 32 is an architectural constant across every NVIDIA GPU to date. It results from three engineering constraints:

- **Instruction-cost amortisation.** Fetching and decoding an instruction costs the same whether it serves one thread or 32. A wider warp spreads that fixed cost over more useful work.
- **Power-of-two addressing.** Warp boundaries fall at 32, 64, 96, and so on, making thread-to-warp arithmetic (division and modulo by 32) cheap in hardware.
- **Memory-system granularity.** 32 threads x 4 bytes = 128 bytes, the size of one cache line (§2.7). A warp of consecutive threads can be satisfied by one memory transaction.

The cost of a wider warp is coarser divergence granularity: one thread taking a different branch makes the whole warp pay. The size 32 balances instruction amortisation against divergence waste, and NVIDIA has retained it across generations.

Execution semantics. The warp scheduler picks an instruction for a warp; the instruction is fetched once and issued to all 32 lanes at the same time. Each lane has its own registers, so lanes can hold different data, but they execute the same instruction at the same time. This is SIMT (Chapter 1, §1.7). A CPU runs one instruction stream per core; a GPU runs one instruction stream per 32 threads.

Divergence. If two threads in a warp take different branches of an `if`, the hardware cannot execute both paths simultaneously. It executes the `then` path with the other lanes masked off, then the `else` path, then reconverges. The two paths run serially, each consuming full-warp instruction slots. A 50/50 branch costs approximately twice the work of a uniform branch. Chapter 5 discusses this in detail.

One instruction, many loads. Because all 32 lanes share one instruction, a single memory load instruction issued to a warp performs 32 loads. How those 32 loads are serviced is the subject of §2.7 (coalescing).

Partial warps. A kernel launched with 1,000 threads creates 31 full warps ($32 \times 31 = 992$ threads) plus one partial warp of 8 threads. The remaining 24 lanes of that last warp are disabled but still occupy scheduling slots. They consume occupancy (§2.9) without doing work. Real kernels normally use block sizes that are multiples of 32 (Chapter 3).

2.4 Blocks, Grids, and the Hardware's View

CUDA's programming model (Chapter 3) organises threads as a **grid** of **thread blocks**, each block containing a group of **threads**. The hardware mapping is:

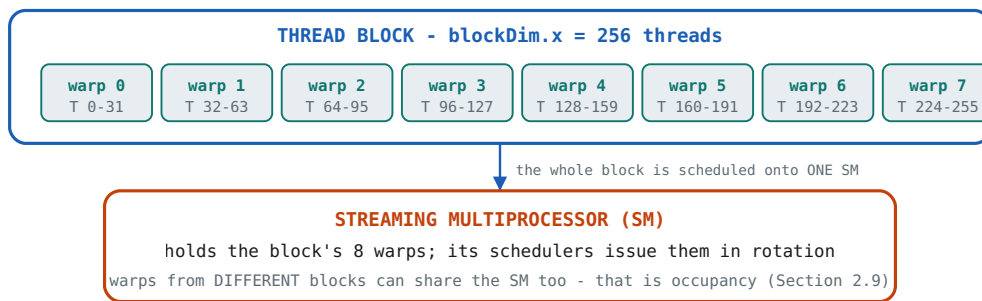
- A **thread block** is scheduled onto one SM as a unit. All threads of a block run on the same SM, which makes block-level shared memory and `__syncthreads()` possible.
- A block is partitioned into **warps** by consecutive thread IDs. Threads 0-31 form warp 0, threads 32-63 form warp 1, and so on. For a 2-D block, threads are linearised in x-major order (x varies fastest).
- An SM runs **many blocks concurrently**, time-slicing its warps. The number depends on occupancy (§2.9).

The block is the unit of cooperation: all threads in it can share memory and synchronise. The warp is the unit of execution: the hardware moves whole warps. The two levels are distinct:

- The programmer chooses the **block** size (**blockDim** in Chapter 3). Sizes are normally multiples of 32 so that no warp is partially empty.
- The hardware partitions blocks into **warps** invisibly. The programmer does not create warps and rarely addresses one directly. The warp exists so that the SM can schedule 32 threads at the cost of one.

A BLOCK OF 256 THREADS = 8 WARPS OF 32

the block is the programmer's unit of cooperation; the hardware slices it into warps - its own unit of execution

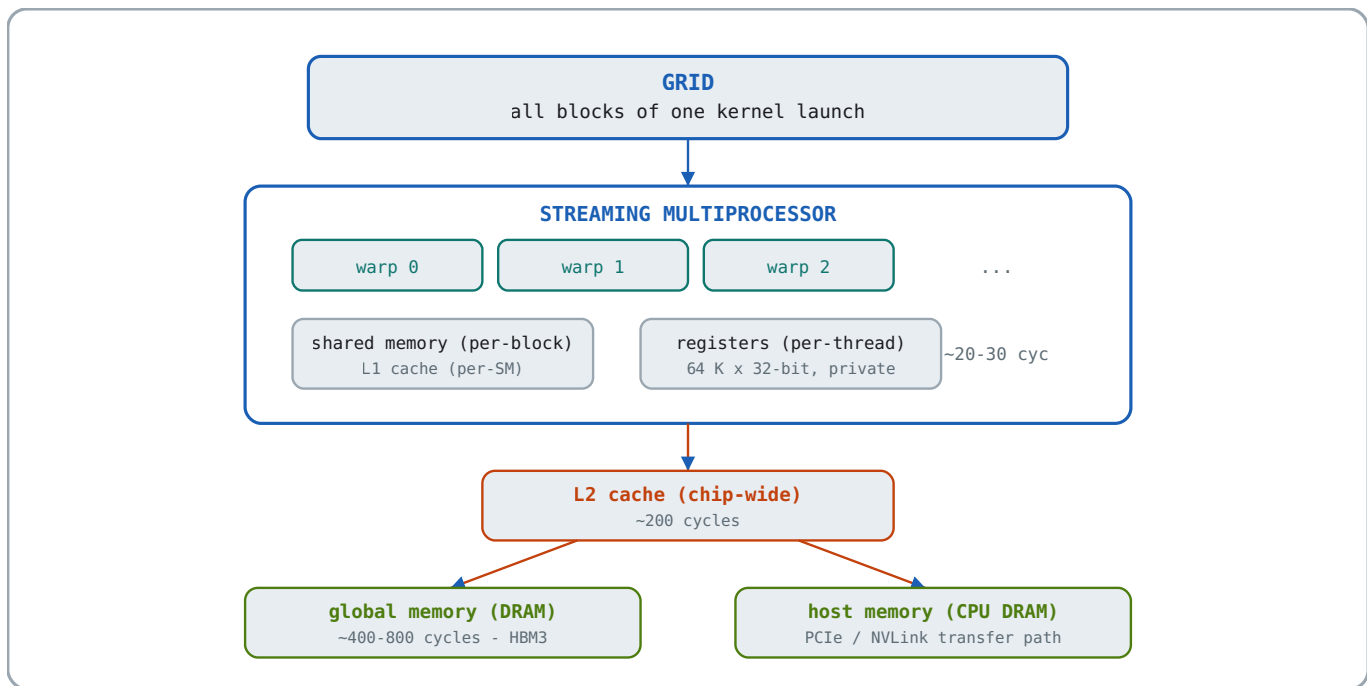


why this two-level design? Because the two levels answer different questions:

- The BLOCK (programmer's level): threads that share shared memory and `__syncthreads()` must be co-located on one SM, so the hardware places a whole block on one SM - it cannot be split.
- The WARP (hardware's level): the SM does not track 256 threads separately - it tracks 8 warps. One scheduler instruction covers 32 threads at once, which is far cheaper to manage.
- Threads are numbered x-fastest: in a 2-D block (`blockDim.x`, `blockDim.y`), thread (x, y) has linear ID $y * blockDim.x + x$, so warp 0 is always the first 32 threads in x-major order.
(The warp is the hardware's unit of execution; never confuse the two levels. A block of 32 threads = exactly 1 warp.)

2.5 The Memory Hierarchy

The GPU memory hierarchy is a hierarchy of distance and size:



From the SM outward, each level is larger and slower:

1. Registers. Private to a single thread; 32 bits wide; up to 255 per thread. A register has no address; instructions name it directly (**R0**, **R1**, ...). Register access is fast, but an SM has only 64 K registers shared by all resident threads. Register pressure directly limits occupancy (§2.9).

2. Shared memory. Private to a block; on-chip; configurable as part of the SM's 228 KB (H100) unified L1/shared resource. Access latency is roughly 20-30 cycles, compared with hundreds of cycles for global memory. Shared memory is the programmer-managed cache and the central subject of Chapter 7.

3. L1 cache. On-chip, per-SM, unified with shared memory. Global loads that hit L1 avoid the trip to DRAM. L1 cache lines are 128 bytes.

4. L2 cache. On-chip, shared by all SMs; 50 MB on the H100. It caches global, constant, and texture accesses. L2 is the coherence point between SMs: blocks on different SMs exchange data through L2 or through atomics (Chapter 5).

5. Global memory. The GPU's DRAM (HBM3) and the largest, slowest level. `cudaMalloc` allocations live here (Chapter 4). Bandwidth is enormous (3.35 TB/s) and latency is hundreds of cycles. Most optimisation work reduces global traffic.

6. Constant and texture memory. Two specialised read-only paths. Constant memory is a 64 KB cache that broadcasts a single value to all threads in a warp when they read the

same address, which makes it suitable for kernel parameters. Texture memory is a cached read-only path with hardware support for 2-D spatial locality and interpolation, used for images. Chapter 7 discusses both.

7. Local memory. Despite the name, “local” memory is global memory allocated per thread. It is used when a thread’s register demand exceeds the register file (a *register spill*). Local memory is slow and spills are avoided when possible. The compiler reports spills with `--ptxas-options=-v`.

2.6 The Latency Table

The following numbers are typical orders of magnitude for a modern GPU. They are teaching figures, not datasheet values:

Resource	Approximate latency	Notes
Register	~0 cycles	Operand to instruction
Shared memory	~20-30 cycles	On-chip, banked
L1 hit	~30 cycles	Per-SM
L2 hit	~200 cycles	Chip-wide
Global DRAM	~400-800 cycles	HBM3
Host memory (PCIe)	~1,000+ cycles + transfer time	Off-chip, CPU side

The table is a map of physical distance. Registers sit on the SM a few millimetres from the arithmetic units. Shared memory and L1 are on the same die. L2 spans the chip. DRAM is a separate package beside the die on an interposer. Host memory is across a bus and an operating-system boundary. Each step off the SM adds distance and arbitration, because more circuits compete for the same wires. The 20x gap between shared memory and DRAM is not a tuning detail; it is the difference between an on-chip wire and an off-chip trip, and it motivates the optimisation chapters.

A practical consequence is that one global memory access costs roughly 30 shared-memory accesses. An algorithm that reuses data in shared memory buys speed with engineering effort. Chapter 7 quantifies the trade.

2.7 Coalescing: How a Warp Reads Memory

Coalescing determines whether a GPU kernel uses its memory system efficiently or wastes most of its available bandwidth.

A warp executing a load consists of 32 threads issuing the same load instruction together. Each thread wants its own piece of data, for example a `float` occupying four bytes. The memory system receives 32 separate requests. Its cost depends on where those addresses lie.

If the 32 addresses are contiguous, the memory system can treat them as one block: it fetches one contiguous region and returns each thread its slice. The warp is served by one transaction, or two at most. If the addresses are scattered, for example every 32nd `float` with no reuse, the memory system cannot group them. Each request may become its own transaction, and the warp pays for many trips to memory for the same amount of useful data.

This property is **coalescing**. A warp whose accesses can be grouped into few transactions is *coalesced*; one whose accesses spread across many sectors is *uncoalesced*.

Primitive - coalescing. A warp load is serviced at sector granularity (32 bytes; four sectors per 128-byte cache line). The load is *coalesced* when the warp's addresses fall in as few sectors as possible and *uncoalesced* when they sprawl. To a first approximation, the cost of a warp load is the number of sectors it touches.

Sectors and transactions. The memory system delivers data in fixed-size chunks of 32-byte **sectors**, a quarter of a 128-byte cache line. The setup cost of a memory transaction (DRAM row activation, address decode, bus transfer) is paid per transaction, not per byte, so a half-empty sector costs almost as much as a full one. The relevant quantity for a warp load is the number of sectors touched:

- Thirty-two lanes reading 32 consecutive `floats` touch exactly 128 bytes: one cache line, one or two transactions.
- Thirty-two lanes reading every 32nd `float` touch 32 different lines: 32 transactions for the same number of useful bytes.

The hardware does not detect access patterns or rearrange requests. It counts the sectors a warp touches and bills accordingly. Global memory bandwidth is the scarcest resource on

a memory-bound kernel (Chapter 1’s roofline). With a stride of 32 floats between consecutive threads, each fetched 128-byte line delivers only one useful 4-byte word, wasting roughly 97% of the transferred bytes.

The layout habit that follows from this model is used throughout the book: arrange data so consecutive threads touch consecutive addresses. Row-major matrices in Chapter 9, thread-to-pixel mappings in Chapter 15, and padded shared-memory arrays in Chapter 7 all follow this rule. It derives from the hardware’s sector-based charging model rather than from a memorised guideline.

2.8 Shared Memory Banks

Shared memory is fast because it is **banked**. It is physically organised into 32 banks, each four bytes wide, that can be accessed simultaneously. A shared memory word maps to a bank by:

$$\text{bank} = \left\lfloor \frac{\text{address in bytes}}{4} \right\rfloor \bmod 32$$

When a warp accesses shared memory, the hardware services one access per bank per cycle. If two threads in the warp access the same bank, the hardware serialises those accesses. This is a **bank conflict**, and it costs extra cycles.

- Threads 0-31 reading consecutive words: all 32 banks are busy, one access, no conflict.
- Threads 0-31 reading words with stride 32: all 32 threads hit bank 0, a 32-way conflict that takes 32 cycles.
- Threads 0-31 reading the same word: the hardware broadcasts the value, one access, no conflict.

Formally, if a warp’s shared-memory accesses hit bank b exactly n_b times, the hardware must issue n_b accesses to that bank. The warp’s access completes in $\max_b n_b$ cycles. A conflict-free access has $\max_b n_b = 1$; the worst case is $\max_b n_b = 32$, when all threads hit one bank. Bank conflicts therefore multiply shared-memory access cost by a factor between 1 and 32 without moving any additional data.

Bank conflicts are a shared-memory phenomenon. Global memory has sectors and lines, not banks. Chapter 7 shows the standard fix for bank conflicts: padding.

2.9 Occupancy

Primitive - occupancy. Occupancy is the ratio of active warps on an SM to the maximum number of warps the SM can hold. An SM with 64 warp slots at 100% occupancy has 64 warps resident.

When a warp stalls on a global load, which takes hundreds of cycles, the scheduler switches to another resident warp (Chapter 1, §1.1). If occupancy is high, another warp is usually ready. If it is low, the SM may idle.

Occupancy is limited by the SM's finite resources:

- **Registers:** 64 K per SM. At 32 registers per thread, the SM can host 2,048 threads (64 K / 32). At 128 registers per thread, only 512 threads.
- **Threads per SM:** a hardware maximum, 2,048 on most modern SMs.
- **Threads per block and blocks per SM:** up to 1,024 threads per block and 32 blocks per SM (architecture-specific).
- **Shared memory:** 228 KB per SM on the H100. A block declaring 100 KB of shared memory leaves room for only two such blocks.

The occupancy of a launch configuration is the minimum over these limits. The occupancy calculator spreadsheet and the runtime function

`cudaOccupancyMaxActiveBlocksPerMultiprocessor` (Chapter 16) compute it.

Writing the arithmetic explicitly makes the trade visible. Let R_{SM} be registers per SM, T_{SM} the hardware thread limit per SM, B_{SM} the block limit per SM, and S_{SM} shared memory per SM. Let a block use T_B threads, R_T registers per thread, and S_B bytes of shared memory. The number of blocks that fit is:

$$B_{\max} = \min \left(\left\lfloor \frac{R_{SM}}{T_B \cdot R_T} \right\rfloor, \left\lfloor \frac{T_{SM}}{T_B} \right\rfloor, B_{SM}, \left\lfloor \frac{S_{SM}}{S_B} \right\rfloor \text{ if } S_B > 0 \right)$$

The number of resident threads is $B_{\max} \cdot T_B$, and occupancy is:

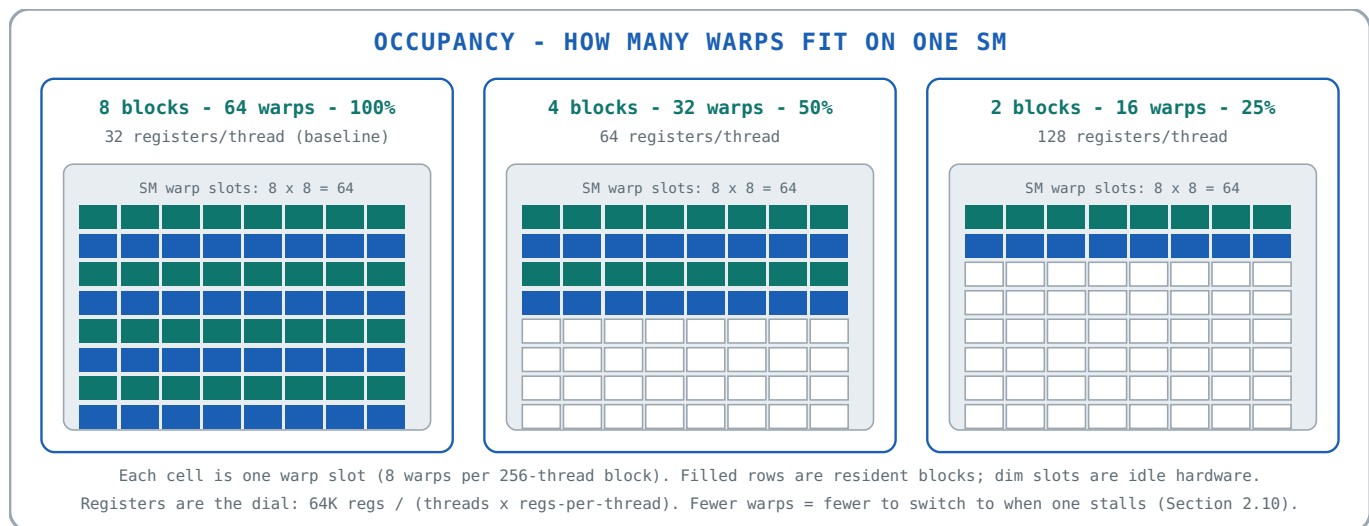
$$\text{occupancy} = \frac{B_{\max} \cdot T_B}{T_{SM}}$$

The four constraints are not alternatives. All four budgets are consumed simultaneously; the minimum is the binding one.

Worked calculation. Assume an SM with 64 K registers, 2,048 threads per SM, 32 blocks per SM, and 228 KB shared memory, and launch blocks of 256 threads (8 warps):

Constraint	Equation	Blocks allowed
Registers (32/thread)	$64\text{ K} / (256 \times 32)$	8 blocks
Threads per SM	$2,048 / 256$	8 blocks
Blocks per SM	hardware limit	32 blocks
Shared memory (0 bytes used)	no demand on the 228 KB budget	not a constraint

The minimum is 8 blocks, or 64 warps resident. If the SM holds at most 64 warps, this is 100% occupancy. With 64 registers per thread, the register term becomes $64\text{ K} / (256 \times 64) = 4$ blocks and occupancy drops to 50%. With 128 registers per thread it becomes 2 blocks, or 25% occupancy. Chapter 9's `__launch_bounds__` controls this trade by telling the compiler how many registers it may use per thread.



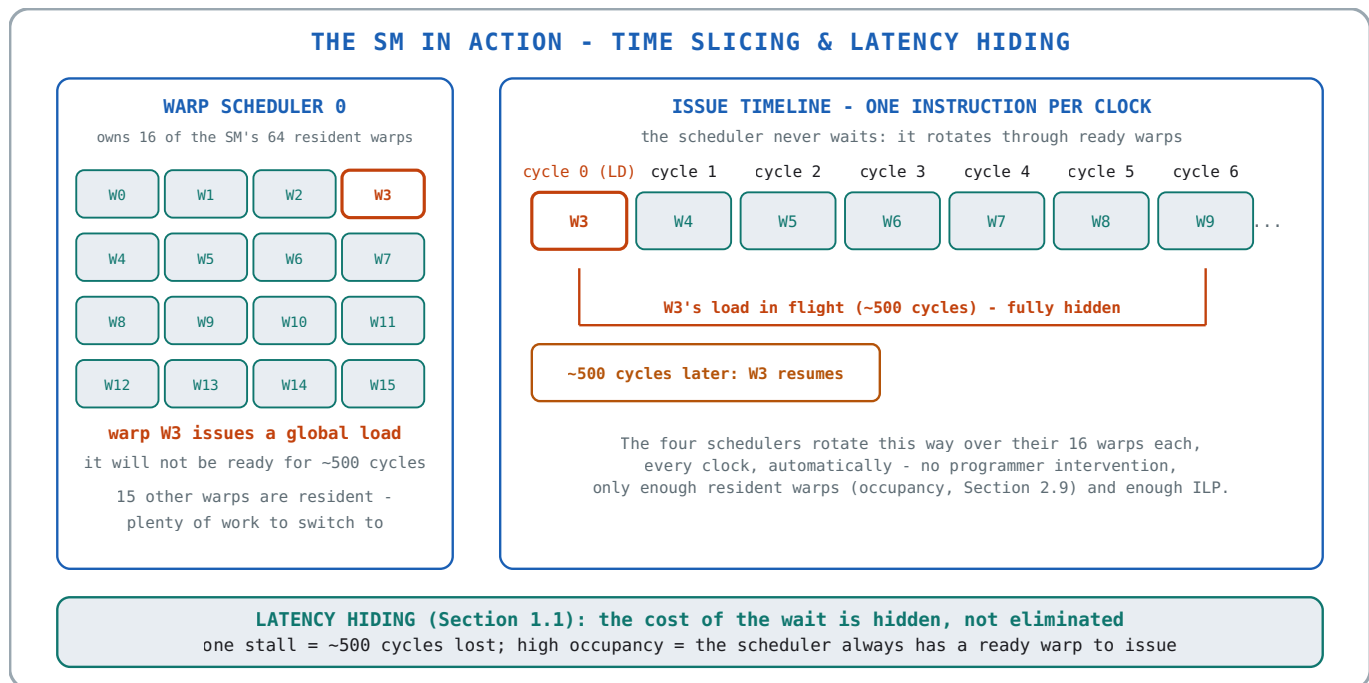
The diagram shows the same arithmetic. Each cell is one warp slot; each row is one block's eight warps; the dim cells are slots the scheduler cannot use because the register file is exhausted.

High occupancy is not always the right goal. A memory-bound kernel with long-latency global loads benefits from a large pool of warps because the pool gives the scheduler alternatives while any one warp waits. A compute-bound kernel whose operands already reside in registers rarely stalls, so a smaller pool may suffice. Raising occupancy by reducing register usage can force the compiler to spill registers to local memory, adding memory traffic and slowing the kernel. A kernel that needs a large shared-memory tile per block may deliberately use fewer blocks per SM, accepting lower occupancy in exchange for less global traffic and more data reuse.

The correct objective is not to maximise occupancy but to give the scheduler enough ready work without starving the kernel of registers or shared memory. `__launch_bounds__` makes this trade explicit at compile time.

2.10 The SM in Action: Time Slicing

Suppose an SM has 64 warp slots and a kernel is launched with blocks of 256 threads (8 warps per block). If the occupancy calculation permits 8 blocks per SM, the SM hosts 8 blocks = 64 warps = 100% occupancy.



Each of the four warp schedulers owns 16 warps. A scheduler can issue an instruction from one of its warps each clock. When warp 3 issues a global load, it will not be ready for roughly 500 cycles; the scheduler issues instructions from warps 4, 5, and others in the meantime. When warp 3's load returns, the scheduler resumes issuing for it.

No thread, driver, or explicit scheduling call rotates the warps. The hardware rotates among resident warps automatically. The programmer supplies enough warps (occupancy) and enough independent work per warp (instruction-level parallelism and coalesced accesses) to keep the rotation from stalling.

A first-order estimate makes this concrete. If a warp issues a global load and has nothing else ready for L cycles, a scheduler that issues at most one instruction per cycle needs at least L other ready warps to keep its execution units busy. With $L \approx 500$, that naive

estimate would require 500 warps per scheduler, far more than the hardware can host. Real warps do not stall on every instruction, and memory pipelining allows multiple loads to be outstanding. The estimate is therefore not a literal requirement; it explains why occupancy matters at all.

2.11 Architecture Generations

The structure described in this chapter is stable across NVIDIA GPUs, but the numbers are not. Compute capability (CC) encodes the generation: CC 7.x is Volta, CC 8.x is Ampere, CC 9.0 is Hopper, and CC 10.x is Blackwell. Each generation changes SM size, register file size, warp scheduling, tensor core capabilities, and shared memory capacity. Any performance claim should be read with the target compute capability in mind.

`deviceQuery`, a CUDA sample, reports the SM count, compute capability, register file size, shared memory per SM, and the launch limits from §2.9 for the installed GPU. Chapter 16 explains how to read that output.

Common Pitfalls

- Assuming higher occupancy is always faster. Register spills and reduced shared memory per block can make 50% occupancy beat 100%.
- Treating the warp as the programming unit. Programs address threads; the hardware executes warps. Divergence, coalescing, and shuffle operations all depend on warp boundaries.
- Ignoring bank conflicts. A `tile[32][32]` column access can be 32x slower than a row access. Padding each row by one float removes the conflict at negligible cost.
- Assuming the numbers in this chapter apply to every GPU. Check the compute capability and SM limits with `deviceQuery`.

Check Your Understanding

Must a block be resident on a single SM?

Yes. Block-scoped synchronisation (`__syncthreads`) and shared memory require all threads of a block to be co-located so they can communicate through a common on-chip resource. Splitting a block across SMs would make an efficient block-wide barrier impossible.

A kernel uses 64 registers per thread. How many 256-thread blocks fit in an SM with a 64 K register file?

Each block uses $256 \times 64 = 16,384$ registers. The register file holds $65,536 / 16,384 = 4$ blocks. If the thread and block limits allow more, registers cap occupancy at 4 blocks.

Why do 32 consecutive floats cost one 128-byte line, but 32 floats with stride 32 cost 32 lines?

A warp's 32 consecutive floats span 128 bytes, exactly one cache line. With stride 32, each thread's float lives in a different 128-byte region (for a large array width), so the hardware fetches 32 separate lines. The amount of useful data is the same; the memory traffic is roughly 32x larger.

Key Takeaways

- The warp of 32 threads is the hardware unit of execution, not the thread.
- The physical layout is GPCs of SMs above a chip-wide L2 above HBM3 DRAM, with the host across PCIe/NVLink.
- Blocks map to SMs; warps are consecutive thread IDs inside a block.
- Memory hierarchy: registers, shared memory, L1, L2, global DRAM - each level larger and slower (roughly 20-30 cycles for shared memory, 400-800 for DRAM).
- Coalescing: consecutive threads should read consecutive addresses; the hardware fetches 128-byte lines in 32-byte sectors.
- Shared memory has 32 banks of 4 bytes; a 32-way bank conflict costs 32 cycles; padding fixes it.
- Occupancy is the ratio of resident warps to the SM maximum; registers, threads, blocks, and shared memory each cap it.

2.12 Exercises

1. A kernel uses 64 registers per thread. How many threads can one SM host before the register file is exhausted, with 64 K registers per SM?
2. The same kernel now uses 128 registers per thread. What is the maximum occupancy given a hardware limit of 2,048 threads per SM?
3. A warp reads 32 consecutive `ints` (4 bytes each). How many 128-byte cache lines does the hardware fetch? How many would it fetch if the threads read every 32nd `int`?
4. Why must all threads of a block be resident on the same SM? Which synchronisation and memory primitive does this enable?
5. Estimate the number of cycles a warp-level shared-memory access takes when all 32 threads read the same 4-byte word, and when all 32 threads read words separated by 128 bytes.

Sources and Further Reading

- NVIDIA, *CUDA C++ Programming Guide*, “Hardware Implementation” and “Compute Capabilities”: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- NVIDIA, *H100 Tensor Core GPU Architecture* whitepaper and product page, for generation-specific numbers quoted in this chapter.
- David B. Kirk and Wen-mei W. Hwu, *Programming Massively Parallel Processors: A Hands-on Approach*, 3rd/4th ed., Morgan Kaufmann. Chapter-level treatment of GPU compute architecture, warp scheduling, memory coalescing, and occupancy.

Chapter 3: The CUDA Programming Model

Code companion: the complete, buildable code for this chapter lives in `code/ch03_vector_add/` in the repository.

This chapter introduces the CUDA programming model: how a function becomes a kernel, how a launch describes a grid of work, and how data moves between the CPU (the *host*) and the GPU (the *device*). The first complete program, a vector addition, is presented with line-by-line commentary.

3.1 Host and Device

CUDA programs are divided into two worlds:

- **Host** - the CPU and its memory. The host launches kernels and moves data.
- **Device** - the GPU and its memory (global memory, §2.5). The device executes kernels.

The two worlds do not share an address space. A pointer returned by `cudaMalloc` is a device pointer: dereferencing it on the host is undefined behaviour and, in practice, a crash. Data crosses the boundary explicitly with `cudaMemcpy`. This separation is the most common source of confusion for new CUDA programmers, and it remains the mental model even after Chapter 4 introduces pinned memory and unified memory.

Primitive - host. The CPU side of a CUDA program. **Primitive - device.** The GPU side of a CUDA program. **Primitive - kernel.** A function that runs on the device, launched by the host, and executed by many threads.

The separation corresponds to physical reality. The host is a CPU across a bus; the device is the GPU die with its GPCs of SMs, chip-wide L2, and DRAM. Every `cudaMemcpy` crosses that bus; every kernel launch delivers work to the SMs. The two address spaces are separate because the hardware is physically separate: different DRAM, different caches, different

execution units. The programming model does not hide that boundary, and most of the CUDA API’s “ceremony” follows from it.

3.2 Function Qualifiers

CUDA extends C++ with three function qualifiers:

- `__global__` - the kernel qualifier. The function runs on the device and is called from the host (or from the device in cooperative launch on later CUDA generations). A `__global__` function must return `void`. Its arguments are copied from host memory to the device before launch.
- `__device__` - the function runs on the device and can be called only from device code (from a kernel or from another `__device__` function).
- `__host__` - the default: a normal host function. It can be combined as `__host__ __device__` to compile one function for both sides, a common pattern in modern CUDA (Chapter 10).

A `__device__` function cannot call a `__host__` function because the device has no host runtime. A `__global__` function cannot be called recursively on most architectures and cannot take a variable number of arguments.

3.3 The Launch Configuration: Grids and Blocks

A kernel launch has the form:

```
myKernel<<<gridDim, blockDim>>>(args...);
```

The double-angle-bracket expression is the **execution configuration**. It describes the shape of the work. Both arguments have type `dim3`, a three-component vector type with fields `x`, `y`, and `z`, each an unsigned integer.

- **blockDim** is the number of threads per block, with one to three dimensions. The total number of threads per block is `blockDim.x * blockDim.y * blockDim.z` and must not exceed 1,024 on modern hardware.

- **gridDim** is the number of blocks in the grid, with one to three dimensions. The total number of threads in the kernel is the product of **gridDim** and **blockDim** over all dimensions.

The CUDA Programming Guide states the following launch limits for current architectures:

Dimension	Limit
Threads per block	1,024
Block size x	1,024
Block size y	1,024
Block size z	64
Grid size x	$2^{31} - 1$
Grid size y	65,535
Grid size z	65,535

These are architectural limits, not suggestions. A launch that violates them fails on the host before any kernel runs, so the error appears in a `cudaGetLastError()` check rather than in device code.

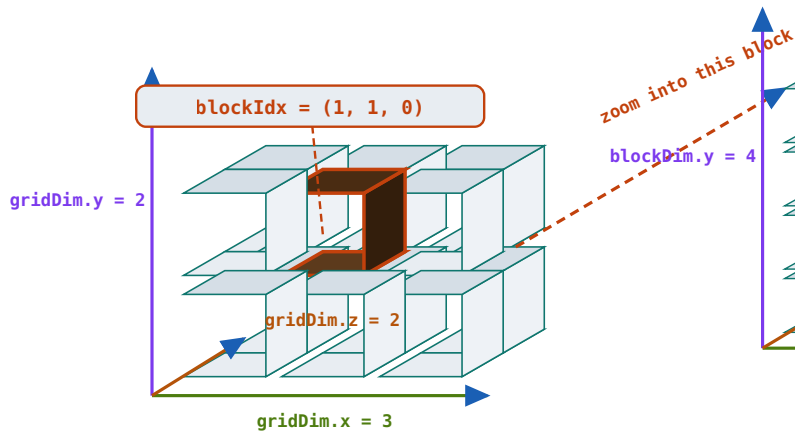
Two 3-D vectors therefore describe the entire launch. **gridDim** specifies the number of blocks along each axis; **blockDim** specifies the number of threads along each axis inside every block. Together they describe a 3-D grid of 3-D blocks.

THE LAUNCH HIERARCHY IN 3-D: `gridDim` AND `blockDim` ARE 3-D VECTORS

left: the GRID (array of blocks) | right: ONE BLOCK (array of threads) | total threads = `gridDim * blockDim` per axis

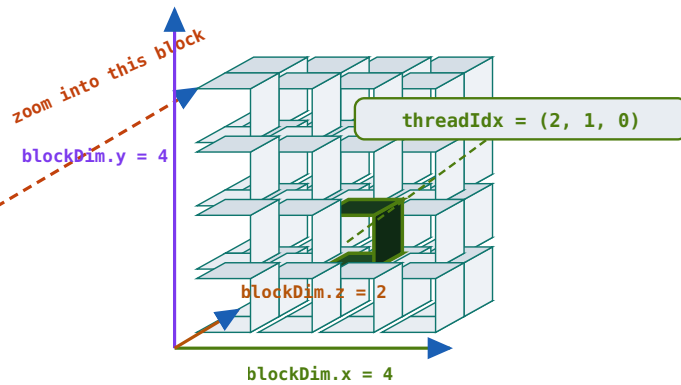
THE GRID

each cube = one thread block



THE BLOCK

each cube = one thread: $4 \times 4 \times 2 = 32 = \text{one warp}$



global position = blockIdx * blockDim + threadIdx (per axis)

$gx = blockIdx.x * blockDim.x + threadIdx.x$ $gy = blockIdx.y * blockDim.y + threadIdx.y$

$gz = blockIdx.z * blockDim.z + threadIdx.z$

example: `blockIdx = (1,1,0)`, `blockDim = (4,4,2)`, `threadIdx = (2,1,0)`

$gx = 1*4 + 2 = 6$ $gy = 1*4 + 1 = 5$ $gz = 0*2 + 0 = 0$
threads linearise x-fastest, then y, then z (Section 3.5)

The diagram has three parts:

1. **The grid** is a 3-D array of blocks. `gridDim = (3, 2, 2)` means 3 blocks along x, 2 along y, and 2 along z: 12 blocks. Each cube is a block with coordinates `blockIdx = (x, y, z)`.
2. **Each block** is itself a 3-D array of threads. `blockDim = (4, 4, 2)` means 4 threads along x, 4 along y, and 2 along z: $4 \times 4 \times 2 = 32$ threads, which is exactly one warp (§2.3). Each thread has coordinates `threadIdx = (x, y, z)` inside its block.
3. **The global position of a thread** is the block's position times the block's size plus the thread's position inside the block:

$$gx = blockIdx.x \times blockDim.x + threadIdx.x$$

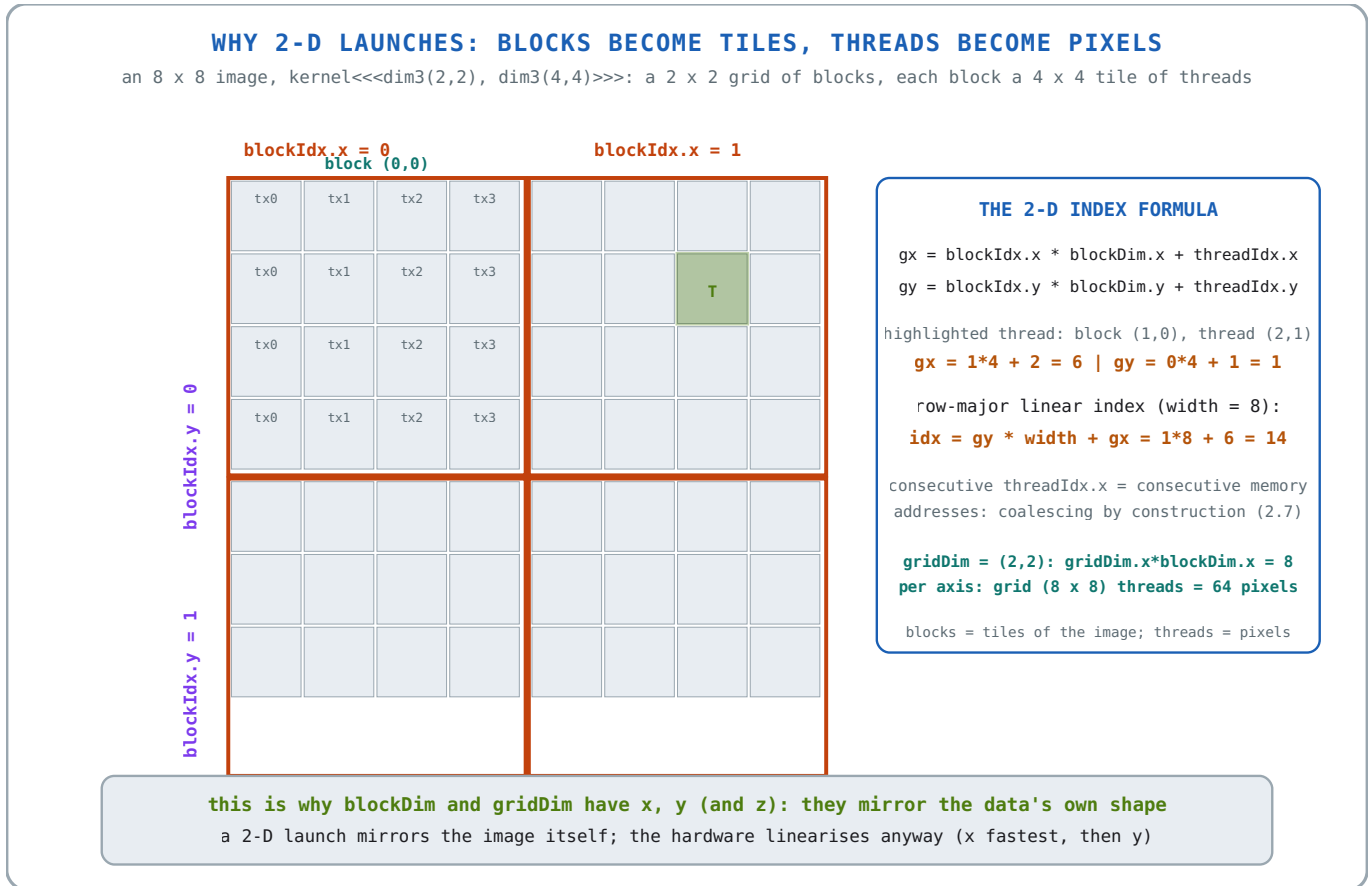
$$gy = blockIdx.y \times blockDim.y + threadIdx.y$$

$$gz = blockIdx.z \times blockDim.z + threadIdx.z$$

The 1-D formula used in §3.5, `blockIdx.x * blockDim.x + threadIdx.x`, is the x-component of this vector identity. The total number of threads is:

$$\text{gridDim.x} \cdot \text{gridDim.y} \cdot \text{gridDim.z} \cdot \text{blockDim.x} \cdot \text{blockDim.y} \cdot \text{blockDim.z}$$

Three dimensions exist because real data is often two- or three-dimensional (images, volumes, grids). A 2-D launch lets a kernel index an image as (x, y) instead of flattening it by hand:

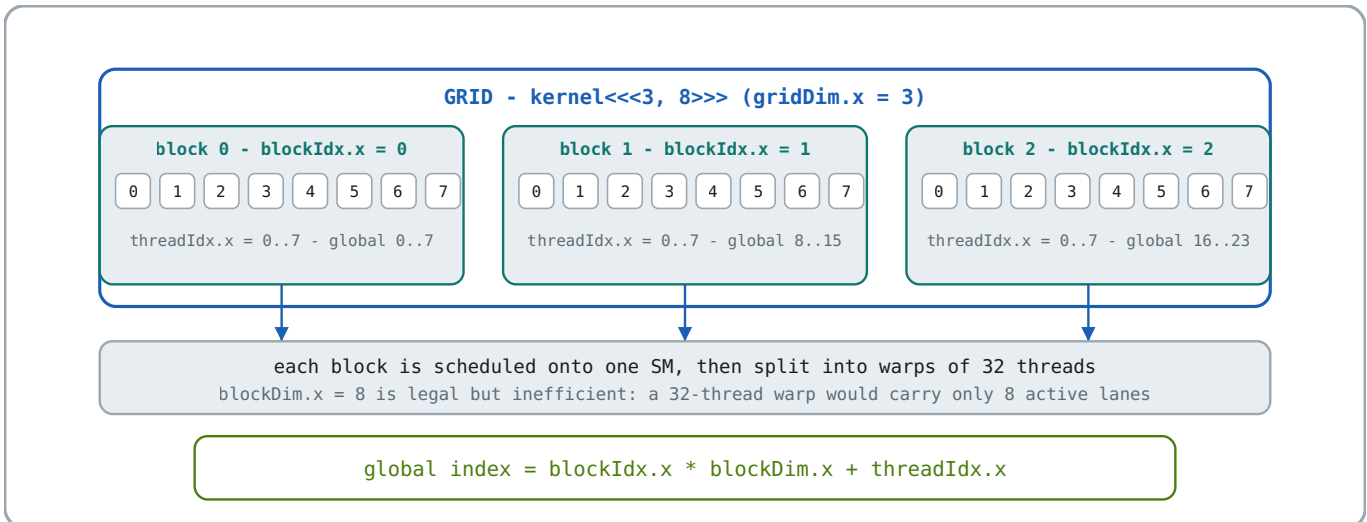


The image example gives the working model: **blocks tile the data; threads fill each tile.** The hardware linearises thread IDs (x fastest, then y, then z), but the kernel thinks in the data's own shape.

The hardware view (from Chapter 2) is: blocks are assigned to SMs, each block is partitioned into warps of 32 consecutive threads, and warps execute in lockstep.

The launch is a declaration, not a loop. $\lll\text{grid}, \text{block}\ggg$ does not describe an order of execution. It states how much work exists and how it is shaped; the hardware decides which SM runs which block and when. The launch is therefore a contract with the machine: enough blocks to fill every SM, blocks small enough to fit SM resources (registers, shared memory, the 1,024-thread cap), and a shape that maps to the data's natural dimensions.

The following figure shows a small launch, $\text{kernel}\lll3, 8\ggg$ (3 blocks of 8 threads):



3.4 Built-in Variables

Inside a kernel, four read-only built-in variables describe the launch:

Variable	Type	Meaning
<code>threadIdx</code>	<code>dim3</code>	The thread's position within its block: <code>threadIdx.x</code> , <code>.y</code> , <code>.z</code>
<code>blockIdx</code>	<code>dim3</code>	The block's position within the grid: <code>blockIdx.x</code> , <code>.y</code> , <code>.z</code>
<code>blockDim</code>	<code>dim3</code>	Threads per block (the <code>blockDim</code> passed at launch)
<code>gridDim</code>	<code>dim3</code>	Blocks per grid (the <code>gridDim</code> passed at launch)

These are provided by the hardware, not declared by the program. `blockIdx` identifies the block, `threadIdx` identifies the thread within the block, and `blockDim`/`gridDim` give the sizes of the two containers. The global formula `blockIdx * blockDim + threadIdx` translates thread identity into a position in the data.

3.5 The Global Index Formula

The core indexing expression for a 1-D problem is:

```
// Global linear index of this thread, assuming a 1-D grid and 1-D blocks.
unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
```

The derivation is direct: thread `threadIdx.x` lives in block `blockIdx.x`. Each block contains `blockDim.x` threads, so block number `blockIdx.x` starts at `blockIdx.x * blockDim.x`. Adding the position inside the block gives the global position. A 1-D launch is the general 3-D launch with the y and z components equal to one. For a 2-D problem the formula composes per axis:

```
// 2-D indexing: x and y are independent linear indices in their dimension.
unsigned int ix = blockIdx.x * blockDim.x + threadIdx.x; // column
unsigned int iy = blockIdx.y * blockDim.y + threadIdx.y; // row
// Row-major flattening of a width x height image:
unsigned int idx = iy * width + ix;                      // linear memory
index
```

In row-major layout, consecutive `ix` values are consecutive in memory. Because consecutive threads have consecutive `threadIdx.x`, and therefore consecutive `ix`, this indexing scheme is coalesced by construction (§2.7).

Worked example. Launch `kernel<<<4, 256>>>` (4 blocks of 256 threads, covering 1,024 global indices). For the thread with `blockIdx.x = 2` and `threadIdx.x = 137`:

```
global index = blockIdx.x * blockDim.x + threadIdx.x
              = 2 * 256 + 137
              = 512 + 137
              = 649
```

That thread owns element 649. Block `blockIdx.x = 2` covers global indices 512..767. Its warp 0 contains threads 0..31, or global indices 512..543: 32 consecutive addresses, coalesced by construction. If the array has 900 elements rather than a multiple of 1,024, the threads owning indices 900..1,023 are masked by the `if (i < n)` guard in the kernel.

3.6 The First Kernel: Vector Addition

The first complete program adds two `float` arrays element-wise: `c = a + b` for arrays of length `n`.

3.6.1 The Kernel

```
// kernel.cu
// -----
// __global__ : this function runs on the DEVICE, launched from the host.
// Return type must be void. The argument is a device pointer to n floats.
// -----
__global__ void addVectors(const float* a, const float* b, float* c, int n)
{
    // --- Thread identity -----
    // blockIdx.x : index of this block within the grid (0-based).
    // blockDim.x : number of threads in this block (set at launch).
    // threadIdx.x: index of this thread within its block (0-based).
    // The product blockIdx.x * blockDim.x is the first global thread index
    // covered by this block; adding threadIdx.x gives the global index.
    const int i = blockIdx.x * blockDim.x + threadIdx.x;

    // --- Boundary guard -----
    // The grid may cover more threads than n because the host rounds the grid
    // size up. Threads whose index is >= n must do nothing. Without this
guard
    // the kernel would read and write past the end of the arrays.
    if (i < n)
    {
        // Each thread owns exactly one output element, so no two threads
write
        // the same address and no race is possible.
        c[i] = a[i] + b[i];
    }
}
```

3.6.2 The Host Code

```

#include <stdio>
#include <cuda_runtime.h>    // All CUDA runtime API declarations live here.

// -----
// Error-checking helper (see 3.8). Every CUDA call that can fail is routed
// through CHECK, which prints the file and line on failure and aborts.
// -----
#define CHECK(call) \
    do { \
        const cudaError_t err = (call); \
        if (err != cudaSuccess) { \
            std::fprintf(stderr, "CUDA error at %s:%d: %s\n", \
                __FILE__, __LINE__, cudaGetErrorString(err)); \
            std::exit(EXIT_FAILURE); \
        } \
    } while (0)

int main()
{
    // --- Problem size -----
    // n is the number of elements; nBytes is the byte size of each array.
    // size_t is used because array sizes can exceed the range of int.
    const int    n        = 1 << 20;    // 1,048,576 elements (a power of two)
    const size_t nBytes = n * sizeof(float);

    // --- Host allocations (pageable memory, see Chapter 4) -----
    float* h_a = new float[n];    // host input A
    float* h_b = new float[n];    // host input B
    float* h_c = new float[n];    // host output C

    // Fill the inputs with a deterministic pattern so the result is
    verifiable.
    for (int i = 0; i < n; ++i) { h_a[i] = 1.0f * i; h_b[i] = 2.0f * i; }

    // --- Device allocations -----
    // cudaMalloc allocates in GLOBAL MEMORY on the device. The returned
    // pointers are valid only on the device (see 3.1).
    float* d_a = nullptr;    // device input A
    float* d_b = nullptr;    // device input B
    float* d_c = nullptr;    // device output C
    CHECK(cudaMalloc((void**)&d_a, nBytes));
    CHECK(cudaMalloc((void**)&d_b, nBytes));
    CHECK(cudaMalloc((void**)&d_c, nBytes));

    // --- Host -> device copy -----

```

```

// cudaMemcpy(dst, src, bytes, kind). The kind cudaMemcpyHostToDevice
// tells the runtime the direction of the copy (see 3.7).
CHECK(cudaMemcpy(d_a, h_a, nBytes, cudaMemcpyHostToDevice));
CHECK(cudaMemcpy(d_b, h_b, nBytes, cudaMemcpyHostToDevice));

// --- Launch configuration -----
// Block size: 256 threads per block. This is a multiple of the warp size
// (32), so every warp is full, and small enough that several blocks fit
// per SM (see occupancy, 2.9). Values of 128-512 are typical.
const int threadsPerBlock = 256;
// Grid size: ceil(n / threadsPerBlock). The + (threadsPerBlock - 1)
// rounds up so that the grid covers every element. Some threads will
// therefore have i >= n and hit the boundary guard in the kernel.
const int blocksPerGrid = (n + threadsPerBlock - 1) / threadsPerBlock;

// --- Launch -----
// Kernel launches are asynchronous: the host does not wait for the kernel
// to finish and control returns to the host immediately (Chapter 6).
addVectors<<<blocksPerGrid, threadsPerBlock>>>(d_a, d_b, d_c, n);

// Kernel launches do not report errors synchronously. Check the last
// error now; if the launch itself failed (bad configuration or pointer),
// this call catches it.
CHECK(cudaGetLastError());

// --- Synchronise
-----
// cudaDeviceSynchronize blocks the host until all device work issued so
// far has completed. This is required before copying the results back.
CHECK(cudaDeviceSynchronize());

// --- Device -> host copy
-----
CHECK(cudaMemcpy(h_c, d_c, nBytes, cudaMemcpyDeviceToHost));

// --- Verify
-----
// The expected value is h_c[i] == 3*i. Check all elements and report the
// largest error.
double maxErr = 0.0;
for (int i = 0; i < n; ++i)
{
    const double err = std::abs(static_cast<double>(h_c[i]) - 3.0 * i);
    if (err > maxErr) maxErr = err;
}

```

```

std::printf("max error = %g\n", maxErr);

// --- Cleanup
-----

delete[] h_a; delete[] h_b; delete[] h_c;
CHECK(cudaFree(d_a)); CHECK(cudaFree(d_b)); CHECK(cudaFree(d_c));
return 0;
}

```

3.6.3 Design Rationale

- **One thread per element.** The work is perfectly partitioned, no thread depends on another, and coalescing is automatic because the global index increases with `threadIdx.x`.
- **Rounded-up grid with a boundary guard.** Rounding the grid up to a multiple of the block size means the guard `if (i < n)` is required. The guard costs one comparison per thread and makes any problem size safe; computing an exact grid adds complexity without a performance benefit.
- **Power-of-two size in the example.** `n = 1 << 20` is used for simplicity. Production code uses arbitrary sizes and relies on the boundary guard.

3.7 Memory API Primitives

The runtime API functions used above appear throughout the book:

Function	Behaviour
<code>cudaMalloc(void** p, size_t bytes)</code>	Allocate <code>bytes</code> of device global memory and store the device pointer in <code>*p</code> . Returns <code>cudaSuccess</code> or an error code.
<code>cudaFree(void* p)</code>	Free a device allocation made by <code>cudaMalloc</code> .
<code>cudaMemcpy(dst, src, bytes, kind)</code>	Copy <code>bytes</code> between host and device. <code>kind</code> is one of <code>cudaMemcpyHostToDevice</code> , <code>cudaMemcpyDeviceToHost</code> , <code>cudaMemcpyDeviceToDevice</code> , or <code>cudaMemcpyHostToHost</code> . The copy is synchronous and completes before the call returns.
<code>cudaGetLastError()</code>	Return and clear the last asynchronous error recorded for the calling thread.
<code>cudaGetErrorString(err)</code>	Return human-readable text for a <code>cudaError_t</code> .
<code>cudaDeviceSynchronize()</code>	Block the host until all preceding device work completes.

`cudaMalloc` takes `void**` because it is a C-style output-parameter function: it must write a pointer into the caller's variable. In C++, an allocation function would return a pointer; CUDA's C heritage writes through a pointer-to-pointer. The explicit `(void**)` cast is required because C++ does not allow an implicit conversion from `float**` to `void**`; only `T*` to `void*` is implicit.

`cudaMemcpy` needs a direction argument because a host pointer and a device pointer cannot be distinguished by address alone. On some platforms the two address ranges overlap numerically. The direction flag removes the ambiguity.

3.8 Error Handling

CUDA runtime functions return a `cudaError_t`, an enum in which `cudaSuccess` is 0 and every other value is an error code. There are two failure modes:

1. **Synchronous errors** are detected by the call itself, such as an invalid argument or an illegal `cudaMemcpy` kind. The call returns the error code.
2. **Asynchronous errors** are detected after the call, such as an invalid kernel launch or an illegal memory access inside a kernel. The launch returns `cudaSuccess`; the error surfaces on the next CUDA API call from the same thread. This is why `cudaGetLastError()` is called immediately after the launch.

The `CHECK` macro reports the offending source line through `cudaGetErrorString`. Production code should do something more graceful than `std::exit`, but the discipline of checking every call is not optional. An unchecked CUDA error can produce a silently wrong answer or a corrupt image.

3.9 Compilation and the Build Pipeline

CUDA source files use the `.cu` extension and are compiled by `nvcc`, NVIDIA's compiler driver. The pipeline has two phases:

1. **Host pass.** `nvcc` separates the host code, compiles it with the host C++ compiler (`g++` or `clang++`), and replaces each kernel launch (`kernel<<<...>>>`) with runtime-API calls that package the arguments and launch the kernel.

2. **Device pass.** `nvcc` compiles the `__global__` and `__device__` functions to **PTX** (Parallel Thread Execution), NVIDIA's portable virtual instruction set, and then to **SASS**, the machine code of the target GPU, via `ptxas`.

```
# Compile for a specific architecture. This book's portable default is
# compute_60 (Pascal-class PTX): the driver JIT-compiles it to any CUDA 12.x
# GPU, from T4 and P100 to A100, Jetson Orin, and H100.
nvcc -arch=compute_60 kernel.cu -o kernel
# Native SASS alternatives (faster startup, less portable):
#   Jetson Orin : nvcc -arch=sm_87 kernel.cu -o kernel
#   A100        : nvcc -arch=sm_80 kernel.cu -o kernel
#   H100        : nvcc -arch=sm_90 kernel.cu -o kernel
```

Primitive - PTX. The intermediate virtual ISA (Chapter 12 covers it in detail). PTX is portable across GPU generations and is translated to SASS by the driver at load time when no SASS is embedded. **Primitive - SASS.** The GPU's machine code, tied to a specific compute capability.

`nvcc` can compile `.cu` files on a machine without an NVIDIA GPU; the resulting binary will not run there. Every `.cu` file in this book can be compiled with `nvcc -arch=compute_60 -o bin src.cu` and run on any CUDA 12.x GPU (Pascal or newer) through the driver's JIT.

3.10 Summary

- The launch configuration is a declaration of parallelism, not a loop. The hardware schedules the grid onto SMs and the blocks onto warp slots.
- `threadIdx`, `blockIdx`, `blockDim`, and `gridDim` are hardware-provided primitives. The global index formula `blockIdx.x * blockDim.x + threadIdx.x` maps thread identity to data address.
- Host and device have separate address spaces. Transfers are explicit (`cudaMemcpy`) and allocations are explicit (`cudaMalloc/cudaFree`).
- Check every CUDA call. Kernel launches are asynchronous, so `cudaGetLastError()` after the launch and `cudaDeviceSynchronize()` before reading results are both required.

Launch Semantics: Declaration, Not Order

The conceptual shift in CUDA is to read launch syntax as a declaration of work rather than a loop. A CPU `for` loop specifies an order of execution. A CUDA launch specifies how much work exists, shaped in a particular way, and leaves the mapping to blocks, SMs, and issue order to the hardware.

This has practical consequences. Because the hardware may schedule blocks in any order, a kernel must not depend on block order for correctness. Two blocks that need to exchange data must do so through explicit mechanisms: atomics, separate kernel launches, or cooperative groups. Because the same launch can run on a GPU with 10 or 100 SMs, a kernel must not assume a particular SM count. Well-written CUDA kernels are portable across the NVIDIA product line without source changes for this reason.

The boundary guard is a second consequence of the same design. Rounded-up grids are the standard way to handle problem sizes that are not multiples of the block size. The guard `if (i < n)` turns extra threads into no-ops. Its performance cost is small: at most one warp in the last block diverges. It prevents out-of-bounds accesses that could otherwise corrupt adjacent memory and produce silently wrong results.

Common Pitfalls

- Using `int` for sizes that can exceed 2 billion bytes or indices. `nBytes` should be `size_t`; grid and block dimensions are unsigned and have hardware limits.
- Launching a grid that does not cover all elements and omitting the boundary guard. Rounding up plus `if (i < n)` is the safe pattern.
- Forgetting that the launch is asynchronous. Checking errors immediately after the launch is necessary but not sufficient; the host must synchronise before reading device results.
- Assuming blocks execute in order. Block scheduling order is not defined and must never affect correctness.

Check Your Understanding

Why must a `__global__` function return void?

A kernel is launched for thousands of threads and has no single caller to receive a return value. Its output consists of the memory writes it performs. Results are communicated through output buffers rather than return values.

For $n = 1,000,000$ and 256 threads per block, how many blocks are launched and how many threads are masked?

$\text{blocks} = \text{ceil}(1,000,000 / 256) = 3,907$. Total threads = $3,907 \times 256 = 1,000,192$, so 192 threads in the last block are masked by `if (i < n)`.

What does `cudaGetLastError()` catch that `cudaDeviceSynchronize()` does not?

`cudaGetLastError()` returns launch-configuration errors recorded asynchronously (for example, an invalid block size or kernel pointer) before the host synchronises.

`cudaDeviceSynchronize()` waits for completion and surfaces execution errors such as illegal memory accesses. Both are needed.

Key Takeaways

- Host and device have separate address spaces; every transfer is an explicit `cudaMemcpy`.
- Function qualifiers: **global** (kernel called from host), **device** (device only), **host** (host).
- The launch configuration `<<<grid, block>>>` is a declaration of parallelism, not a loop.
- `threadIdx`, `blockIdx`, `blockDim`, and `gridDim` are hardware-provided primitives; the global index is `blockIdx.x * blockDim.x + threadIdx.x`.
- Boundary guards make rounded-up grids safe; they diverge only in the last partial block.
- Check every CUDA call: `cudaGetLastError()` after the launch, `cudaDeviceSynchronize()` before copying results back.

3.11 Exercises

1. Write the 2-D global index formula for a `width x height` image and show that consecutive threads in a warp read consecutive memory addresses when the block covers a contiguous row segment.

2. Why must a `__global__` function return `void`? What would a return value mean for 1,000,000 threads?
3. Compute `blocksPerGrid` for `n = 1,000,000` and `threadsPerBlock = 256`. How many threads in the last block are masked by the boundary guard?
4. What happens if `cudaMemcpy` is called with `cudaMemcpyHostToDevice` but a device pointer is passed as the source? Do not try it on a machine you care about.

Sources and Further Reading

- NVIDIA, *CUDA C++ Programming Guide*, “Programming Model” chapter: thread hierarchy, memory hierarchy, heterogeneous programming, and kernel launch syntax: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- NVIDIA, *CUDA C++ Best Practices Guide*, for host-device transfer and launch-configuration guidance: <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>
- NVIDIA, `deviceQuery` CUDA sample, for reading a GPU’s compute capability and resource limits.

Chapter 4: Memory Management & Data Movement

Chapter 3 moved data with `cudaMalloc` and `cudaMemcpy` without describing how the transfer happens. This chapter covers the mechanism. The GPU's data path is a pipeline with three distinct actors: host DRAM, the transfer bus (PCIe or NVLink), and device DRAM. The performance of a real application is often dominated by the slowest stage of that pipeline. The chapter covers the memory kinds a CUDA program can allocate, the copy primitives that move data, and the unified memory model in which the host and device share a virtual address space.

4.1 The Transfer Pipeline

A `cudaMemcpy` from host to device executes in stages:

1. The CPU reads the data from **pageable host memory**, the ordinary `malloc/new` kind used in Chapter 3.
2. The runtime copies it through a staging area. The PCIe/NVLink controller cannot DMA directly from a pageable page, because the operating system may swap that page out at any moment. The runtime therefore copies host data to a **pinned staging buffer**, then to the device. This adds an extra copy and an extra latency.
3. The device writes the data into device global memory over the transfer bus.

Each stage has a bandwidth. Total transfer time is bounded by the slowest stage, and identifying that stage is the central problem of host-device data movement.

4.2 Pageable vs Pinned Host Memory

Primitive - pageable memory. Ordinary host memory allocated with `malloc/new`. The OS may move or swap the underlying pages at any time, so the GPU's DMA engine cannot access them directly. **Primitive - pinned (page-locked) memory.** Host

memory whose pages the OS has agreed not to swap. The DMA engine can access it directly. Pinned memory is allocated with `cudaMallocHost` or `cudaHostAlloc`.

Pinned memory provides two properties:

1. **Direct DMA.** The runtime skips the staging copy, so a host-device copy is one transfer rather than two.
2. **Asynchronous transfers.** `cudaMemcpyAsync` (Chapter 6) requires pinned host memory. A pageable pointer cannot be used asynchronously because the DMA engine would need to chase OS page tables.

Pinned memory is not swappable. A large pinned allocation reduces the OS's freedom and can degrade system performance. The usual policy is to pin buffers on the streaming path and leave everything else pageable.

```
// -----
// Pinned allocation. cudaMallocHost allocates host memory that the CUDA
// runtime has pinned. It must be freed with cudaFreeHost, not free/delete.
// -----
float* h_pinned = nullptr;
CHECK(cudaMallocHost((void**)&h_pinned, nBytes)); // pinned, DMA-able, non-
swappable

float* h_pageable = new float[n]; // ordinary heap memory, swappable

// ... use ...

CHECK(cudaFreeHost(h_pinned)); // correct deallocation for pinned
delete[] h_pageable; // correct deallocation for heap
```

The pinned pages carry bookkeeping in the CUDA runtime. Freeing them with `free()` or `delete` would corrupt the runtime's view of the allocation. `cudaMallocHost` must be paired with `cudaFreeHost`; `cudaHostAlloc` also uses `cudaFreeHost`. `cudaMalloc` is paired with `cudaFree`.

4.3 Transfer Bandwidth: The Numbers

As teaching figures for a PCIe Gen4 x16 link (about 25-32 GB/s effective) and a modern GPU (about 1 TB/s HBM for the RTX family and 3.35 TB/s for the H100):

Copy	Approximate effective bandwidth
Host pageable → device	6-8 GB/s (staging hop dominates)
Host pinned → device	20-25 GB/s (PCIe-limited)
Device → host pinned	20-25 GB/s
Device → device	1-3 TB/s (HBM)

For N bytes and effective bandwidth B , a transfer takes:

$$T = \frac{N}{B}$$

Copying 1 GB host-to-device takes roughly 40 ms from pinned memory and 140 ms from pageable memory at the bandwidths above. A kernel that processes that 1 GB may run for 1 ms. Transfer time can exceed computation time by two orders of magnitude, which is why streaming (Chapter 6) overlaps transfers with computation instead of serialising them.

4.4 Measuring Your Own Transfer Time

A transfer should be measured as an average over repeated copies so that one-time overheads amortise. The following host-only snippet (no CUDA kernel required) compares pinned and pageable copies. It uses `std::chrono` because CUDA events have not yet been introduced (Chapter 6).

```

#include <chrono>
#include <cstdio>
#include <cuda_runtime.h>

// Time (in milliseconds) one cudaMemcpy of 'bytes' bytes from host to device.
double timeHostToDeviceCopy(void* dst, const void* src, size_t bytes)
{
    const int reps = 100;    // repeat to amortise launch overhead
    // Warm up once so page tables and caches are not part of the timing.
    cudaMemcpy(dst, src, bytes, cudaMemcpyHostToDevice);

    const auto t0 = std::chrono::steady_clock::now();
    for (int r = 0; r < reps; ++r)
        cudaMemcpy(dst, src, bytes, cudaMemcpyHostToDevice);
    cudaDeviceSynchronize();    // ensure the last copy actually
    finished
    const auto t1 = std::chrono::steady_clock::now();

    const double ms = std::chrono::duration<double, std::milli>(t1 -
t0).count();
    return ms / reps;          // average per-copy time
}

```

The first copy touches the pages, populates TLB entries, and performs the pin-on-demand work for pageable memory. Excluding it gives a steady-state number, which describes sustained behaviour.

`cudaMemcpy` is synchronous in the sense that the data copy completes before the call returns, so the last timed copy has already finished. The `cudaDeviceSynchronize()` at the end is defensive: it also waits for any asynchronous kernel work issued earlier, keeping the measurement clean.

4.5 Unified Memory: `cudaMallocManaged`

Primitive - unified memory (UM). A single virtual address space shared by host and device. A pointer allocated with `cudaMallocManaged` can be dereferenced on the host and in kernels without explicit copies. The driver migrates pages on demand.

Unified memory changes the programming model: explicit `cudaMemcpy` calls are not required. The driver decides when data moves, and its decisions can carry hidden per-page costs.

```
// One allocation, usable on both sides:
float* m = nullptr;
CHECK(cudaMallocManaged((void**)&m, nBytes));

// Host fills it like an ordinary array:
for (int i = 0; i < n; ++i) m[i] = static_cast<float>(i);

// Kernel reads it directly - no copy issued:
addVectors<<<blocksPerGrid, threadsPerBlock>>>(m, m, m, n); // m = m + m
CHECK(cudaDeviceSynchronize()); // page faults migrate data on demand
```

The driver splits the allocation into pages. When the host touches a page, the page is placed in host memory. When a kernel touches a page that is not on the device, a device-side page fault migrates it from host memory over the bus. The first touch of each page pays this migration cost; later accesses are local.

Two functions make migration explicit:

```
// Hint the driver to migrate the range [m, m + nBytes) to the device now,
// so the kernel does not pay page faults during execution:
CHECK(cudaMemPrefetchAsync(m, nBytes, 0 /* device 0 */));

// When the host needs the results back, prefetch to the host
(cudaCpuDeviceId):
CHECK(cudaMemPrefetchAsync(m, nBytes, cudaCpuDeviceId));
```

`cudaMemPrefetchAsync` is the explicit form of what the driver otherwise does lazily. A kernel that page-faults through 1 GB of unified memory pays a fault per page, which can stall the kernel for milliseconds. Prefetching before the kernel moves the cost out of kernel time.

Unified memory is useful when explicit copies are inconvenient, such as data structures with complex pointer graphs, or when an allocation is too large for device memory and can be streamed through with `cudaMemAdvise`. Its cost is reduced deterministic control over data movement. A working rule is to understand `cudaMemcpy` first, use unified memory where it fits the access pattern, and measure both options.

4.6 Zero-Copy Host Memory

Primitive - zero-copy. Host memory mapped into the device address space. Kernels access it directly over the bus; no explicit copy occurs. Each access pays bus latency, so zero-copy is useful only for small or rarely re-read data.

```
float* h_mapped = nullptr;
// cudaHostAllocMapped: allocate pinned host memory AND map it into the device
// address space (zero-copy).
CHECK(cudaHostAlloc((void**)&h_mapped, nBytes, cudaHostAllocMapped));

// Obtain the device-side pointer for the same memory:
float* d_mapped = nullptr;
CHECK(cudaHostGetDevicePointer((void**)&d_mapped, h_mapped, 0 /* flags, must
be 0 */));

// d_mapped can now be passed to kernels; the kernel's reads and writes go
// directly over PCIe to host memory.
```

Zero-copy is useful in two cases: data so small that a copy costs more than the kernel, and data produced by a kernel that the host must observe immediately without a copy-back. It is not useful for large data that is re-read, because every access crosses the bus at full latency.

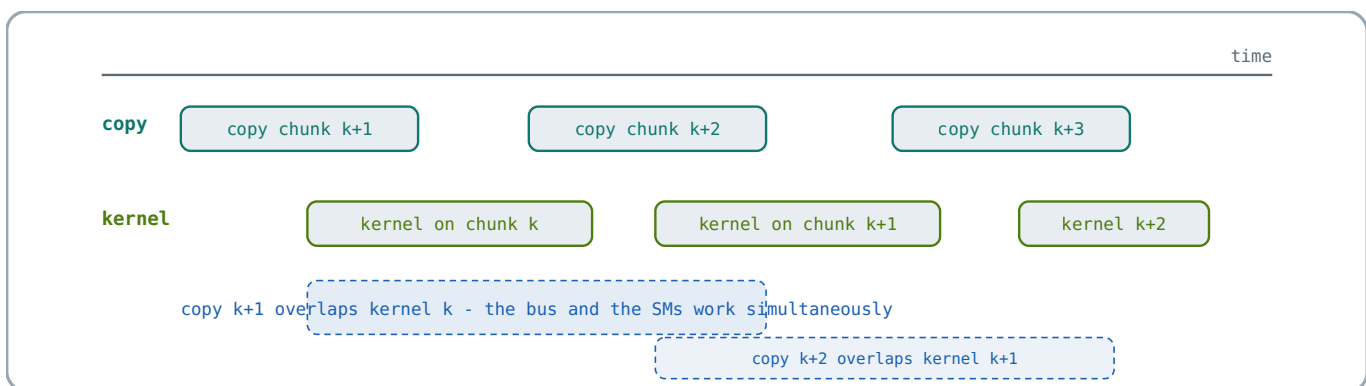
4.7 Choosing a Memory Kind

Need	Tool	Rationale
One-time setup copy	<code>cudaMemcpy</code> (pageable)	Simplicity; the staging hop occurs once.
Streaming or repeated copies	<code>cudaMallocHost</code> pinned + <code>cudaMemcpy</code> (or async, Ch. 6)	Direct DMA, no staging hop.
Pointer-heavy structures	<code>cudaMallocManaged</code> + <code>cudaMemPrefetchAsync</code>	Driver migrates whole graphs.
Tiny, frequently read host data	Zero-copy <code>cudaHostAllocMapped</code>	No explicit copy; bus latency is acceptable for small data.
Same-GPU scratch space	<code>cudaMalloc</code> device memory	Full HBM bandwidth, no bus.

A common failure mode is using `cudaMallocManaged` for every allocation because it is convenient, then observing low bandwidth because lazy migration converts a streaming copy into per-page faults. The memory kind is part of the algorithm design, not a setup detail.

4.8 Transfer Overlap: A Preview

A transfer and a kernel that operate on different data can run concurrently if the runtime can see that they are independent. The canonical shape is **double buffering**:



With two buffers, the copy for the next chunk overlaps computation on the current chunk. Transfer cost disappears from the critical path, provided the transfers are pinned and asynchronous. Chapter 6 builds this pipeline in full; the capstone (Chapter 15) uses it for images.

Memory Kinds and Their Cost Models

Memory allocation is not plumbing. Each memory kind embodies a different cost model, and the cost model determines whether a program runs at memory speed or at latency speed.

Pageable memory lives in ordinary OS pages that the kernel can swap or move at any time. The GPU's DMA engine cannot safely touch those pages because their physical addresses may change mid-transfer. The runtime therefore copies the data into a pinned staging buffer and then DMA's from there. That extra copy adds latency and consumes host memory bandwidth. Pinned memory (`cudaMallocHost`) locks the physical pages in place so the DMA engine can access them directly. This is not a micro-optimisation: it is the difference between one transfer and two.

Unified memory (`cudaMallocManaged`) presents one virtual address usable by both the CPU and GPU, and the driver migrates pages on demand. The cost hidden by the API is the page fault: every migration crosses the bus, and a kernel that touches a gigabyte of unified memory for the first time may pay a fault per page. `cudaMemPrefetchAsync` moves those migrations out of the kernel's critical path.

Zero-copy memory (`cudaHostAllocMapped`) maps host memory into the device address space. A kernel can read or write it directly over the bus with no explicit copy. Every access crosses the bus at PCIe latency, so zero-copy wins only for data that is small, accessed rarely, or produced by the kernel for immediate host consumption. For large, repeatedly read data, per-access bus latency exceeds the cost of one bulk copy.

The unifying principle is that data movement is not free, and memory kinds move data at different times: eagerly for copies, on demand for unified memory, and on every access for zero-copy. Choosing the wrong kind for an access pattern can change a kernel from bandwidth-bound to latency-bound.

Common Pitfalls

- Using pageable memory for frequent transfers. Pin what is streamed.
- Using unified memory for everything because it is convenient. Lazy migration turns a streaming copy into per-page faults; prefetch explicitly.

- Mismatching allocation and free: `cudaMalloc` -> `cudaFree`, `cudaMallocHost` -> `cudaFreeHost`, `cudaHostAlloc` -> `cudaFreeHost`. Mismatched frees corrupt runtime bookkeeping.
- Using zero-copy for large, repeatedly read data. Every access crosses the bus; zero-copy is fast only for small, rarely re-read data.

Check Your Understanding

Why can't `cudaMemcpyAsync` use pageable memory?

Asynchronous copies are performed by the DMA engine, which needs physical pages that will not move. Pageable pages can be swapped, so the runtime would have to stage through a pinned buffer synchronously, destroying the asynchrony. Pinned memory guarantees stable physical pages.

What cost does unified memory hide?

Page faults and migrations. When the GPU touches a page resident on the host, or the host touches a page resident on the device, the driver migrates the page over the bus. The first touch of each page pays this cost. `cudaMemPrefetchAsync` makes migrations explicit and removes them from kernel time.

When is zero-copy the right choice?

When the data is small enough that a copy costs more than direct bus access, or when the kernel produces data the host must see immediately. It is the wrong choice for large or repeatedly read data, because every access crosses the bus at full latency.

Key Takeaways

- Transfers can cost 100x more than the kernel that uses the data; data movement is part of the computation.
- Pinned memory (`cudaMallocHost`) enables direct DMA and asynchronous transfers; pageable memory goes through a staging copy.
- Unified memory (`cudaMallocManaged`) hides the copy but migrates pages lazily; prefetch explicitly with `cudaMemPrefetchAsync`.

- Zero-copy (`cudaHostAllocMapped`) suits small or rarely re-read data; device memory suits hot data.
- Match every allocation with its paired free (`cudaFree`, `cudaFreeHost`) and measure bandwidth before optimising.

4.9 Exercises

1. A 4 GB dataset is copied host-to-device from pageable and from pinned memory. Using the bandwidths in §4.3, by how many milliseconds is the pinned copy faster?
2. Why can `cudaMemcpyAsync` not be used with pageable host memory? Trace the sequence of events the DMA engine would need.
3. A kernel reads each element of a 1 GB unified-memory array exactly once. Where would you place `cudaMemPrefetchAsync`, and why?
4. Zero-copy memory is described as fast for small, rarely re-read data. Using the latency numbers of Chapter 2, explain what happens if a kernel re-reads the same 1 MB zero-copy region 1,000 times.

Chapter 5: Synchronisation, Atomics & Race Conditions

Code companion: the complete, buildable code for this chapter lives in `code/ch05_histogram/` in the repository.

Chapters 3 and 4 built kernels whose threads never communicated. Real kernels share data, and sharing data in parallel is where correctness is decided. This chapter covers three mechanisms provided by CUDA: **warp divergence** (the cost of independent control flow), **barriers** (`__syncthreads`), and **atomics** (hardware-arbitrated read-modify-write operations). It also defines the failure mode they address: the **race condition**.

5.1 The Race Condition

Primitive - race condition. Two or more threads access the same memory location, at least one access is a write, and the accesses are not ordered by any synchronisation mechanism. The result depends on the execution order chosen by the hardware, which the programmer cannot predict.

Races in CUDA are more severe than races on a CPU for two reasons:

1. **Scale.** A kernel has thousands to millions of threads. A race between any two of them can corrupt the result, and the corrupted result may appear only for one input in a thousand.
2. **Silent failure.** The kernel reports success while the corrupted value is stored. There is no exception, only a wrong answer.

The tools in this chapter order accesses. Every race fix installs an ordering between conflicting accesses.

Formally, two accesses *conflict* when they touch the same location and at least one is a write. The CUDA memory model defines a program's result only when conflicting accesses are ordered by a *happens-before* edge: a barrier, an atomic operation, a fence, a stream order, or an explicit device-wide synchronisation. Without such an edge, the hardware may execute the accesses in any order and different executions may produce different values. A race is therefore not a probabilistic glitch; it is an absence of ordering in the model.

5.2 Warp Divergence: Control Flow in SIMT

From Chapter 2, a warp executes one instruction at a time for all 32 lanes. Consider:

```
__global__ void conditionalAdd(const float* a, float* out, int n)
{
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n)
    {
        // Some threads take this path, others do not.
        out[i] = a[i] + 1.0f;
    }
}
```

If every thread in a warp has `i < n`, the warp takes the branch unanimously and there is no cost. If some threads have `i >= n` and others do not, the hardware must:

1. Execute the `then` path with the `i < n` lanes active and the other lanes masked;
2. Execute the `else` path with the remaining lanes active;
3. Rejoin the warp.

WARP DIVERGENCE - ONE INSTRUCTION, TWO PATHS, SERIALY

```

if (val[i] > 0)
y[i] = 2.0f * x[i]; // path A - taken by lanes 0-4
else
y[i] = 0.5f * x[i]; // path B - taken by lanes 5-7

```

a warp of 8 lanes drawn here; each lane represents 4 real lanes of a 32-thread warp



PASS 1 - then-path (A): lanes 0-4 execute, lanes 5-7 masked off



PASS 2 - else-path (B): lanes 5-7 execute, lanes 0-4 masked off



RECONVERGE - the warp resumes lockstep; both paths used 2 instruction slots

Divergent branch = 2 passes for one if; uniform branch = 1 pass; a 50/50 split doubles the cost (Section 2.3)

The two paths run serially. This is **warp divergence**, the price SIMT pays for per-thread control flow.

Divergence is a per-warp phenomenon. If a branch depends on `threadIdx.x % 2`, every warp in the grid has alternating lanes and pays the cost of both paths. If a branch depends on `blockIdx.x % 2`, whole warps agree and there is no cost. Data-dependent branches should be structured so that contiguous ranges of threads take the same path where possible.

The boundary guard `if (i < n)` from Chapter 3 diverges only in the last partial block of the grid, at most one warp per grid. Its cost is negligible.

5.3 `__syncthreads()`: The Block Barrier

Primitive - barrier. A point at which every thread in a group must arrive before any thread may proceed. `__syncthreads()` is a barrier over one thread block, provided by the hardware.

When a thread executes `__syncthreads()`, it waits until all threads of its block have reached that same call. The barrier orders memory within the block: a write by one thread before the barrier is visible to another thread after the barrier. Shared-memory algorithms (Chapter 7) rely on exactly this property.

```
__global__ void blockReduceDemo(const float* in, float* out, int n)
{
    __shared__ float s_partial[256];    // shared array, one slot per thread

    const int i = blockIdx.x * blockDim.x + threadIdx.x;

    // Phase 1: every thread reads its element and stashes it in shared
    // memory.
    // No ordering is needed yet because each thread writes its own slot.
    s_partial[threadIdx.x] = (i < n) ? in[i] : 0.0f;

    // Phase 2: before any thread reads another thread's slot, all writes must
    // be complete and visible. The barrier provides both the arrival
    // condition
    // and the visibility guarantee.
    __syncthreads();                    // required, see below

    // Phase 3: thread 0 can now safely read every slot.
    if (threadIdx.x == 0)
    {
        float sum = 0.0f;
        for (int k = 0; k < blockDim.x; ++k) sum += s_partial[k];
        out[blockIdx.x] = sum;
    }
}
```

Without the barrier, thread 0 could read `s_partial[5]` before thread 5 has written it. Thread 5's write may still be in the memory pipeline, and the read could return an undefined value. The barrier makes the phase structure valid.

`__syncthreads()` has two failure modes:

1. **Divergent barriers.** If some threads of a block reach a `__syncthreads()` while others do not, because they took a different branch, the barrier waits for threads that will never arrive. The block deadlocks. The hardware does not detect this; the kernel hangs until a watchdog timeout or `cudaDeviceReset`.

2. **Barriers in loops with variable trip counts.** The same deadlock occurs when threads execute the barrier different numbers of times. The barrier must be uniformly reachable: every thread executes it the same number of times.

```
// DEADLOCK: threads with even index skip the barrier, odd ones wait forever.
if (threadIdx.x % 2 == 0) { /* no barrier here */ }
__syncthreads();           // threads with even index never arrive
```

There is no cheap grid-wide barrier. Blocks on different SMs may not be resident at the same time, so the hardware cannot synchronise them without additional constraints. A grid-wide barrier exists through cooperative groups, but it requires a cooperative launch in which every block is resident simultaneously, which caps the grid size. For cross-block communication, use atomics (§5.5) or split the work into two kernel launches.

5.4 Visibility: Caches, `volatile`, and Fences

A barrier orders block-internal accesses. Cross-block and host-device visibility follow different rules because modern GPUs have caches.

- L1 caches are per-SM and are not coherent between SMs. A thread on SM 0 may read a stale value of a location written by SM 1 unless the access is ordered through L2, the chip-wide coherence point.
- The compiler may reorder or cache loads and stores in registers unless told otherwise.

Two tools address these rules:

Primitive - `volatile`. Tells the compiler that memory may change outside its knowledge. The compiler must not cache the value in a register and must emit the access each time it appears in the source.

Primitive - memory fence. An instruction that orders memory operations at a given scope. `__threadfence()` orders global-memory accesses for the device; `__threadfence_block()` for the block; `__threadfence_system()` for host and device. A

fence does not make other threads wait; it forces this thread's earlier writes to become visible before its later writes.

The following example shows a device-scope flag protected by a fence and volatile access:

```
// Shared state: a buffer and a "ready" flag, both in global memory.
__device__ float  g_buffer[1024];
__device__ int    g_ready = 0;

// Producer kernel: writes data, then sets the flag.
__global__ void producer()
{
    for (int i = threadIdx.x; i < 1024; i += blockDim.x)
        g_buffer[i] = static_cast<float>(i);

    // Make all preceding writes visible device-wide before the flag write.
    // Without the fence, another SM could observe g_ready == 1 while some
    // g_buffer writes are still pending in L1.
    __threadfence();

    if (threadIdx.x == 0)
        g_ready = 1;           // must be volatile; compiler cannot cache it
}

// Consumer kernel: polls the flag.
__global__ void consumer()
{
    while (volatileLoad(&g_ready) == 0) { /* spin */ }
    // The buffer writes are now guaranteed visible.
    float x = g_buffer[threadIdx.x];
}
```

`volatileLoad` denotes a read through a `volatile int*`. In production code, prefer the higher-level abstractions: atomics (§5.5) and cooperative groups. Hand-written fences are useful for understanding what those abstractions do and for rare low-level cases.

5.5 Atomics: Hardware-Arbitrated Read-Modify-Write

Primitive - atomic operation. A read-modify-write, such as read, add, and write, that the hardware guarantees to execute indivisibly with respect to other threads. Two `atomicAdd` calls on the same location cannot interleave; the result is exactly as if the two additions occurred in some serial order. The hardware chooses the order; no thread ever observes a torn value.

CUDA provides atomic functions for `int`, `unsigned int`, `unsigned long long`, `float` (for addition), and, on modern GPUs, `double`:

Function	Operation	Returns
<code>atomicAdd(addr, v)</code>	<code>*addr += v</code>	the <i>old</i> value
<code>atomicSub(addr, v)</code>	<code>*addr -= v</code>	the old value
<code>atomicExch(addr, v)</code>	<code>*addr = v</code>	the old value
<code>atomicCAS(addr, cmp, v)</code>	if <code>*addr == cmp</code> then <code>*addr = v</code>	the old value
<code>atomicMin(addr, v)</code>	<code>*addr = min(*addr, v)</code>	the old value
<code>atomicMax(addr, v)</code>	<code>*addr = max(*addr, v)</code>	the old value
<code>atomicAnd(addr, v)</code>	<code>*addr &= v</code>	the old value
<code>atomicOr(addr, v)</code>	<code>*addr = v</code>	the old value
<code>atomicXor(addr, v)</code>	<code>*addr ^= v</code>	the old value

Atoms operate on global and shared memory. The returned old value is central to lock-free algorithms: `atomicCAS` (compare-and-swap) is the primitive from which other synchronisation structures can be built.

5.5.1 Worked Example: Histogram

A histogram counts occurrences of values. If every thread increments counters in the same array, the increments must be atomic:

```

// Count how many elements fall into each of 256 bins.
// data : device array of unsigned char (0..255), n elements
// hist : device array of 256 ints, zero-initialised on the host
// bins : the bin count, 256
__global__ void histogram(const unsigned char* data, int* hist, int n)
{
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n)
    {
        const int bin = data[i];           // value is the bin index
        // The atomic serialises concurrent increments to the same bin.
        // Without it, two threads could read the same old value, add 1, and
        // both write old+1, losing one count.
        atomicAdd(&hist[bin], 1);
    }
}

```

Atomics on the same address from many threads serialise because hardware arbitration is a bottleneck. The standard fix is **privatisation** (Chapter 8): give each block its own histogram in shared memory, accumulate with shared atomics, and fold the private histograms into global memory at the end.

5.5.2 Worked Example: A Spinlock Built from `atomicCAS`

`atomicCAS` implements test-and-set, and test-and-set can implement a lock. The following example uses a lock in shared memory to protect a critical section within one block:

```

// A simple lock in shared memory. One mutex per block.
__global__ void lockedCriticalSection(float* g_data, int n)
{
    __shared__ int s_lock;    // 0 = unlocked, 1 = locked

    // Thread 0 initialises the lock.
    if (threadIdx.x == 0) s_lock = 0;
    __syncthreads();          // everyone must see the lock before using it

    // Each thread processes its own element. The lock protects the block-wide
    // critical section in this pedagogical example; it does not coordinate
    // between blocks. A lock that must span blocks belongs in global memory.
    for (int i = blockIdx.x * blockDim.x + threadIdx.x;
         i < n; i += blockDim.x * blockDim.x)
    {
        // Acquire: atomically swap 1 into the lock. If the old value was 0,
        // this thread acquired the lock. If it was 1, another thread holds
        it.
        while (atomicCAS(&s_lock, 0, 1) != 0) { /* spin */ }

        // Critical section.
        g_data[i] = g_data[i] * 2.0f + 1.0f;

        // Release: order the critical-section writes before clearing the
        lock.
        __threadfence_block();
        s_lock = 0;
    }
}

```

`atomicCAS(&s_lock, 0, 1)` means: if the lock is currently 0 (unlocked), set it to 1 and return the old value. If the returned value is 0, this thread acquired the lock; otherwise it retries. The spin loop is the cost of contention.

The fence before release is required so that the release store is not observed before the critical-section writes become visible.

Locks in GPU kernels are usually a poor choice because they serialise work on a machine designed for parallelism. The patterns that avoid locks (privatisation, partitioning, and lock-free atomics) are typically faster. This example exists to show how atomics can implement synchronisation, not as a recommended pattern.

5.6 Floating-Point Non-Determinism

There is a race that passes functional tests and still changes the answer: floating-point addition is not associative.

```
// (a + b) + c may differ from a + (b + c) in the last bits.
```

If a reduction accumulates partial sums in different orders across runs (because atomics or scheduling choose different orders), the results can differ in the last bits. For most applications this is acceptable. Applications that require bit-exact reproducibility, such as scientific publishing or distributed training checkpoints, need a **fixed reduction order**. The tree reduction in Chapter 8 is deterministic because its order is fixed; an `atomicAdd`-based reduction is not.

5.7 A Decision Procedure

When a kernel writes shared state, check the following:

1. **Who writes?** If more than one thread writes the same location, use atomics or restructure so each thread owns its locations.
2. **Who reads after whom?** If a thread reads another thread's write, place a barrier (`__syncthreads()`) between write and read, uniformly reachable by all threads of the block.
3. **Across blocks?** There is no block-wide barrier in a normal launch. Use atomics, separate kernels, or cooperative groups. Never spin on a non-atomic, non-volatile flag.
4. **Is the order deterministic?** If reproducibility matters, prefer fixed tree orders over atomic accumulation.

Race Fixes as Ordering

A data race can seem mysterious when it appears only on rare inputs or after a compiler update. The definition is mechanical, and so is the cure. Every synchronisation tool in this chapter orders a specific kind of access:

- `__syncthreads()` orders the memory accesses of a block relative to a barrier. Threads that write before the barrier are guaranteed to have their writes visible to threads that read after it.
- Atomics order read-modify-write cycles at one memory location. When two threads execute `atomicAdd` on the same address, the hardware chooses a serial order and applies both updates; no torn value is observed.
- Fences order one thread's own memory operations with respect to what other threads can observe. A fence does not make anyone wait; it guarantees that this thread's earlier writes become visible before its later writes.
- `volatile` prevents the compiler from caching a value in a register so that every access reaches memory. It does not make an operation atomic, but it is often necessary for flags polled by other threads.

Debugging a race then becomes a systematic procedure: identify the shared location, identify writers and readers, and ask what orders them. If the answer is nothing, the program contains a race even if it produces the right answer on the test inputs tried.

GPU races are worse than CPU races because of scale and silence. A race may involve any two of millions of threads, so the exposing interleaving may occur once in billions of executions. The GPU reports success while the corrupted value is stored, so the failure may pass unit tests. Compute Sanitizer's `racecheck` (Chapter 16) instruments memory accesses and reports unsynchronised read/write pairs directly.

Common Pitfalls

- Placing a `__syncthreads()` inside a divergent branch. The barrier is safe only if every thread in the block reaches it the same number of times.
- Using atomics on a contended hot path. Atomics serialise; privatise first (Chapter 8) and fold atomically at the end.
- Releasing a spinlock without a fence. The lock owner's critical-section writes may not be visible to the next acquirer.

- Assuming `volatile` makes an access atomic. It prevents compiler caching; it does not make a read-modify-write atomic.

Check Your Understanding

Why is a race on a GPU worse than on a CPU?

Scale and silence. A race may involve any two of millions of threads, so it can appear only on rare inputs, and the GPU reports success while the corrupted value is stored. There is no exception, only a wrong answer that may take hours to reproduce.

Does divergent control flow cost performance or correctness?

It costs performance, not correctness. Divergent branches in a warp execute serially: the hardware runs the then-path with some lanes masked and then the else-path with the others. The result is correct, but the warp consumes more instruction slots than a uniform branch would.

What is the difference between a barrier and a fence?

A barrier makes all threads in a group wait at a point and orders their memory accesses with respect to each other. A fence orders one thread's memory accesses with respect to what other threads can observe but does not make anyone wait. Barriers are for block-scoped phases; fences are for device-scoped flags and lock release.

Key Takeaways

- A race is two threads accessing one location with a write and no ordering; on a GPU it fails silently.
- Warp divergence serialises divergent paths; branches uniform across a warp are free.
- `__syncthreads()` is a block barrier; it must be uniformly reachable or the block deadlocks.
- Atomics are indivisible read-modify-write operations; contention is the cost and privatisation the fix.
- Floating-point addition is not associative; fixed tree orders give bit-reproducible results.

5.8 Exercises

1. Explain why the `if (i < n)` boundary guard diverges in at most one warp per grid and why that is negligible.
2. The divergent-barrier example deadlocks. Rewrite the pattern so every thread reaches the barrier exactly once.
3. A histogram kernel with 256 global bins suffers heavy contention. Sketch the privatised version: per-block shared histograms, block-level atomic accumulation, and a final fold. Chapter 8 gives the full recipe.
4. `atomicAdd` on `float` returns the old value. Describe an algorithm for a global running maximum that does not need a lock, using `atomicMax`.

Chapter 6: Streams, Events & Asynchronous Execution

Chapter 3 noted that a kernel launch is asynchronous: the host does not wait. This chapter makes that asynchrony useful. The tools are **streams** (ordered queues in which device work executes), **events** (markers for measuring and ordering work), and the **double-buffered pipeline** that overlaps transfers with computation. The chapter closes with **CUDA Graphs**, which capture and replay fixed pipelines with reduced launch overhead.

6.1 The Problem: Serial Execution

Consider the naive vector-add pipeline from Chapter 3, repeated for k chunks:

```
for (int c = 0; c < k; ++c)
{
    cudaMemcpy(d_in, h_in + c * chunk, chunkBytes, cudaMemcpyHostToDevice);
    kernel<<<grid, block>>>(d_in, d_out, chunkElems);
    cudaMemcpy(h_out + c * chunk, d_out, chunkBytes, cudaMemcpyDeviceToHost);
}
```

On the default (legacy) stream, `cudaMemcpy` is synchronous and each kernel launch waits for the previous work. The timeline is serial: transfer, kernel, transfer, kernel. The bus is idle during kernels and the SMs are idle during transfers. The machine is capable of overlapping these operations, but this code does not use that capability.

6.2 Streams: Ordered Queues of Work

Primitive - stream. An ordered sequence of device operations (copies, kernel launches, and events) that executes in FIFO order on the device. Work in different

streams is unordered and may overlap. A stream is created with `cudaStreamCreate` and destroyed with `cudaStreamDestroy`.

Stream semantics:

1. **Order within a stream is guaranteed.** Operations issued to the same stream execute in the order issued.
2. **Order across streams is not guaranteed.** Operations in different streams may execute in any order, or concurrently if resources permit.
3. **Asynchronous by construction.** `cudaMemcpyAsync`, with pinned host memory (Chapter 4), returns immediately; the copy is queued in the stream.

```
// Two streams, each an independent queue.
cudaStream_t s1, s2;
CHECK(cudaStreamCreate(&s1));
CHECK(cudaStreamCreate(&s2));

// Pinned host memory is REQUIRED for async copies (see 4.2).
float *h_pinnedA, *h_pinnedB, *d_A, *d_B, *d_out;
CHECK(cudaMallocHost((void**)&h_pinnedA, chunkBytes));
CHECK(cudaMallocHost((void**)&h_pinnedB, chunkBytes));
CHECK(cudaMalloc((void**)&d_A, chunkBytes));
CHECK(cudaMalloc((void**)&d_B, chunkBytes));
CHECK(cudaMalloc((void**)&d_out, chunkBytes));

// Queue a copy of chunk A in stream 1 and chunk B in stream 2. The copies may
// run concurrently because they are in different streams.
CHECK(cudaMemcpyAsync(d_A, h_pinnedA, chunkBytes,
                      cudaMemcpyHostToDevice, s1));
CHECK(cudaMemcpyAsync(d_B, h_pinnedB, chunkBytes,
                      cudaMemcpyHostToDevice, s2));

// Queue each kernel after its own copy in its own stream. Each kernel must
// write its own output buffer; sharing d_out between streams would be a race.
float *d_outA, *d_outB;
CHECK(cudaMalloc((void**)&d_outA, chunkBytes));
CHECK(cudaMalloc((void**)&d_outB, chunkBytes));
kernel<<<grid, block, 0, s1>>>(d_A, d_outA, chunkElems);
kernel<<<grid, block, 0, s2>>>(d_B, d_outB, chunkElems);
```

The launch syntax gains a fourth argument: `kernel<<<grid, block, sharedBytes, stream>>>`. `sharedBytes` is dynamic shared memory (Chapter 7); `stream` selects the queue. Both default to zero, which is why earlier chapters did not need them.

Async copies require pinned host memory because the DMA engine reads directly from pinned pages (Chapter 4). A pageable pointer forces the runtime into a synchronous staging copy and silently removes the asynchrony.

6.3 The Default Stream and Implicit Synchronisation

A launch without a stream argument uses the **legacy default stream** (stream 0). This stream synchronises with all other streams: any operation in it waits for all previously issued work in every stream and blocks other streams from starting. One streamless launch can therefore serialise an otherwise concurrent pipeline.

Two remedies exist:

- **Per-thread default stream.** Compile with `--default-stream per-thread`, giving each host thread its own non-blocking default stream.
- **Explicit streams.** Name every stream in every launch and copy.

Both are valid; explicit naming is safer because it makes dependencies visible at the call site.

6.4 Events: Markers and Stopwatches

Primitive - event. A marker queued into a stream. It has no payload; it records when the stream reaches it. Events measure time, order cross-stream dependencies, and let the host wait for specific milestones.

```

cudaEvent_t start, stop;
CHECK(cudaEventCreate(&start));
CHECK(cudaEventCreate(&stop));

// Record "start" into stream s1.
CHECK(cudaEventRecord(start, s1));
kernel<<<grid, block, 0, s1>>>(d_A, d_out, chunkElems);
// Record "stop" into stream s1, after the kernel.
CHECK(cudaEventRecord(stop, s1));

// Block the host until the event is reached (the kernel has finished).
CHECK(cudaEventSynchronize(stop));

// Elapsed time in milliseconds between the two events:
float ms = 0.0f;
CHECK(cudaEventElapsedTime(&ms, start, stop));
std::printf("kernel took %.3f ms\n", ms);

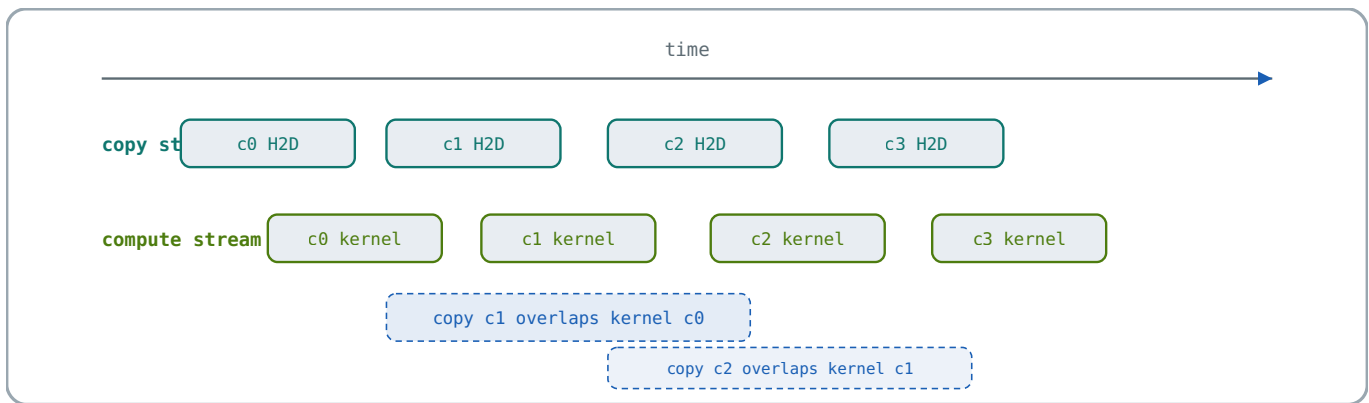
```

Events measure device time. The device records an event when the stream reaches it, so the elapsed interval excludes host-side launch overhead and queueing delay. A `std::chrono` measurement around a launch measures host wall time, which includes whatever the host was doing while the device worked. CUDA events are the reliable instrument for kernel timing (Chapter 16).

Events also order work across streams. `cudaStreamWaitEvent(stream, event)` makes one stream wait for an event recorded in another stream, creating a cross-stream dependency without blocking the host. This is the primitive behind producer/consumer pipelines.

6.5 The Double-Buffered Pipeline

The canonical overlap pattern uses two host buffers. While the GPU computes on chunk c , the DMA engine copies chunk $c + 1$ into the other buffer. The transfer cost moves off the critical path:



Without double buffering, the timeline is serial: copy, kernel, copy, kernel. With it, the only serial residue is the first copy (the pipeline prime) and the last kernel (the pipeline drain).

```

// -----
// Streamed processing of k chunks with double buffering.
// Assumes h_pinned[0] and h_pinned[1] are pinned host buffers, each of
// chunkElems floats, and d_buf[0], d_buf[1] are matching device buffers.
// -----
void runPipelined(int k, int chunkElems, cudaStream_t computeStream,
                  cudaStream_t copyStream)
{
    const size_t chunkBytes = chunkElems * sizeof(float);
    float* h_pinned[2]; float* d_buf[2]; float* d_out;
    cudaEvent_t copyDone[2]; // one event per buffer (created in the omitted
    setup)
    // ... (allocations omitted for brevity; see 6.2) ...

    // Prime the pipeline: copy chunk 0 into device buffer 0 and record the
    // event the first iteration will wait on.
    CHECK(cudaMemcpyAsync(d_buf[0], h_pinned[0], chunkBytes,
                          cudaMemcpyHostToDevice, copyStream));
    CHECK(cudaEventRecord(copyDone[0], copyStream));

    for (int c = 0; c < k; ++c)
    {
        const int cur = c % 2; // buffer used for THIS chunk
        const int nxt = (c + 1) % 2; // buffer used for the NEXT chunk

        // If there is a next chunk, its copy goes into the other buffer in
the
        // COPY stream while the kernel runs in the COMPUTE stream. Record an
        // event after it for the next iteration's kernel to wait on.
        if (c + 1 < k)
        {
            CHECK(cudaMemcpyAsync(d_buf[nxt], h_pinned[nxt], chunkBytes,
                                cudaMemcpyHostToDevice, copyStream));
            CHECK(cudaEventRecord(copyDone[nxt], copyStream));
        }

        // The kernel must wait for its copy. This chunk's copy was queued in
        // the previous iteration (or during the prime), and its completion is
        // marked by copyDone[cur]. cudaStreamWaitEvent installs the
dependency
        // without blocking the host.
        CHECK(cudaStreamWaitEvent(computeStream, copyDone[cur], 0));

        kernel<<<grid, block, 0, computeStream>>>(d_buf[cur], d_out,
                                                    chunkElems);
    }
}

```

```

    }
    CHECK(cudaStreamSynchronize(computeStream));
}

```

The essence is the alternation: copy $c + 1$ into the idle buffer while kernel c runs. The two streams provide the queues; events provide the dependencies; pinned memory provides direct DMA. Chapter 15’s capstone uses this shape for image frames.

One buffer would force copy $c + 1$ to wait for kernel c , because both would touch the same data. Two buffers let the DMA engine and the SMs work on different memory at the same time.

6.6 Stream Priorities and Concurrency Limits

Not every pair of operations can overlap. The hardware limits include:

- **One copy engine per direction** (host-to-device and device-to-host) on most GPUs. Two simultaneous host-device copies are possible, one in each direction.
- **Limited concurrent kernels.** Older GPUs could run only a few kernels concurrently; modern GPUs can run more, but SMs time-slice among resident work.

Priorities hint the scheduler:

```

int lo = 0, hi = 0;
CHECK(cudaDeviceGetStreamPriorityRange(&lo, &hi)); // hi = highest priority
cudaStream_t sHigh, sLow;
CHECK(cudaStreamCreateWithPriority(&sHigh, cudaStreamNonBlocking, hi));
CHECK(cudaStreamCreateWithPriority(&sLow, cudaStreamNonBlocking, lo));

```

Priorities matter when compute and copies compete for the same SMs. Give latency-critical work the high priority and bulk work the low priority. `cudaStreamNonBlocking` makes a stream ignore the default-stream synchronisation rule (§6.3).

6.7 CUDA Graphs: Pipelines Without Per-Launch Overhead

Every `kernel<<<>>>` and `cudaMemcpyAsync` call has host-side overhead for argument marshalling and queueing, roughly 3-10 microseconds per operation. A pipeline with many operations pays that cost per operation. **CUDA Graphs** capture the dependency structure once and replay it with one launch:

Primitive - CUDA graph. A captured, reusable description of device work (kernel launches, copies, and events) and their dependencies. It is captured once, replayed many times, and amortises launch overhead.

```
// Capture phase: record the operations into a graph.
cudaGraph_t graph;
cudaStream_t captureStream;
CHECK(cudaStreamCreateWithFlags(&captureStream, cudaStreamNonBlocking));
CHECK(cudaStreamBeginCapture(captureStream,
    cudaStreamCaptureModeThreadLocal));

// Issue work exactly as in a normal stream, into the capture stream.
kernel<<<grid, block, 0, captureStream>>>(d_A, d_out, chunkElems);
cudaMemcpyAsync(h_out, d_out, chunkBytes, cudaMemcpyDeviceToHost,
    captureStream);

// End capture and instantiate an executable graph.
cudaGraphExec_t exec = nullptr;
CHECK(cudaStreamEndCapture(captureStream, &graph));
CHECK(cudaGraphInstantiate(&exec, graph, 0));

// Replay phase: one call replaces the whole sequence.
for (int frame = 0; frame < 10000; ++frame)
    CHECK(cudaGraphLaunch(exec, /* any stream */ 0));

CHECK(cudaGraphExecDestroy(exec));
CHECK(cudaGraphDestroy(graph));
```

Graphs pay off when launch overhead is a significant fraction of kernel time: many small kernels, or a fixed pipeline replayed thousands of times, such as inference loops and

render pipelines. For large kernels the overhead is negligible, and graphs add complexity without benefit. Chapter 15 measures both regimes.

6.8 Synchronisation Reference

Call	Effect
<code>cudaDeviceSynchronize()</code>	Wait for all device work issued by this host thread
<code>cudaStreamSynchronize(s)</code>	Wait for all work queued in stream <code>s</code>
<code>cudaEventSynchronize(e)</code>	Wait for the device to reach event <code>e</code>
<code>cudaStreamWaitEvent(s, e)</code>	No host wait; install a dependency so stream <code>s</code> waits for event <code>e</code>
<code>cudaMemcpy</code> (sync)	Wait for the copy itself (in the legacy default stream)
<code>cudaMemcpyAsync(..., stream)</code>	No wait; queue the copy in <code>stream</code> and return

Streams as Dependency Graphs

It is natural to think of a stream as a thread or queue and to imagine that two streams behave like two threads. The hardware model is more precise. The GPU contains several independent execution engines: copy engines for host-device transfers, SMs for kernels, and other specialised units. These engines can run concurrently as long as they do not contend for the same data or resources. A stream is an ordered sequence of work for one logical device timeline; work in different streams is unordered and may overlap.

A multi-stream program is therefore a dependency graph. Each operation is a node; each “must wait for” relationship is an edge. Recording an event in a copy stream and making a compute stream wait on it adds an edge: the kernel depends on the copy. Using the legacy default stream implicitly adds edges between everything because it synchronises with all other streams. The graph then has no concurrency; it is one long chain.

The dependency-graph view explains the double-buffered pipeline. The copy of chunk $n + 1$ and the kernel of chunk n operate on different buffers, so no edge connects them. The graph has two independent paths and the hardware can run them concurrently. Events keep the graph correct without serialising it: the compute stream waits only for the event that marks its input’s copy, not for all copies.

CUDA Graphs make the same structure explicit and reusable. A graph is the dependency structure captured once and replayed many times, removing the host-side cost of issuing

each operation and dependency individually. The graph captures addresses and parameters; if buffers move or launch parameters change, the graph must be updated or re-captured.

Common Pitfalls

- Accidentally using the legacy default stream, which synchronises with all other streams and serialises the pipeline. Name streams explicitly or compile with `--default-stream per-thread`.
- Using pageable memory with `cudaMemcpyAsync`. The call silently becomes synchronous and the overlap disappears without an error.
- Recording events on the wrong stream or waiting on the wrong event. `cudaStreamWaitEvent` must reference the event that marks the intended dependency.
- Replaying a CUDA Graph with changed input pointers. Graphs capture addresses; if buffers move, the replay uses stale pointers.

Check Your Understanding

Why does the legacy default stream serialise work?

The legacy default stream synchronises with all other streams: any operation in it waits for previously issued work in every stream and blocks other streams from starting. A single streamless launch therefore injects a full pipeline barrier.

Why are events better than `std::chrono` for kernel timing?

Events are recorded by the device when the stream reaches them, so the elapsed time excludes host launch overhead and queueing delay. `std::chrono` measures host wall time around a launch, which includes whatever the host was doing.

Why are two events enough for any number of double-buffered chunks?

There are two buffers, so there are two copy-completion conditions to track: the current chunk's copy and the next chunk's copy. Each buffer reuses the same event slot every two chunks.

Key Takeaways

- A stream is an ordered FIFO queue of device work; work in different streams may overlap.
- The legacy default stream synchronises with all other streams; name streams or use `cudaStreamNonBlocking`.
- Events measure device time and install cross-stream dependencies via `cudaStreamWaitEvent`.
- Double buffering overlaps the next copy with the current kernel and hides transfer cost.
- CUDA Graphs capture and replay fixed pipelines, amortising launch overhead.

6.9 Exercises

1. Explain why `cudaMemcpyAsync` with pageable memory silently becomes synchronous. What does that do to a double-buffered pipeline?
2. The legacy default stream synchronises with all other streams. Draw the timeline when a pipeline alternates `cudaMemcpyAsync(..., s1)` and `kernel<<<...>>>` with no stream argument.
3. The pipeline loop uses one event per buffer (`copyDone[0]`, `copyDone[1]`). Explain why two events are enough for any number of chunks, then extend the loop to time each kernel with events (§6.4) and report the median per-chunk kernel time.
4. A graph captures a sequence of 500 kernel launches of 2 microseconds each. Host launch overhead is 5 microseconds per launch. How much time does one replay save compared with 500 individual launches?

Chapter 7: Memory Optimisation Deep Dive

This chapter examines the three tools that dominate GPU memory performance: **coalescing and the grid-stride loop, shared memory and bank conflicts**, and **vectorised and specialised memory paths**. It ends with an optimised matrix transpose, the standard exercise in memory optimisation.

7.1 Where Memory Time Goes

From the latency table in Chapter 2, a global load costs roughly 400-800 cycles and a shared-memory access 20-30 cycles. A warp of 32 threads performing one global load each spends about 500 cycles waiting; in that time the SM could have executed roughly 16 shared-memory accesses per lane. Memory-bound kernels (the roofline test of Chapter 1) spend most of their time in this wait.

Two levers reduce it:

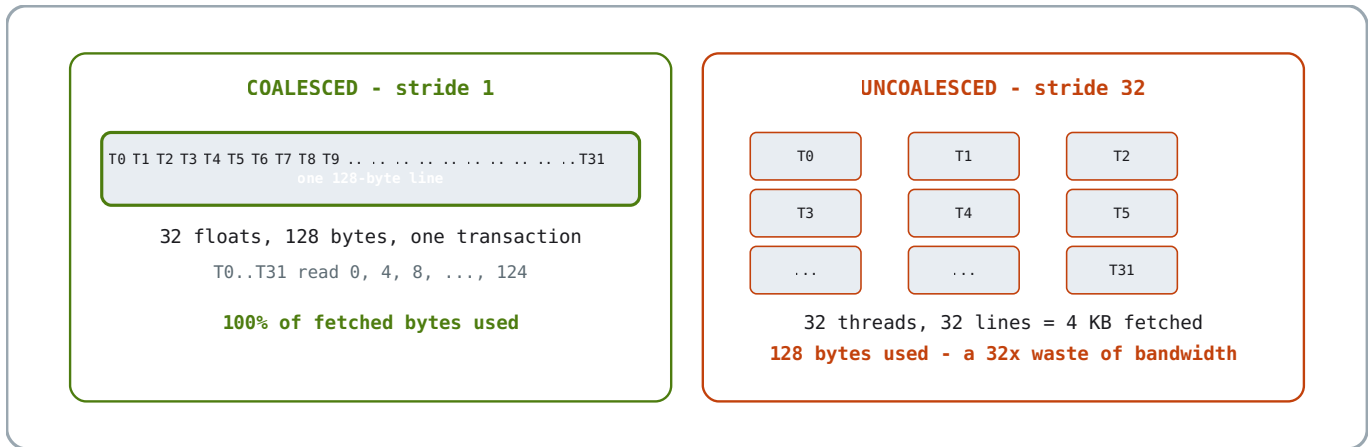
1. **Fewer transactions** - coalescing, so that every fetched byte is used.
2. **Fewer round trips** - reusing fetched data in shared memory or registers.

Everything in this chapter is one of these two levers.

7.2 Coalescing, Quantified

The hardware fetches global memory in **128-byte cache lines** and services requests at **32-byte sector** granularity. A warp's load is coalesced when its 32 addresses fall in as few lines as possible.

- 32 consecutive **floats** (128 bytes) - one line, one transaction.
- 32 **floats** with stride 1 but a misaligned start - two lines.
- 32 **floats** with stride 32 - 32 lines, or 32x the necessary traffic.



The left panel is why the global-index formula exists (Chapter 3, §3.5): it assigns consecutive threads consecutive addresses by construction. The right panel is the result of inverting that mapping, the classic column-access bug in row-major data.

The rule follows from the sector model (§2.7): consecutive thread IDs should map to consecutive addresses. The global-index formula satisfies this for 1-D arrays and row-major 2-D data.

```
// GOOD (coalesced): thread t reads element t of each row.
// For a row-major matrix in[row][col], col varies fastest:
float v = in[row * width + col];           // col == threadIdx.x mapped to col

// BAD (uncoalesced): thread t reads element t*width - a stride of width
// floats.
// 32 threads span 32 different cache lines for a large width:
float v = in[col * width + row];           // col varies slowest
```

7.3 The Grid-Stride Loop

A kernel launched with fewer threads than elements under-utilises the GPU; one launched with far more threads than elements wastes launch resources. The **grid-stride loop** decouples the launch size from the problem size:

```
// One grid covers the array in "rounds": each thread strides forward by the
// total number of threads (gridDim.x * blockDim.x) each iteration.
__global__ void saxpyGridStride(float alpha, const float* x, float* y, int n)
{
    // Total threads in the grid:
    const int stride = gridDim.x * blockDim.x;

    // First element this thread owns:
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    // March forward by 'stride' until past the end.
    for (; i < n; i += stride)
    {
        y[i] = alpha * x[i] + y[i];    // SAXPY: single-precision A·X + Y
    }
}
```

The launch size becomes a tuning parameter, often chosen to saturate the device, and is independent of n . Each thread processes multiple elements, amortising index arithmetic and enabling per-thread data reuse. Coalescing is preserved because consecutive threads still read consecutive addresses within each iteration.

7.4 Shared Memory: The Explicit Cache

Shared memory is the programmer-managed cache of Chapter 2. Its use follows a tile pattern:

1. Cooperatively load a tile of global data into shared memory (coalesced reads);
2. Call `__syncthreads()` to make the tile visible;
3. Compute from shared memory, reusing each loaded value many times;
4. Call `__syncthreads()` before the next tile overwrites this one.

The classic example is the matrix transpose. A naive transpose kernel reads rows (coalesced) and writes columns (uncoalesced), or vice versa; one side always pays. The shared-memory version makes both sides coalesced:

```
// -----
// Shared-memory tiled transpose for a WIDTH x WIDTH matrix of floats, with
// WIDTH a multiple of the tile size TILE (32 here).
//
// Stage 1 (coalesced read):  each thread reads a[iy][ix] from global memory;
//   consecutive threads map to consecutive ix, hence consecutive addresses.
// Stage 2 (shared memory):   the tile is stored in shared as tile[ty][tx].
// Stage 3 (coalesced write): each thread writes tile[tx][ty] to a[jx][jy];
//   consecutive threads (consecutive tx) map to consecutive jx within the
//   same output row (jy), hence consecutive addresses in the output.
//   The transpose happens in shared memory, so both global accesses are
//   coalesced.
// -----
#define TILE 32

__global__ void transposeTiled(const float* in, float* out, int width)
{
    // Shared tile with a padding column (see 7.5 for why +1 exists).
    __shared__ float tile[TILE][TILE + 1];

    // Global coordinates of this thread's element:
    const int ix = blockIdx.x * TILE + threadIdx.x; // column
    const int iy = blockIdx.y * TILE + threadIdx.y; // row

    // Stage 1: coalesced read from global memory.
    if (ix < width && iy < width)
        tile[threadIdx.y][threadIdx.x] = in[iy * width + ix];

    __syncthreads(); // every tile element must be visible before reads

    // Transposed output coordinates:
    const int jx = blockIdx.y * TILE + threadIdx.x; // column of output
    const int jy = blockIdx.x * TILE + threadIdx.y; // row of output

    // Stage 3: write the transposed element. Consecutive threads (x) map to
    // consecutive jx columns within the same output row (jy), i.e.,
    // consecutive addresses in the row-major output. Coalesced.
    if (jx < width && jy < width)
        out[jy * width + jx] = tile[threadIdx.x][threadIdx.y];
}
```

A naive kernel that performs `out[j][i] = in[i][j]` has threads with consecutive IDs reading consecutive `i` (coalesced) but writing `j`-major addresses (uncoalesced). The tile

transposes the data layout in shared memory, so both the global read and the global write are coalesced.

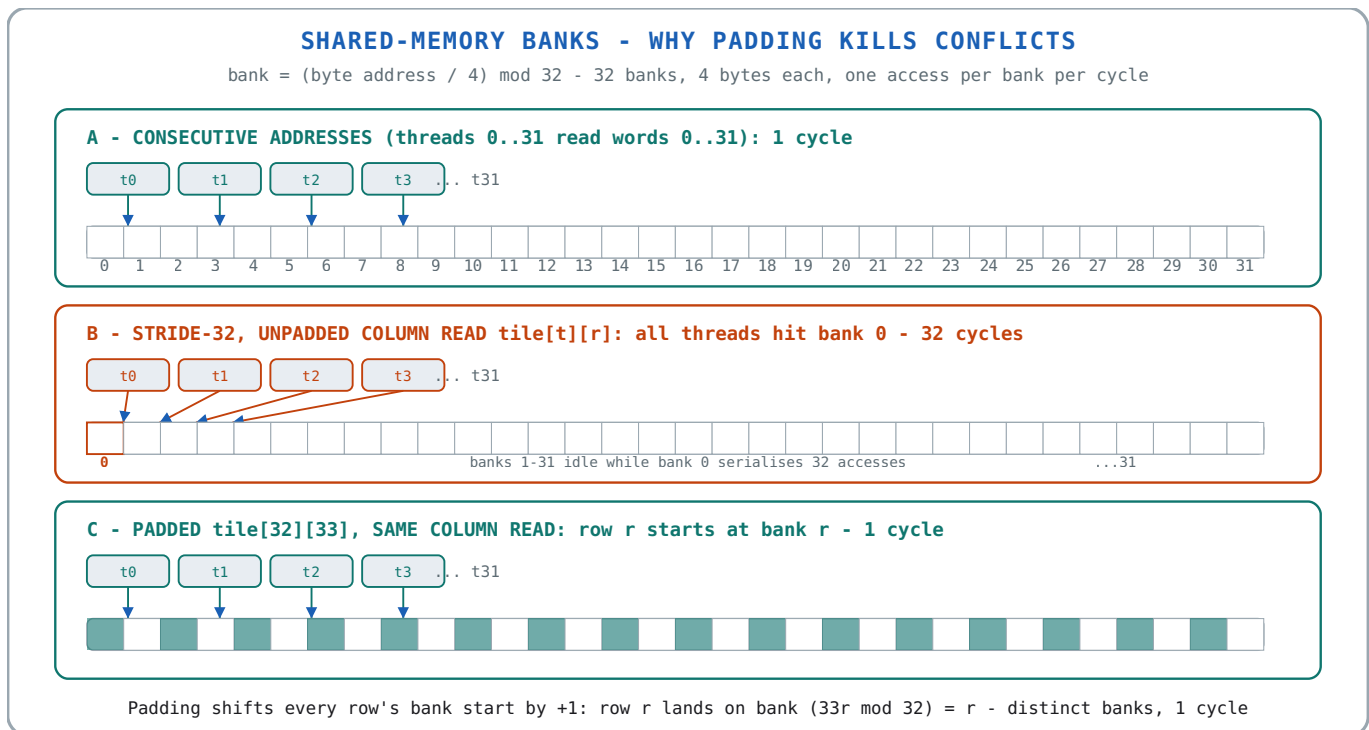
7.5 Bank Conflicts and Column Padding

From Chapter 2, shared memory consists of 32 banks of 4 bytes. The bank of an address is $(\text{address} / 4) \bmod 32$. Consider the unpadded tile `float tile[32][32]`:

- Row r starts at byte $r \times 128$, so every row starts on bank $(r \times 32) \bmod 32 = 0$.
- A row read `tile[threadIdx.y][threadIdx.x]` with consecutive `threadIdx.x` uses all 32 banks once and is conflict-free.

A column read `tile[threadIdx.x][threadIdx.y]` has consecutive threads reading addresses 32 words apart, so all 32 threads hit the same bank. This is a 32-way bank conflict that takes 32 cycles instead of one.

Padding the tile by one column, `float tile[32][33]`, makes row r start at byte $r \times 132$, hence bank $(r \times 33) \bmod 32 = r$. A column read now visits banks 0..31 exactly once. One float of padding per row converts a 32-cycle stall into a one-cycle access.



The padding works because the row stride of 33 words is coprime with the bank count of 32. The same coprime rule keeps the SGEMM tiles in §9.4 conflict-free.

```
// Padding rule:
//  float tile[TILE][TILE];          // 32-way conflicts on column access
//  float tile[TILE][TILE + 1];      // conflict-free column access
```

7.6 Vectorised Loads: `float4` and Alignment

A warp loading 32 `floats` fetches 128 bytes in one memory transaction, but the load instruction moves 4 bytes per lane. A 128-bit vector load moves 16 bytes per lane, so a warp moves 512 bytes per instruction. **Vectorised loads** reduce instruction count and memory requests:

```
// Load four floats per thread, one instruction per four floats.
__global__ void saxpyVec4(float alpha, const float4* x, float4* y, int n4)
{
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n4)
    {
        float4 xv = x[i];          // 16-byte aligned load
        float4 yv = y[i];
        yv.x = alpha * xv.x + yv.x; // four lanes of arithmetic per thread
        yv.y = alpha * xv.y + yv.y;
        yv.z = alpha * xv.z + yv.z;
        yv.w = alpha * xv.w + yv.w;
        y[i] = yv;                 // 16-byte aligned store
    }
}
```

`float4` requires 16-byte alignment. A buffer allocated with `cudaMalloc` is aligned to at least 16 bytes, so converting a base pointer to `float4*` is safe. A pointer offset by an odd number of floats is not aligned. If data does not satisfy the alignment requirement, pad the allocation or handle tail elements with scalar loads.

Vectorisation helps for three reasons: fewer instructions, fewer memory requests, and more efficient use of the 128-bit memory path. On memory-bound kernels, `float4`-style access commonly adds 20-40% throughput. The `int4`, `double2`, and `uint4` types follow the same rules.

7.7 Constant Memory

Primitive - constant memory. A 64 KB read-only memory space cached in a dedicated per-SM cache. It is optimised for broadcasts: when all threads of a warp read the same address, the hardware serves all 32 lanes in one access. When threads read different addresses, the access serialises, the inverse of shared memory's behaviour.

```
// Declared at file scope, device-side:
__constant__ float g_coeffs[16];

// Host fills it with cudaMemcpyToSymbol (note: symbol, not pointer):
float h_coeffs[16] = { /* ... */ };
CHECK(cudaMemcpyToSymbol(g_coeffs, h_coeffs, sizeof(h_coeffs)));

// Kernel reads: all threads read the SAME coefficient per call - a broadcast,
// served in one access from the constant cache.
__global__ void applyCoeffs(const float* in, float* out, int n, int c)
{
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) out[i] = in[i] * g_coeffs[c];    // same address for all threads
}
```

Constant memory suits kernel parameters, coefficients, and lookup tables indexed by a value uniform across the warp. It should be avoided when each thread indexes different data; divergent constant accesses serialise and can be slower than global memory.

7.8 Texture and Surface Memory

Texture memory is a cached read-only path with two special capabilities: **spatial locality** for image-like data, through caches that retain neighbouring pixels, and **hardware interpolation** such as bilinear filtering. On modern GPUs the ordinary L1/L2 caches largely match its raw bandwidth for linear access. The remaining reasons to use a texture object are the interpolation hardware and the `cudaTextureObject` API for image-like data.

For a kernel that reads images with strong 2-D locality, a texture object can be a legitimate optimisation. For linear 1-D access, coalesced global memory is equal or better. The capstone (Chapter 15) uses plain global memory for its image pipeline and operates near peak bandwidth, a useful baseline: coalescing first, specialised paths second.

7.9 Optimisation Checklist

When a kernel is memory-bound, evaluate in order:

1. **Coalescing.** Do consecutive threads access consecutive addresses?
2. **Launch shape.** Is the kernel a grid-stride loop sized to the device, or one thread per element?
3. **Data reuse.** If data is reused, tile it in shared memory (§7.4) and check for bank conflicts (§7.5).
4. **Vectorisation.** Can accesses use `float4/double2` with correct alignment (§7.6)?
5. **Uniform values.** Are warp-uniform constants in constant memory (§7.7)?
6. **Transfer pipeline.** Is the host-device path streamed with pinned memory and streams (Chapters 4 and 6)? A memory-bound kernel is not faster than the copies that feed it.

Each step is cheap to test and easy to measure (Chapter 16). Measure after each change.

Mechanism: Thread Identity and Physical Layout

Coalescing is often taught as a rule: consecutive threads should read consecutive addresses. The rule is correct, but the mechanism behind it is worth stating precisely.

The memory system fetches 128-byte cache lines and services them in 32-byte sectors. When a warp issues a load, the hardware examines all 32 addresses and satisfies them with as few line fetches as possible. Thirty-two consecutive floats span exactly 128 bytes: one line and every byte is used. Thirty-two floats with a stride of 32 words each lie in a different 128-byte region, so the hardware fetches 32 lines to deliver 32 floats. Useful data is identical; traffic is not. Uncoalesced access can multiply memory traffic by 20-30x without changing the arithmetic.

The global-index formula from Chapter 3 gives coalescing by construction: `threadIdx.x` varies fastest, and the fastest-varying memory index of a row-major array is the column. The grid-stride loop preserves this property because consecutive threads still access consecutive addresses in every iteration.

Shared memory has a different physical structure but the same lesson. A warp's shared access completes in one cycle only when its 32 addresses hit 32 different banks. A row read of `float tile[32][32]` hits banks 0..31 once. A column read hits one bank 32 times. Padding the row stride from 32 to 33 words makes row-start banks cycle through 0..31, so column reads also visit every bank once. The formula `bank = (byte_address / 4) mod 32` verifies any layout, including vectorised types and structs.

The unifying idea is that a GPU's memory system rewards aligning thread identity with physical layout: consecutive threads to consecutive global addresses, and distinct threads to distinct shared-memory banks. Coalescing, tiling, padding, vectorisation, and constant-memory broadcasts are all ways to make that alignment exact.

Common Pitfalls

- Mapping `threadIdx.x` to the slowest-varying dimension, the classic column-access bug. Put the fastest-varying thread index on the fastest-varying memory index.
- Assuming `float4` is safe on any pointer. It requires 16-byte alignment; an unaligned offset may compile but fault or corrupt data.
- Padding only one tile dimension. If a tile is read both by row and by column, both shared tiles need `[T][T+1]` or the equivalent.
- Using constant memory for divergent per-thread lookups. Constant memory is fast for broadcasts and slow when a warp reads 32 different addresses.

Check Your Understanding

When does the bank-conflict formula matter more than the "pad by one" rule?

The rule works when the row stride is coprime with 32, as with 33 words. The formula `bank = (address / 4) mod 32` verifies any layout, including wider tiles, vectorised types, and structs, without applying a blind +1.

A warp loads 32 floats starting at byte offset 4. How many lines?

32 floats span bytes 4..131, crossing the 0..127 and 128..255 lines: two 128-byte lines. Perfectly aligned 32 floats (bytes 0..127) touch one line.

When is constant memory slower than global memory?

When threads in a warp read different addresses. The constant cache is optimised for broadcasts; divergent access serialises one address per cycle, which can be slower than a coalesced global load.

Key Takeaways

- Coalescing means touching the fewest 128-byte cache lines per warp access: consecutive threads, consecutive addresses.
- The grid-stride loop decouples launch size from problem size and preserves coalescing.
- Shared memory is the explicit cache; one column of padding removes bank conflicts.
- float4-style vectorised loads move 16 bytes per thread and require 16-byte alignment.
- Constant memory broadcasts uniform reads cheaply; divergent per-thread reads are slower than global memory.

7.10 Exercises

1. A warp loads 32 `floats` starting at byte offset 4 (misaligned by one float). How many 128-byte lines are touched? How many would be touched if the start were aligned to 128?
2. Explain why padding `[TILE][TILE + 1]` fixes column-access bank conflicts using the formula `bank = (byte_address / 4) mod 32`.
3. You are transposing a `1024 x 1024` float matrix with `TILE = 32`. Count the shared-memory traffic per tile for the padded and unpadded versions (assume one column read per thread).
4. When would you not use constant memory for a lookup table? Give a concrete access pattern that makes it slower than global memory.

Chapter 8: Reduction, Scan & Histogram

Code companion: the complete, buildable code for this chapter lives in [code/ch08_reduction/](#) in the repository.

This chapter covers three canonical data-parallel algorithms that appear, in disguise, in almost every real GPU application: **reduction** (sum, max, or min of an array), **scan** (prefix sums, an array of partial results), and **histogram** (counting occurrences). Each is developed from a naive version to an optimised version, with the reasoning for every transformation. These patterns are the vocabulary of Chapter 9’s matrix multiplication and the capstone’s image pipeline.

8.1 The Reduction Problem

Given an array a of n elements, compute $\sum_{i=0}^{n-1} a_i$. On a CPU this is a loop of n additions. On a GPU the challenge is different: additions are cheap, but combining partial results across threads requires communication, and communication is expensive. A good reduction minimises the number of communication rounds and keeps every thread busy between them.

8.2 Stage 0: One Thread Does It All

```
__global__ void reduceNaive(const float* in, float* out, int n)
{
    if (threadIdx.x == 0)                // ONE thread
    {
        float sum = 0.0f;
        for (int i = 0; i < n; ++i) sum += in[i];    // serial, n additions
        out[0] = sum;
    }
}
```

This kernel is correct but uses one thread. It is useful as a reference implementation whose result the optimised kernels are checked against.

8.3 Stage 1: Tree Reduction in Shared Memory

Addition is associative, so the additions can be reordered into a tree. In round 1, threads $0..n/2 - 1$ each add two elements. In round 2, threads $0..n/4 - 1$ each add two partial sums, and so on. The tree has $\log_2 n$ levels, so $n/2$ additions complete in $\log_2 n$ parallel rounds.

```
// Reduce one block's worth of data (one element per thread) using shared
memory.
__global__ void reduceTree(const float* in, float* out, int n)
{
    __shared__ float s[TILE];          // TILE = blockDim.x, a power of two

    const int i = blockIdx.x * blockDim.x + threadIdx.x;

    // Load with a boundary guard; out-of-range elements contribute zero.
    s[threadIdx.x] = (i < n) ? in[i] : 0.0f;

    // Tree: each level halves the number of active threads.
    // Level 0: threads 0..TILE/2-1 add s[t] and s[t + TILE/2].
    // Level k: active threads < TILE >> (k+1).
    for (int stride = TILE / 2; stride > 0; stride >>= 1)
    {
        __syncthreads();                // make all writes visible
        if (threadIdx.x < stride)
            s[threadIdx.x] += s[threadIdx.x + stride];
    }

    // Thread 0 owns the block's total.
    if (threadIdx.x == 0) out[blockIdx.x] = s[0];
}
```

The barrier inside the loop is required because, at each level, thread t reads a partial sum written by thread $t + \textit{stride}$ at the previous level. The barrier guarantees the previous level's writes are complete before the next level reads them.

The halving scheme assumes **TILE** is a power of two, so every level halves evenly and the active set $\textit{threadIdx.x} < \textit{stride}$ is contiguous. Non-power-of-two block sizes complicate

the active-set arithmetic without benefit; 128, 256, and 512 dominate practice. For unsigned sizes, `stride >>= 1` and `stride /= 2` are equivalent; the shift form documents the halving intent.

Cost. The tree has $\log_2 TILE$ levels and one barrier per level. This kernel pays $\log_2 256 = 8$ barriers per block for a 256-thread block. Stage 3 removes all but one.

8.4 Stage 2: Thread Coarsening

One element per thread leaves most threads idle after the initial load: each thread loads one value and then participates in $\log_2 TILE$ additions. **Thread coarsening** makes each thread process many elements in a grid-stride loop (Chapter 7, §7.3) before entering the tree:

```

// Each thread accumulates ELEMS_PER_THREAD elements first (coalesced
// grid-stride loop), then ONE tree reduction over the block.
#define ELEMS_PER_THREAD 4

__global__ void reduceCoarsened(const float* in, float* out, int n)
{
    __shared__ float s[TILE];

    const int stride = blockDim.x * blockDim.x;    // total threads in grid
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    // Grid-stride accumulation: each thread sums its share of the array.
    float sum = 0.0f;
    for (; i < n; i += stride)
        sum += in[i];                                // coalesced within each
pass

    s[threadIdx.x] = sum;

    for (int stride2 = TILE / 2; stride2 > 0; stride2 >>= 1)
    {
        __syncthreads();
        if (threadIdx.x < stride2)
            s[threadIdx.x] += s[threadIdx.x + stride2];
    }
    if (threadIdx.x == 0) out[blockIdx.x] = s[0];
}

```

Coarsening helps for three reasons: each thread issues several independent loads before communicating, increasing memory-level parallelism; the serial addition into a local register uses no barrier or shared memory; and fewer blocks means fewer partial results to combine later. The grid-stride loop also makes the kernel correct for any n , not only multiples of the grid size.

8.5 Stage 3: Warp Shuffles

The tree's barriers are its main cost. A warp's 32 threads, however, can exchange data through registers without touching memory or using a barrier. The instruction is the **warp shuffle**:

Primitive - warp shuffle. `__shfl_down_sync(mask, value, delta)` moves `value` from lane `lane + delta` to lane `lane` within one warp, through the register file. No memory access or barrier is required. `mask` is the set of participating lanes (all 32: `0xffffffff`). All 32 lanes must execute the shuffle or the behaviour is undefined.

```
// Reduce a warp's 32 lanes to lane 0 using only shuffles: 5 steps.
__device__ float warpReduce(float val)
{
    // mask: all 32 lanes participate. Steps move data rightward by
    // 16, 8, 4, 2, 1 - the warp-size equivalents of the tree's strides.
    for (int offset = 16; offset > 0; offset >>= 1)
        val += __shfl_down_sync(0xffffffffu, val, offset);
    return val;        // lane 0 now holds the warp total
}
```

A warp has 32 lanes. The first shuffle moves the partial sums of lanes 16..31 into lanes 0..15, the second folds lanes 8..15 into 0..7, and so on. Five steps reduce a warp, matching the $\log_2 32$ levels of the shared-memory tree, but at register speed with no barrier and no shared memory.

8.6 The Complete Block Reduction

The production form combines coarsening (§8.4), warp shuffles (§8.5), and one shared-memory round per block. Each warp writes one value to shared memory, one barrier makes those values visible, and the first warp combines them with shuffles:

```

#define TILE 256                // block size, a multiple of the warp size 32
#define WARPS (TILE / 32)      // 8 warps per block

// Warp-level reduction (from 8.5), returns the warp's total in lane 0.
__device__ float warpReduce(float val)
{
    for (int offset = 16; offset > 0; offset >>= 1)
        val += __shfl_down_sync(0xffffffffu, val, offset);
    return val;
}

__global__ void reduceFull(const float* in, float* out, int n)
{
    __shared__ float s[WARPS];          // one slot per warp

    // --- Coarsened accumulation over the grid -----
    const int stride = gridDim.x * blockDim.x;
    float sum = 0.0f;
    for (int i = blockIdx.x * blockDim.x + threadIdx.x; i < n; i += stride)
        sum += in[i];

    // --- Warp-level reduction: 8 warps * 5 shuffle steps, no barriers -----
    // Lane 0 of each warp now holds that warp's partial total.
    sum = warpReduce(sum);

    // --- One shared-memory round -----
    const int lane = threadIdx.x % 32;    // position within my warp
    const int warpId = threadIdx.x / 32;  // which warp am I

    if (lane == 0) s[warpId] = sum;        // each warp writes ONE value
    __syncthreads();                      // the ONLY barrier per block

    // --- First warp combines the 8 warp totals -----
    if (warpId == 0)
    {
        // Lane < WARPS loads a warp total; others load identity (0.0f).
        const float v = (lane < WARPS) ? s[lane] : 0.0f;
        sum = warpReduce(v);              // shuffle again over 8 values
        if (lane == 0) out[blockIdx.x] = sum; // block total
    }
}

```

Cost. One barrier per block, down from eight; one shared-memory round trip per block; and five shuffle steps per warp. The only global traffic is the coalesced read. This kernel is

the standard against which reductions are judged and routinely achieves over 90% of peak bandwidth.

Determinism. The tree order is fixed by the code, so the summation order is fixed and the result is bit-reproducible across runs. An atomic-based reduction (Chapter 5, §5.6) is not.

8.7 Scan (Prefix Sum)

Primitive - inclusive scan. Given a , compute $y_i = a_0 + a_1 + \dots + a_i$. **Primitive - exclusive scan.** Given a , compute $y_i = a_0 + \dots + a_{i-1}$, with $y_0 = 0$. An exclusive scan of an array is an inclusive scan shifted right by one.

Scans appear in stream compaction (filtering), radix sort, and workload equalisation: algorithms that need to know where data lands. The work-efficient **Blelloch scan** is the canonical GPU formulation. It has two phases:

1. **Upsweep** - a tree reduction that computes partial sums, as in §8.3, but stores the internal nodes instead of discarding them.
2. **Downsweep** - a second tree that propagates totals to produce exclusive prefix sums.

```

// Exclusive scan of a BLOCK's data, in place in shared memory.
// Assumes blockDim.x is a power of two. After the call,
//   s[0] = 0, s[i] = a_0 + ... + a_{i-1} (exclusive prefix sums)
// The block total ends in s[blockDim.x - 1].
__device__ void scanBlock(float* s)
{
    const int n = blockDim.x;

    // --- Phase 1: upsweep (tree reduction, storing internal nodes) -----
    // After level k, s[i] holds the sum of the 2^(k+1) elements ending at i.
    for (int stride = 1; stride < n; stride <= 1)
    {
        __syncthreads();
        const int t = (threadIdx.x + 1) * 2 * stride - 1; // right child
        if (t < n)
            s[t] += s[t - stride]; // parent = sum
    }

    // --- Phase 2: downsweep (propagate carries back down) -----
    // First, the root's carry is the identity for addition: the sum of the
    // (empty) sequence before the whole array.
    if (threadIdx.x == 0) s[n - 1] = 0.0f;

    // Walk the tree top-down. At each level, the pair rooted at right child
    // t (left child t - stride) receives its CARRY - the sum of everything
    // before its subtree, which the parent level stored in s[t]:
    // - the left child inherits the carry unchanged;
    // - the right child gets carry + (its old value, the left subtree sum).
    // Inductively every slot ends up holding the sum of the elements before
    // it: the exclusive prefix.
    for (int stride = n / 2; stride > 0; stride >>= 1)
    {
        __syncthreads();
        const int t = (threadIdx.x + 1) * 2 * stride - 1; // right child
        if (t < n)
        {
            const float carry = s[t]; // sum before this pair
            s[t] = carry + s[t - stride]; // right child: carry + left
sum
            s[t - stride] = carry; // left child: just the carry
        }
    }
    __syncthreads();
}

```

The index arithmetic $(\text{threadIdx.x} + 1) * 2 * \text{stride} - 1$ identifies the right child of each subtree: within a group of size $2 * \text{stride}$, these are exactly the odd indices. The important property is that each level performs disjoint writes; no two threads write the same slot, so no atomicity is needed.

The downsweep produces exclusive sums through the invariant of the carry. At each level, $s[t]$ holds the sum of everything before the pair $(t - \text{stride}, t)$. The root's carry is set to zero before the loop. Each step passes the carry unchanged to the left child and passes $\text{carry} + \text{left_subtree_sum}$ to the right child. Inductively, slot i ends with the sum of elements $0..i - 1$. A hand trace on $[1, 2, 3, 4]$ is Exercise 3. Note that $s[t]$ must be read into carry before $s[t - \text{stride}]$ is overwritten.

Cost. The scan has two passes, each with $\log_2 n$ barrier levels. Here the barrier count is inherent to the algorithm rather than an implementation defect. A single block can scan at most 1,024 elements. Larger arrays require a two-level scheme of block scans plus a scan of block totals, which the CUB library provides (Chapter 11).

8.8 Histogram: Privatisation

The naive histogram of Chapter 5 performs one global `atomicAdd` per element and serialises on contended bins. The standard fix is **privatisation**: each block accumulates into its own shared-memory histogram, where atomics are cheaper, and one thread per block folds the private histogram into global memory at the end.

```

#define BINS 256
#define TILE 256

__global__ void histogramPrivatised(const unsigned char* data, int* g_hist,
                                    int n)
{
    // Private histogram for THIS block, in shared memory.
    __shared__ int s_hist[BINS];

    // Initialise the private histogram (all threads help; one barrier).
    for (int b = threadIdx.x; b < BINS; b += TILE) s_hist[b] = 0;
    __syncthreads();          // all bins zeroed before any thread counts

    // Accumulate. Each thread counts its elements into shared memory.
    // Shared-memory atomics are fast; contention is spread across BINS.
    for (int i = blockIdx.x * TILE + threadIdx.x; i < n; i += blockDim.x *
TILE)
    {
        const int bin = data[i];
        atomicAdd(&s_hist[bin], 1);
    }

    __syncthreads();          // all counts complete before folding

    // Fold: one thread per bin adds this block's count to global memory.
    for (int b = threadIdx.x; b < BINS; b += TILE)
        if (s_hist[b] != 0)          // skip empty bins: less global
traffic
            atomicAdd(&g_hist[b], s_hist[b]);
}

```

Contention now occurs in shared memory, where an atomic is roughly an order of magnitude cheaper than a global atomic, and it is spread across 256 bins rather than concentrated at one global address. The global fold touches each bin once per block. For highly skewed data, all elements fall in one bin and shared atomics still contend; the next-level fix is per-warp histograms. Privatisation handles the common case.

The kernel uses two barriers, one after zeroing and one before the fold. Both are uniformly reachable because the loops have compile-time trip counts.

8.9 Summary Table

Algorithm	Naive cost	Optimised cost	Key tool
Reduction	$O(n)$ serial or $\log_2 n$ barriers	1 barrier, $O(\log_2 32)$ shuffles	Warp shuffle + coarsening
Scan	$O(n)$ serial	$2 \log_2 n$ barriers	Blelloch upsweep/ downsweep
Histogram	global atomic per element	shared privatisation + fold	Per-block private bins

Where Partial Results Live

A reduction's arithmetic is simple; its performance is determined by how partial results travel between threads, how often they touch memory, and how often threads wait for one another.

A single-threaded reduction keeps a running sum in one register. There is no communication, but only one thread works. A GPU has thousands of execution units, so the problem is not how to add but how to divide the array among threads, combine their results, and minimise the cost of combining.

Each reduction version answers that question differently:

1. **The naive kernel** keeps everything in one thread. It is correct but uses one thread out of thousands.
2. **The shared-memory tree** divides work among all threads of a block and combines partial sums through shared memory. Its cost is a barrier at every tree level.
3. **The coarsened kernel** accumulates several elements per thread in a register before entering the tree, reducing the frequency of communication.
4. **The warp-shuffle kernel** moves data between lanes through registers. Shuffles need no barrier because the lanes of a warp execute together.
5. **The full kernel** combines all of these: coarsen, shuffle within warps, write one value per warp to shared memory, one barrier, and a final shuffle over warp totals.

Scan is a reduction with a harder requirement: every output position needs the sum of everything before it. The Blelloch scan solves it with an upsweep that stores internal tree nodes and a downsweep that propagates carries down the tree. The arithmetic is still addition; the work is in the index arithmetic and in ordering writes so that a thread reads a value before it is overwritten.

Histogram privatisation applies the same principle to atomics. Global atomics are expensive when many threads target the same bin because the hardware serialises read-modify-write cycles. A privatised histogram accumulates in shared memory and folds into global memory once. Communicate as little as possible, and when communication is unavoidable, do it in bulk.

Reduction, scan, and histogram are three views of one problem: combining distributed data with minimal communication. The patterns recur in matrix multiplication (Chapter 9), library primitives (Chapter 11), and multi-GPU collectives (Chapter 19).

Common Pitfalls

- Using non-power-of-two block sizes with tree algorithms. The halving scheme assumes `TILE` is a power of two.
- Calling `warpReduce` from only some lanes. All 32 lanes must execute `__shfl_down_sync` with the same mask, or the behaviour is undefined.
- Forgetting the final `__syncthreads()` after a shared-memory scan. The last read must not begin before the last write is visible.
- Using global atomics for the whole histogram instead of privatising. Global contention serialises; shared privatisation plus one fold per block is the standard fix.

Check Your Understanding

Why is one barrier per block enough in `reduceFull`?

Each warp reduces its 32 lanes with shuffles, using no memory or barrier. Only one value per warp is written to shared memory, so one barrier makes those values visible to warp 0, which then combines them with shuffles.

What does `warpReduce` return for lanes other than lane 0?

The shuffle loop leaves partial values in every lane. The documented result, and the value used by the kernel, is the total in lane 0. Other lanes contain intermediate sums and should not be used.

Why is a fixed tree reduction bit-reproducible but an atomic reduction is not?

A fixed tree always sums in the same order, so floating-point rounding is identical on every run. Atomics allow the hardware to choose an order that can differ between runs, changing the last bits.

Key Takeaways

- The optimised reduction: coarsen (grid-stride), reduce within each warp by shuffle, then one shared-memory round and one barrier per block.
- Warp shuffles exchange values through registers, with no memory or barrier.
- The Blelloch scan is an upsweep that stores internal nodes and a downsweep that propagates carries to build exclusive prefixes.
- Histograms should be privatised per block in shared memory and folded into global memory once.
- A fixed tree order makes reductions bit-reproducible across runs.

8.10 Exercises

1. Derive the number of barriers in the Stage-1 tree reduction for `TILE = 512`, and compare it with the full kernel of §8.6.
2. In `reduceFull`, why does lane 0 of each warp write `s[warpId]` and not every lane? What would happen if every lane wrote its own `sum`?
3. Trace the Blelloch downsweep on `[1, 2, 3, 4]` by hand, showing the state of `s` after each level. Verify the exclusive prefix `[0, 1, 3, 6]`.
4. The histogram fold skips empty bins with `if (s_hist[b] != 0)`. Is this correct? Is it always faster? Consider a case where every bin is non-empty.

Chapter 9: Optimised Matrix Multiplication

Code companion: the complete, buildable code for this chapter lives in [code/ch09_sgemm/](#) in the repository.

Matrix multiplication ($C = A \times B$, all $N \times N$) is the canonical GPU workload: it is the inner loop of deep learning, linear algebra, and scientific computing. It is also a useful teaching kernel because it is compute-bound at large N , exercises ideas from Chapters 2, 7, and 8, and has an optimised form similar in structure to the kernels shipped in cuBLAS (Chapter 11). This chapter develops it in stages and gives the reasoning at each step.

9.1 SGEMM Is Compute-Bound

Single-precision GEMM performs N^3 multiply-adds, or $2N^3$ FLOPs. The inputs are N^2 elements of A and N^2 of B; the output is N^2 elements of C. With perfect caching, the minimum traffic is $3N^2$ elements, or $12N^2$ bytes. The arithmetic intensity (Chapter 1) is:

$$I = \frac{2N^3}{12N^2} = \frac{N}{6} \text{ FLOP/byte}$$

For $N = 4096$, $I \approx 683$ FLOP/byte, two orders of magnitude above the roughly 18 FLOP/byte ridge point of a modern GPU (Chapter 2). At this size SGEMM is **compute-bound**: the memory system is not the constraint. Keeping the arithmetic units fed is the constraint. Each optimisation below increases reuse so that every value loaded from memory feeds as many FLOPs as possible.

9.2 Stage 0: The Naive Kernel

The simplest kernel assigns one thread to one output element and loops over the shared dimension k :

```

// One thread per output element C[i][j]. Each thread loops over k.
// This mapping makes threadIdx.x the ROW index, which is the wrong choice
// for row-major memory.
__global__ void sgemmNaive(const float* A, const float* B, float* C,
                          int N)
{
    const int i = blockIdx.x * blockDim.x + threadIdx.x;    // row of C
    const int j = blockIdx.y * blockDim.y + threadIdx.y;    // col of C

    float sum = 0.0f;
    for (int k = 0; k < N; ++k)
        // A[i][k] : consecutive threads (consecutive i) read A[i*N + k] with
        //   stride N between lanes. Uncoalesced.
        // B[k][j] : consecutive threads have the same j within a warp, so
this
        //   address is a broadcast, not a stream.
        // C[i][j] : consecutive threads write C[i*N + j] with stride N.
        //   Uncoalesced.
        sum += A[i * N + k] * B[k * N + j];
    C[i * N + j] = sum;
}

```

The read of A is stride- N and the write of C is stride- N . Every output element also re-reads a full row of A and a full column of B from global memory: $2N^3$ bytes moved for $2N^3$ FLOPs, intensity near 1, far below the ridge. The kernel is memory-bound because of its access pattern, not because matrix multiplication is inherently memory-bound.

9.3 Stage 1: Make the Block Shape Match Memory

A cheap fix changes the thread-to-element mapping so that `threadIdx.x` indexes the column and `threadIdx.y` indexes the row. For a warp whose `x` lanes run across a row of the output tile:

- `B[k][j]`: consecutive threads read consecutive `j`, so the read is coalesced.
- `A[i][k]`: all threads in a warp row have the same `i`, so they read the same address. The hardware broadcasts it.
- `C[i][j]`: consecutive threads write consecutive `j`, so the write is coalesced.

```

__global__ void sgemmCoalesced(const float* A, const float* B, float* C,
                               int N)
{
    const int i = blockIdx.y * blockDim.y + threadIdx.y;    // row
    const int j = blockIdx.x * blockDim.x + threadIdx.x;    // col

    float sum = 0.0f;
    for (int k = 0; k < N; ++k)
        sum += A[i * N + k] * B[k * N + j];    // B coalesced, A broadcast
    C[i * N + j] = sum;
}

```

This removes the uncoalesced traffic, but reuse is still zero: each output element re-reads $2N$ floats from global memory, and intensity remains near

1. The fix for reuse is tiling.

9.4 Stage 2: Shared-Memory Tiling

A block of $T \times T$ threads computes a $T \times T$ tile of C . It loads a $T \times T$ tile of A and a $T \times T$ tile of B into shared memory, advances k in steps of T , and each loaded value feeds T threads. The reuse factor is T : one global load serves T^2 multiply-adds instead of T .

```

#define T 16                // tile size: 16x16 threads per block

__global__ void sgemmTiled(const float* A, const float* B, float* C, int N)
{
    // Tiles in shared memory. The +1 padding (Chapter 7, 7.5) avoids bank
    // conflicts on column access; the A-tile is read by column in the k-loop.
    __shared__ float sA[T][T + 1];
    __shared__ float sB[T][T + 1];

    // Output coordinates of this thread:
    const int row = blockIdx.y * T + threadIdx.y;    // global row of C
    const int col = blockIdx.x * T + threadIdx.x;    // global col of C

    float acc = 0.0f;    // this thread's partial C[row][col]

    // Sweep k in tiles of T. The block needs the A-tile columns [k0..k0+T)
    // and the B-tile rows [k0..k0+T) for each k0 step.
    for (int k0 = 0; k0 < N; k0 += T)
    {
        // Coalesced global loads into shared memory.
        // sA[ty][tx] = A[row][k0+tx] (row segment, coalesced)
        // sB[ty][tx] = B[k0+ty][col] (column segment, coalesced)
        sA[threadIdx.y][threadIdx.x] = A[row * N + k0 + threadIdx.x];
        sB[threadIdx.y][threadIdx.x] = B[(k0 + threadIdx.y) * N + col];

        __syncthreads();    // tile complete before any thread reads it

        // Inner product over the tile. Each thread reads:
        //   sA[ty][k] - row of the A-tile (bank-conflict-free due to pad)
        //   sB[k][tx] - column of the B-tile (broadcast along the row)
        #pragma unroll
        for (int k = 0; k < T; ++k)
            acc += sA[threadIdx.y][k] * sB[k][threadIdx.x];

        __syncthreads();    // tile done before the next tile overwrites it
    }

    C[row * N + col] = acc;
}

```

Role of `#pragma unroll`. The inner k loop has a compile-time trip count of 16. Unrolling emits straight-line FMAs, removes loop bookkeeping, and lets the compiler schedule shared-memory loads ahead of the arithmetic, hiding shared-memory latency behind FMA work. On this kernel the pragma is typically worth 10-20%.

Bank-conflict analysis. For `sA[threadIdx.y][k]`, a warp of 32 threads with $T = 16$ spans two block rows. Within one row, all 16 lanes read the same address (broadcast). Across the two rows, addresses differ by the padded row stride of 17 words, hence by 17 banks. No conflict occurs because the padding keeps the row stride coprime with the 32-bank layout. For `sB[k][threadIdx.x]`, consecutive `threadIdx.x` read consecutive columns of one padded row; lanes in the second block row repeat the same addresses, which the hardware serves as broadcasts.

Choice of $T = 16$. A 16x16 tile uses $2 \times 16 \times 17 \times 4 = 2,176$ bytes of shared memory and 256 threads per block. The size balances reuse (16x) against occupancy. Larger tiles such as 32x32 give more reuse but fewer resident blocks; §9.6 shows the occupancy trade.

Reuse accounting. Each element of A loaded into shared memory is used by $T = 16$ threads, and each element of B by 16 threads. Global traffic drops by a factor of 16 relative to the naive kernel. The kernel is now compute-bound.

9.5 Stage 3: Register Tiling

The tiled kernel still reads shared memory for every FMA: one shared load per multiply-add. Shared-memory bandwidth is finite. With 32 banks of 4 bytes, an SM can supply at most 128 bytes per cycle, and modern FP32 units can consume 128 FLOPs per cycle. The next step gives each thread more than one output element so that each shared-memory value is reused from registers:

```

#define T 16
#define RM 2          // rows of output per thread
#define RN 2          // cols of output per thread

__global__ void sgemmRegisterTiled(const float* A, const float* B, float* C,
                                   int N)
{
    __shared__ float sA[T][T + 1];
    __shared__ float sB[T][T + 1];

    // Each thread now owns an RM x RN micro-tile of C.
    // The block covers a T x T output tile with T*T/(RM*RN) threads.
    const int tx = threadIdx.x;          // 0..T/RN-1
    const int ty = threadIdx.y;          // 0..T/RM-1

    // Global coordinates of this thread's micro-tile (top-left corner):
    const int row0 = blockIdx.y * T + ty * RM;
    const int col0 = blockIdx.x * T + tx * RN;

    // Accumulators live in registers, one per micro-tile element:
    float acc[RM][RN];
    #pragma unroll
    for (int r = 0; r < RM; ++r)
        for (int c = 0; c < RN; ++c) acc[r][c] = 0.0f;

    for (int k0 = 0; k0 < N; k0 += T)
    {
        // Tile load: T*T elements spread over T*T/(RM*RN) threads, so each
        // thread loads an RM x RN patch of each tile. Every element of the
        // A-tile and B-tile is loaded exactly once: thread (tx, ty) covers
        // rows ty*RM..ty*RM+RM-1 and columns tx*RN..tx*RN+RN-1. Consecutive
        // tx cover consecutive columns -> coalesced row segments.
        #pragma unroll
        for (int r = 0; r < RM; ++r)
            for (int c = 0; c < RN; ++c)
                sA[ty * RM + r][tx * RN + c] =
                    A[(row0 + r) * N + k0 + tx * RN + c];
        #pragma unroll
        for (int r = 0; r < RM; ++r)
            for (int c = 0; c < RN; ++c)
                sB[ty * RM + r][tx * RN + c] =
                    B[(k0 + ty * RM + r) * N + col0 + c];

        __syncthreads();
    }
}

```

```

// Micro-tile FMA loop: for each k, each shared value feeds RM*RN
// FMAs, all from registers. Shared loads drop by a factor RM*RN.
#pragma unroll
for (int k = 0; k < T; ++k)
{
    // Load the A-row segment and B-col segment once into registers:
    float a_reg[RM], b_reg[RN];
    #pragma unroll
    for (int r = 0; r < RM; ++r)
        a_reg[r] = sA[ty * RM + r][k];
    #pragma unroll
    for (int c = 0; c < RN; ++c)
        b_reg[c] = sB[k][tx * RN + c];

    // RM*RN FMAs, zero shared-memory traffic in the inner product:
    #pragma unroll
    for (int r = 0; r < RM; ++r)
        for (int c = 0; c < RN; ++c)
            acc[r][c] += a_reg[r] * b_reg[c];
}

__syncthreads();
}

// Write the micro-tile back:
#pragma unroll
for (int r = 0; r < RM; ++r)
    for (int c = 0; c < RN; ++c)
        C[(row0 + r) * N + col0 + c] = acc[r][c];
}

```

Micro-tile size and registers. Each shared-memory value now feeds $RM \times RN$ FMAs from registers. With a 2x2 micro-tile, shared-memory traffic drops by 4x; with 4x4, by 16x. The limit is register pressure. Each accumulator, plus the `a_reg` and `b_reg` arrays, consumes registers per thread, and register pressure caps occupancy (§2.9). A 4x4 micro-tile uses roughly 32 or more registers; 8x8 would spill on most GPUs. Production kernels in cuBLAS use larger micro-tiles with specialised register allocation; the 2x2 and 4x4 versions capture the mechanism.

Tile-load indexing. In the A-tile load, each thread writes an RM -row patch at `sA[ty*RM + r][tx*RN + c]`. The RN elements of one row are covered by RN different threads because `tx` ranges over T/RN values. The loads are coalesced because consecutive `tx` cover consecutive columns. In the B-tile load, `col0` already contains the thread's $tx \times RN$ column

offset, so the global column is `col0 + c`. Adding `tx*RN` a second time would read the wrong `B` elements.

9.6 Occupancy and Launch Bounds

Register tiling increases registers per thread. If the compiler uses, for example, 40 registers, occupancy falls to $64K/(40 \times 256) \approx 6$ blocks per SM. That may be acceptable because the kernel is compute-bound and register tiling provides instruction-level parallelism. The compiler should be told the budget so it does not silently spill:

```
// Tell ptxas: this kernel must fit in at most 256 threads/block, and at least
// 4 blocks/SM must be resident. ptxas trades registers for occupancy within
// the budget rather than spilling.
__global__ void __launch_bounds__(256, 4)
sgemmRegisterTiled(const float* A, const float* B, float* C, int N) { /* ...
*/ }
```

The body is omitted here; apply the attribute to the full kernel of §9.5. The companion [code/ch09_sgemm/sgemm.cu](#) ships the combined final definition.

Without `__launch_bounds__`, `ptxas` minimises register use for correctness but may use more registers than the target occupancy allows. `__launch_bounds__(maxThreads, minBlocksPerSM)` turns the occupancy reasoning of Chapter 2 into a compiler constraint.

The right setting is found by measurement. For a given micro-tile, run the kernel with `minBlocksPerSM` equal to 2, 4, 6, and 8 and compare (Chapter 16). The balance between register-tiling ILP and occupancy latency hiding has no universal answer.

9.7 What the Optimised Kernel Achieves

With $T = 16$, a 2x2 register tile, padding, and `__launch_bounds__`, the kernel of §9.5 typically reaches 60-75% of peak FP32 on a modern GPU. The remaining gap comes from

shared-memory FMA supply and tile-loop overhead. Closing it further requires techniques beyond this chapter:

- **Warp-level tiling**, in which each warp computes a wide micro-tile with `ldmatrix`, the layout-descriptor load instruction used by cuBLAS;
- **Tensor cores**, whose `mma` instructions multiply 16x16x16 tiles per instruction on a separate pipeline (Chapter 11).

The progression from naive to register-tiled follows the method used throughout the book: find the bottleneck, remove it, measure, and repeat.

What Each Optimisation Stage Does

SGEMM contains, in one program, every optimisation idea introduced earlier in the book.

The naive kernel's problem is memory, not arithmetic. With `threadIdx.x` mapped to the row, consecutive threads in a warp access A with stride N and write C with stride N . Half of the traffic is uncoalesced, so the kernel becomes memory-bound despite performing useful arithmetic.

The coalesced kernel maps `threadIdx.x` to the column. B reads are coalesced, A reads are broadcast, and C writes are coalesced. Both kernels still re-read operands from global memory for every output element, so intensity remains low.

Shared-memory tiling introduces reuse. A block of 16x16 threads loads a 16x16 tile of A and a 16x16 tile of B into shared memory and computes the output tile. Each global load is used by 16 threads instead of one, reducing global traffic by a factor of 16. The new bottleneck is shared-memory bandwidth, because the inner product reads shared memory for every FMA.

Register tiling removes that bottleneck. With a 2x2 micro-tile, each thread loads row and column segments into registers once per k and performs four FMAs with no shared-memory traffic in the inner loop. The FMA-to-shared-load ratio improves by a factor of four at the cost of registers, which is where `__launch_bounds__` enters. The compiler chooses a register count, and that choice trades occupancy (Chapter 2) against spills.

The same progression applies to any optimised kernel: identify the current bottleneck (uncoalesced access, no reuse, shared-memory bandwidth, or registers), remove it, and measure. The roofline model explains why each stage matters: each stage raises arithmetic

intensity by moving data closer to the arithmetic units, from global memory to shared memory to registers.

Common Pitfalls

- Getting tile-load indexing wrong by double-counting an offset, such as writing `col0 + tx*RN + c` when `col0` already contains the thread's column offset. Trace one thread's load by hand before launching.
- Using `__launch_bounds__` without measuring. Forcing occupancy can increase register spills and slow the kernel.
- Forgetting the second `__syncthreads()` after the inner-product loop. The next tile load would overwrite shared memory while some threads still read it.
- Assuming the naive kernel is fast because it is simple. With the wrong thread-to-data mapping, uncoalesced A and C traffic can be 20-30x larger than necessary.

Check Your Understanding

Why is SGEMM compute-bound at $N=4096$?

Its intensity is $N/6 \approx 683$ FLOP/byte, far above typical ridge points of 20-40 FLOP/byte. The memory system can feed the arithmetic units; the limit is keeping the FMAs fed with data reused from shared memory and registers.

What does the +1 padding in `sA[T][T+1]` do?

It makes the row stride 17 words instead of 16. Because 17 is coprime with the 32-bank shared-memory layout, column accesses land on distinct banks rather than forming a 32-way bank conflict.

Why can `__launch_bounds__` decrease performance?

It constrains the compiler to a register budget. If the kernel needs more registers than the budget allows, ptxas spills to local memory, and the spill traffic can cost more than the occupancy gain.

Key Takeaways

- SGEMM is compute-bound (intensity $N/6$ FLOP/byte); the goal is arithmetic reuse, not just coalescing.
- The naive kernel with the wrong thread-to-element mapping has zero reuse and uncoalesced A and C traffic.
- Shared-memory tiling makes each loaded element feed T threads; global traffic drops by T .
- Register tiling makes each shared-memory value feed $RM \times RN$ FMAs from registers.
- `__launch_bounds__` trades registers for occupancy; the right balance is found by measuring.

9.8 Exercises

1. Verify the arithmetic-intensity claim: show that for $N = 4096$, $I = N/6 \approx 683$ FLOP/byte and compare it with the ridge point of Chapter 2.
2. In §9.4, count the shared-memory bytes loaded per block per k_0 step. How many FLOPs do they feed? Show that the ratio is T FLOPs per shared byte.
3. Explain why `__launch_bounds__(256, 4)` can decrease performance even though it increases occupancy.
4. In the §9.5 tile load, trace which thread loads `sB[3][7]` for $T = 16$, $RM = RN = 2$.

Chapter 10: Modern C++ for CUDA

*“Within C++, there is a much smaller and cleaner language struggling to get out.” — Bjarne Stroustrup, *The Design and Evolution of C++* (1994)*

Code companion: the complete, buildable code for this chapter lives in `code/ch10_device_buffer/` in the repository.

Chapters 3-9 wrote CUDA in a C-style dialect: raw `cudaMalloc` pointers, manual `cudaFree` calls, and a `CHECK` macro for errors. This chapter applies modern C++ (C++17/20) to GPU programming: **RAII** to manage allocations, **templates** to make kernels generic, `constexpr` to make configuration compile-time, and **exceptions** to make errors visible. The result is the style used by the rest of the book and the capstone.

10.1 Problems with Raw CUDA C

The Chapter 3 vector-add had three structural weaknesses, all inherited from the C API:

1. **Resource leaks.** Every `cudaMalloc` must be matched with `cudaFree`. An early `return` or an exception between the two leaks device memory. Device memory is scarce and process-scoped; a leaked allocation is unavailable until the process exits.
2. **Unchecked errors.** The `CHECK` macro routes calls through an error check, but nothing enforces the discipline. A call added without `CHECK` is unchecked.
3. **Weak type safety.** The API receives `void**`, so `float*` and `int*` buffers are not distinguished. A wrong cast can compile and corrupt memory.

The C++ remedies are RAII, templates, and exceptions.

10.2 RAI: The Device Buffer

Primitive - RAI (Resource Acquisition Is Initialisation). A resource (here, a device allocation) is acquired in a constructor and released in the corresponding destructor. The language guarantees the destructor runs when the object dies, whether by scope exit, `return`, or exception, so the resource cannot leak.

```

#include <cuda_runtime.h>
#include <stdexcept>
#include <string>
#include <type_traits>

// Throw a std::runtime_error describing a failed CUDA call.
[[noreturn]] inline void throwCudaError(cudaError_t err, const char* what)
{
    throw std::runtime_error(std::string(what) + ": " +
                             cudaGetErrorString(err));
}

// RAII wrapper for a device allocation of T.
template <typename T>
class DeviceBuffer
{
public:
    // --- Construction: allocate on the device -----
    // static_assert: only trivially-copyable types may live in device memory
    // without custom copy semantics. This converts a runtime confusion into
    // a compile-time error.
    static_assert(std::is_trivially_copyable_v<T>,
                  "DeviceBuffer<T> requires a trivially copyable T");

    explicit DeviceBuffer(std::size_t count) : count_(count)
    {
        const cudaError_t err = cudaMalloc((void**)&ptr_, count_ * sizeof(T));
        if (err != cudaSuccess) throwCudaError(err, "cudaMalloc");
    }

    // --- No copying (a device buffer is a unique resource) -----
    DeviceBuffer(const DeviceBuffer&) = delete;
    DeviceBuffer& operator=(const DeviceBuffer&) = delete;

    // --- Move semantics: transfer ownership, never copy the bytes -----
    // After a move, the source is empty (nullptr). The destructor must
    // handle nullptr gracefully - hence the check in ~DeviceBuffer.
    DeviceBuffer(DeviceBuffer&& other) noexcept
        : ptr_(other.ptr_), count_(other.count_)
    {
        other.ptr_ = nullptr;    // source relinquishes the allocation
        other.count_ = 0;
    }

    DeviceBuffer& operator=(DeviceBuffer&& other) noexcept

```

```

{
    if (this != &other)
    {
        reset();           // release what we held
        ptr_ = other.ptr_; // take ownership
        count_ = other.count_;
        other.ptr_ = nullptr;
        other.count_ = 0;
    }
    return *this;
}

// --- Destruction: release the device allocation -----
~DeviceBuffer() { reset(); }

// --- Accessors -----
T*      data()      noexcept { return ptr_; }
const T* data() const noexcept { return ptr_; }
std::size_t size() const noexcept { return count_; }

// Host <-> device transfer helpers (explicit and checked).
void copyToDevice(const T* hostSrc)
{
    const cudaError_t err = cudaMemcpy(ptr_, hostSrc,
                                       count_ * sizeof(T),
                                       cudaMemcpyHostToDevice);
    if (err != cudaSuccess) throwCudaError(err, "cudaMemcpy H2D");
}

void copyToHost(T* hostDst) const
{
    const cudaError_t err = cudaMemcpy(hostDst, ptr_,
                                       count_ * sizeof(T),
                                       cudaMemcpyDeviceToHost);
    if (err != cudaSuccess) throwCudaError(err, "cudaMemcpy D2H");
}

private:
void reset() noexcept
{
    if (ptr_ != nullptr)
    {
        cudaFree(ptr_); // best-effort: destructors must not throw
        ptr_ = nullptr;
        count_ = 0;
    }
}

```

```

    }
}

T*      ptr_ = nullptr;    // device pointer
std::size_t count_ = 0;    // number of elements
};

```

The `static_assert` requires trivially copyable types because device memory is raw storage. Copying an object with internal pointers or virtual tables through device memory would corrupt it. The constraint moves a runtime hazard into a compile-time error.

The destructor is `noexcept` because destructors must not throw; throwing during stack unwinding would terminate the program. `cudaFree` in a destructor is best-effort. If a program must know about free failures, it should use an explicit `release()` method instead.

Copying a `DeviceBuffer` would duplicate the pointer, leaving two objects that both free the same allocation. Deleting the copy operations and retaining move semantics gives ownership like that of `std::unique_ptr`.

10.2.1 Usage

```

// Allocate 1M floats on the device:
DeviceBuffer<float> d_in(1 << 20), d_out(1 << 20);

// Copy from a host array:
std::vector<float> h_in(1 << 20, 1.0f);
d_in.copyToDevice(h_in.data());

// Launch a kernel (indexing unchanged from Chapter 3):
const int threads = 256;
const int blocks  = (static_cast<int>(d_in.size()) + threads - 1) / threads;
addVectors<<<blocks, threads>>>(d_in.data(), d_out.data(), d_in.size());
CHECK(cudaGetLastError());

// Copy back:
std::vector<float> h_out(d_out.size());
d_out.copyToHost(h_out.data());
// d_in and d_out are freed automatically at scope exit - no cudaFree calls.

```

The Chapter 3 program becomes shorter and its failure modes disappear. The kernel itself is unchanged because `DeviceBuffer::data()` returns the raw device pointer the kernel expects.

10.3 Templates: One Kernel, Many Types

CUDA supports C++ templates in device code. A kernel can be generic over its element type and its operation. The compiler instantiates the specialisations that the program uses:

```
// Generic elementwise transform. F is any callable (function object,
// lambda, function pointer) invocable as F(T) -> T. The compiler
// instantiates a separate device function for each (T, F) pair.
template <typename T, typename F>
__global__ void transformKernel(const T* in, T* out, int n, F f)
{
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) out[i] = f(in[i]);
}

// Host side: launch with a lambda. The lambda must be __device__-compatible;
// a captureless lambda is (it has no state to copy to the device).
void runTransform(const DeviceBuffer<float>& d_in, DeviceBuffer<float>& d_out,
                 int n)
{
    const int threads = 256;
    const int blocks = (n + threads - 1) / threads;
    transformKernel<<<blocks, threads>>>(d_in.data(), d_out.data(), n,
                                         [] (float x) { return x * 2.0f +
1.0f; }));
}
```

A captureless lambda is a legal kernel argument because the CUDA compiler lowers it to an empty struct with an `operator()`: a function object with no state. Passing it costs zero bytes, and the compiler inlines the call. A capturing lambda carries state that must be copied to the device as kernel arguments. Modern CUDA permits small, trivially copyable captures by value; references are not copied.

Templates have no runtime cost because the compiler emits the instantiations. The cost is compile time and binary size: each `(T, F)` pair is a separate kernel.

10.4 `__host__ __device__` Functions

A function qualified with both `__host__` and `__device__` is compiled twice from a single source, once for each side. This is how shared algorithm code is written:

```
// One definition, two compilations. On the host it is ordinary C++;
// on the device it becomes SASS. This function can be called from kernels
// and from host code, and the two sides produce identical results for
// identical inputs - a foundation for differential testing (Chapter 16).
__host__ __device__ inline float clampf(float v, float lo, float hi)
{
    return fminf(fmaxf(v, lo), hi);    // fminf/fmaxf exist on both sides
}
```

A `__device__` compilation cannot call host functions. A `__host__ __device__` function may therefore use only facilities available on both sides: the CUDA math library (`fminf`, `sqrtf`, `sinf`, ...), `constexpr` arithmetic, and plain C++. It cannot use `std::vector`, `new`, or I/O unless the calls are guarded by `#ifdef __CUDA_ARCH__`, which is defined only during device compilation:

```
__host__ __device__ float maybeLog(float x)
{
#ifdef __CUDA_ARCH__
    return logf(x);           // device path: CUDA math library
#else
    return std::log(x);       // host path: standard library
#endif
}
```

This dual compilation is the basis of CUDA's single-source style and of testable kernels: the same function can be exercised on the CPU and the GPU, and a discrepancy indicates a device-side bug (Chapter 16).

10.5 `constexpr` and `static_assert`

Kernel configuration such as tile sizes and unroll factors should be compile-time constants:

```
// Compile-time kernel configuration. These are typed values, not macros.
constexpr int kBlockSize = 256;
constexpr int kUnroll     = 4;
constexpr int kMaxDim     = 1 << 16;

// Compile-time sanity checks: configuration is validated when the file is
// compiled, not when the kernel runs.
static_assert(kBlockSize % 32 == 0,          "block size must be a warp
multiple");
static_assert(kUnroll >= 1 && kUnroll <= 8,  "unroll factor out of range");
static_assert(kMaxDim <= (1 << 20),         "dimension bound too large");
```

`constexpr` is preferable to `#define` because `constexpr` variables are typed, scoped, and can be used in `static_assert`. Macros are textual and untyped; a typo can become a confusing error at a distant use site.

10.6 CUDA 12 and the `cuda::` Namespace

CUDA 12.x continues to modernise the API. The `cuda::` C++ namespace, in headers such as `<cuda/atomic>`, provides safer alternatives (`cuda::stream_ref`, `cuda::event`, `cuda::memcpy_async`, `cuda::barrier`, `cuda::atomic`) with standard-library-compatible names and semantics:

```
#include <cuda/atomic>
// A CUDA-aware atomic that composes with std::atomic's memory-order model:
__device__ cuda::atomic<int, cuda::thread_scope_device> g_counter{0};
```

`cuda::atomic<T, cuda::thread_scope_device>` provides the `std::atomic` interface for device memory with scoped ordering. It is the preferred replacement for raw `atomicAdd` when acquire/release semantics are needed rather than relaxed increments.

The old C API remains fully supported and is the stable foundation taught in Chapters 3-6. New code should prefer the modern idioms where they exist.

10.7 C++20 Concepts

Templates are powerful; concepts make their errors legible. A constrained version of `transformKernel`:

```
#include <concepts>

// The transform operation must be an invocable mapping T to T.
template <typename T, std::invocable<T> F>
    requires std::same_as<std::invoke_result_t<F, T>, T>
__global__ void transformKernel(const T* in, T* out, int n, F f)
{
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) out[i] = f(in[i]);
}
```

Without the constraint, a wrong functor produces a deep error inside the kernel body. With the constraint, the compiler reports at the call site that `F` is not invocable as required. The cost is compile time; the benefit is that kernel templates scale to real codebases.

10.8 Style Summary

Old C-style habit	Modern replacement	Property gained
<code>cudaMalloc/cudaFree</code> by hand	<code>DeviceBuffer<T></code> RAI	No leaks, no double-free
<code>CHECK</code> macro discipline	Exceptions from a checked helper	Errors cannot be ignored
One kernel per type	Template kernels + lambdas	Reuse without copies
<code>#define TILE 16</code>	<code>constexpr int kTile = 16</code>	Typed, scoped, checkable
<code>atomicAdd</code> everywhere	<code>cuda::atomic</code> where ordering matters	Memory-model clarity
Untested device math	<code>__host__ __device__</code> + differential tests	Same code, both sides

Making the Compiler Enforce the Rules

The early chapters ask the programmer to follow rules by hand: check every CUDA call, match every allocation with a free, and never copy a device buffer. These rules are easy to state and easy to violate under refactoring or exception handling. Modern C++ moves the

rules into the type system so that violations become compile errors instead of runtime bugs.

“Never leak or double-free a device allocation” becomes `DeviceBuffer<T>`: the constructor allocates, the destructor frees, and deleted copy operations ensure that two objects cannot own the same pointer. “Only copy trivially copyable types through device memory” becomes a `static_assert` in the class. “Write generic kernels without duplicating code” becomes templates and lambdas. “Make configuration checkable” becomes `constexpr` constants used in `static_assert`.

This is the same philosophy used by safety-critical software: a type system makes illegal states unrepresentable. If a program cannot be written, it cannot fail at runtime. The parts that remain unprovable in C++ - index arithmetic, launch geometry - are exactly the parts later chapters isolate into explicit `unsafe` blocks (Chapter 13) or type-level constructs such as `DisjointSlice` (Chapter 14). The trajectory from raw `cudaMalloc` to Rust kernels is the same: move obligations from “remember to do this” to “the compiler will not let you do otherwise.”

Common Pitfalls

- Copying a `DeviceBuffer` by accident. Copy operations are deleted on purpose; use `std::move` to transfer ownership.
- Passing a capturing lambda with non-trivially-copyable state to a kernel. Captured state travels through the launch; keep it small and trivially copyable.
- Calling host-only facilities such as `std::vector` or I/O inside a `__device__` function. Use `#ifdef __CUDA_ARCH__` to separate paths.
- Letting exceptions cross the CUDA launch boundary without cleanup. RAI handles device memory, but other host state must also be exception-safe.

Check Your Understanding

Why must `DeviceBuffer` delete its copy constructor?

A copy would duplicate the pointer, producing two objects that both believe they own the same device allocation. Both destructors would call `cudaFree`, causing a double-free. Move semantics transfer the pointer and null the source, so only one owner remains.

What type would fail `static_assert(is_trivially_copyable_v<T>)`?

A type with a user-defined copy constructor, virtual functions, or internal pointers that need deep copying, such as `std::string`. Byte-copying it through device memory would duplicate or corrupt its internal state.

Why is a captureless lambda a legal kernel argument?

It lowers to an empty struct with an `operator()`: a function object with no state. Passing it costs zero bytes and the compiler inlines the call on the device. A capturing lambda is legal when its state is trivially copyable and small enough to travel through the launch.

Key Takeaways

- RAII (`DeviceBuffer<T>`) makes device-allocation leaks and double-frees impossible.
- `static_assert` moves invariants (trivially copyable, block sizes) into the type system.
- Templates and captureless lambdas give zero-cost generic kernels.
- **host device** compiles one function for both sides, the foundation of differential testing.
- Use `constexpr` for configuration and `cuda::atomic` for modern memory ordering.

10.9 Exercises

1. Why must `DeviceBuffer` delete its copy constructor? Trace the double-free that a copy would allow.
2. Give a concrete type that would fail `static_assert(std::is_trivially_copyable_v<T>, ...)` and explain what copying it through device memory would corrupt.
3. Write a `__host__ __device__` function `lerp(a, b, t)` and explain what it lets you test on the host that you could not test on the device alone.
4. When is a capturing lambda a legal kernel argument, and what constraint applies to the captured state?

Sources and Further Reading

- NVIDIA, *CUDA C++ Programming Guide*, “Programming Model” and “Memory Hierarchy”: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- NVIDIA, *CUDA C++ Best Practices Guide*, for resource management and performance guidance: <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>
- Bjarne Stroustrup, *The C++ Programming Language* and *The Design and Evolution of C++*, for RAI, value semantics, move semantics, and templates.

Chapter 11: The Library Ecosystem - Thrust, CUB & cuBLAS

Code companion: the complete, buildable code for this chapter lives in `code/ch11_library_examples/` in the repository.

Chapters 3-10 covered writing kernels. This chapter covers when not to. The CUDA ecosystem ships three libraries that implement, in production-tuned form, most of the algorithms of Chapters 8 and 9: **Thrust** (high-level algorithms), **CUB** (block-level primitives), and **cuBLAS** (dense linear algebra). A professional GPU engineer uses them first and writes custom kernels only where the libraries cannot express the problem. The earlier chapters are what you need to understand what the libraries do underneath.

11.1 The Case for Libraries

NVIDIA's libraries are tuned for every generation of GPU by engineers with access to the hardware design:

- They dispatch to **architecture-specific kernels**, including tensor cores for GEMM and custom shuffle reductions for scans.
- They are **hand-optimised** beyond what a compiler alone achieves.
- They are **tested** against known-good references and profiled on every release.

A hand-written SGEMM that reaches 70% of peak (Chapter 9) is good; cuBLAS often reaches about 95% of peak with tensor cores. The engineering decision is not “libraries or custom kernels”. It is “does my problem fit a library?”

11.2 Thrust: STL for the GPU

Thrust is the closest CUDA library to the C++ standard library. It provides containers such as `thrust::device_vector` and algorithms such as `transform`, `reduce`, `sort`, and `exclusive_scan` that operate on device memory with `std::`-like syntax:

```
#include <thrust/device_vector.h>
#include <thrust/transform.h>
#include <thrust/reduce.h>
#include <thrust/sequence.h>
#include <thrust/execution_policy.h>

// -----
// Thrust version of a Chapter 8 reduction and a Chapter 7 SAXPY. The
// algorithms dispatch to tuned kernels internally; the host code describes
// WHAT, not HOW.
// -----
void thrustExample(int n)
{
    // device_vector: an RAII device array (like Chapter 10's DeviceBuffer).
    thrust::device_vector<float> x(n), y(n);

    // thrust::sequence fills x with 0..n-1 (parallel, device-side).
    thrust::sequence(x.begin(), x.end());

    // thrust::transform applies a functor elementwise. thrust::device is the
    // execution policy that says "run on the GPU"; it must be the FIRST
    // argument (policy, first, last, result, op).
    thrust::transform(thrust::device,
                     x.begin(), x.end(), y.begin(),
                     [] __device__ (float v) { return v * 2.0f + 1.0f; });

    // thrust::reduce folds the array (the tuned reduction).
    const float total = thrust::reduce(y.begin(), y.end(), 0.0f,
                                       thrust::plus<float>());

    // thrust::sort, thrust::exclusive_scan, etc. follow the same shape.
    thrust::sort(y.begin(), y.end());
}
```

The lambda is marked `__device__` because Thrust must compile the functor for the device. Modern Thrust can infer `__host__ __device__` for plain lambdas, but the explicit qualifier makes the intent unambiguous and is the documented style.

`thrust::device_vector` and algorithm dispatch carry their own allocation and launch logic. For a one-off reduction of a large array, the overhead is noise. For a per-frame micro-pipeline in a tight loop, it is not. Measure (Chapter 16) before assuming.

11.3 CUB: Block-Level Primitives

Thrust works at the container level. **CUB** works at the block level: it provides `cub::BlockReduce`, `cub::BlockScan`, `cub::BlockHistogram`, and `cub::WarpReduce` for embedding in custom kernels. Use CUB when a kernel needs a block-sized reduction, scan, or histogram inside custom logic:

```
#include <cub/cub.cuh>

// A kernel that reduces its block's partial sums using CUB. CUB's BlockReduce
// is the tuned version of Chapter 8's reduceFull.
template <int BLOCK_THREADS>
__global__ void reduceWithCub(const float* in, float* out, int n)
{
    // CUB block-reduction scratch space (compile-time sized).
    typedef cub::BlockReduce<float, BLOCK_THREADS> BlockReduceT;
    __shared__ typename BlockReduceT::TempStorage temp_storage;

    // Coarsened accumulation (Chapter 8, 8.4):
    float sum = 0.0f;
    for (int i = blockIdx.x * BLOCK_THREADS + threadIdx.x;
         i < n; i += gridDim.x * BLOCK_THREADS)
        sum += in[i];

    // Block-wide reduction with CUB. Sum() is the "combine" operation;
    // cub::Sum is a device-side functor wrapping fadd.
    const float blockSum = BlockReduceT(temp_storage).Sum(sum);

    // Thread 0 of each block writes the block total:
    if (threadIdx.x == 0) out[blockIdx.x] = blockSum;
}
```

CUB's **BlockReduce** handles the edge cases a hand-written kernel would need to debug: non-power-of-two block sizes, the choice between shuffle-only and shuffle-plus-shared strategies, and architecture-specific tuning. The Chapter 8 kernel is the explanation; CUB is the implementation to ship.

CUB is header-only and template-heavy, so compile times grow and error messages can be intimidating. The runtime cost is zero because the templates compile to the same SASS as a hand-written kernel.

11.4 cuBLAS: Dense Linear Algebra

cuBLAS implements the BLAS (Basic Linear Algebra Subprograms) interface for GPUs: **sgemm**, **saxpy**, **sdot**, **sgemv**, and batched variants used by deep learning. Its API uses the classic handle-based C-library style:

```

#include <cublas_v2.h>

// -----
// C = alpha * A * B + beta * C   (the GEMM of Chapter 9, via cuBLAS)
// -----
void gemmViaCublas(const float* dA, const float* dB, float* dC,
                  int m, int n, int k, float alpha, float beta)
{
    // cuBLAS calls are not thread-safe by default; each context needs its own
    // handle. Create one handle per context and reuse it for all calls.
    cublasHandle_t handle;
    cublasCreate(&handle);

    // cuBLAS, like Fortran BLAS, is COLUMN-major: matrix columns are
    // contiguous. Row-major C (m x n) = A (m x k) * B (k x n) has the same
    // flat memory layout as column-major C^T = B^T * A^T, so the operands and
    // sizes are swapped and the transposed product is computed:
    //   cublasSgemm(..., n, m, k, ..., dB, ldb, dA, lda, ..., dC, ldc)
    // Each leading dimension is the number of rows in the column-major view:
    const int ldb = n;   // B is k x n row-major -> B^T is n x k col-major, ld
= n
    const int lda = k;   // A is m x k row-major -> A^T is k x m col-major, ld
= k
    const int ldc = n;   // C is m x n row-major -> C^T is n x m col-major, ld
= n

    // cuBLAS returns a status, not an exception. Check it:
    const cublasStatus_t status =
        cublasSgemm(handle,
                    CUBLAS_OP_N, CUBLAS_OP_N,    // no transposes; the swap
does the work
                    n, m, k,                    // SWAPPED sizes: compute C^T
= B^T * A^T
                    &alpha,
                    dB, ldb,                    // B first (it plays the "A"
role)
                    dA, lda,                    // A second (it plays the "B"
role)
                    &beta,
                    dC, ldc);
    if (status != CUBLAS_STATUS_SUCCESS)
        throw std::runtime_error("cublasSgemm failed");

    cublasDestroy(handle);
}

```

The handle carries per-context state: stream association, workspace, and heuristics. It lets the library keep state without global variables, which would break multi-context and multi-threaded programs. The handle's stream can be set with `cudaSetStream(handle, stream)` so cuBLAS calls participate in the pipeline of Chapter 6.

The column-major convention is the classic source of bugs. Row-major data must either be transposed (with `CUBLAS_OP_T`) or have its dimensions and operands swapped as shown above. When in doubt, verify with a 2x2 example before scaling up.

For large matrices, cuBLAS uses tensor cores and reaches 90%+ of peak with no optimisation effort, compared with roughly 70% for the hand-written Chapter 9 kernel. The Chapter 9 kernel's value is understanding; the cuBLAS call's value is shipping.

11.5 cuFFT and cuRAND

Two more libraries complete the common toolkit:

- **cuFFT** computes Fast Fourier Transforms in 1-D, 2-D, and 3-D, batched, complex, and real. Hand-written GPU FFTs are a research project; cuFFT is a product. Its API follows the FFTW planner model: create a plan describing the transform, then execute it on different data.
- **cuRAND** generates random numbers on the device with several generators (XORWOW, MRG32k3a, Philox, and others) and distributions (uniform, normal, Poisson). It can generate device-side, so kernels can draw random numbers internally for Monte Carlo work.

Both follow the handle/plan pattern: create once, configure, execute repeatedly.

11.6 Library or Custom Kernel?

Given a GPU problem, evaluate in order:

1. **Is it an algorithm in Thrust, CUB, cuBLAS, cuFFT, or cuRAND?** Use the library. The tuned, tested version beats a first custom kernel, and the saved time goes into profiling the parts that matter.

2. **Is the library call the bottleneck?** Profile it (Chapter 16). If it is, determine whether the data layout fits the library’s assumptions (leading dimensions, transposes). That is usually fixable without a custom kernel.
3. **Does the problem require custom per-element logic?** Write a custom kernel, but use CUB block primitives inside it. Custom does not mean from-scratch.
4. **Is the custom kernel measured as the hot path?** Only then hand-roll the full optimisation progression (Chapter 9).

This procedure prevents the reverse failure: rewriting `thrust::sort` because “it might be faster” while the real bottleneck sits in a poorly coalesced custom kernel nearby. Measure first; the library is the default.

Worked decision: normalise a 100M-float array. Normalisation is elementwise, so `thrust::transform` with a functor is the library answer. There is no custom logic to isolate, and no measured hot path yet. The correct move is one `thrust::transform` call, which reaches roughly 95% of bandwidth. A hand-tuned kernel would gain nothing because the transform is memory-bound and Thrust’s dispatcher already coalesces the access (Chapter 1).

Worked decision: a 7-tap separable blur per frame. No library call matches a stencil with a halo. The halo logic is custom, so write a custom kernel with coalesced row reads and clamped indices; Chapter 15’s `blurH` is exactly this shape. Only if the profiler shows that kernel as the pipeline bottleneck should shared-memory tiling (Chapter 7) be added.

In both cases the decision is driven by the algorithm’s shape and the profiler’s numbers, not by preference.

Libraries and Understanding

Using Thrust, CUB, or cuBLAS does not remove the need to understand reductions, scans, or GEMM. These libraries are tuned implementations of the algorithms studied in Chapters 8 and 9, and their value depends on understanding what they do underneath.

When `thrust::reduce` is called, the library solves the same reduction problem with the same strategies: tree reductions, warp shuffles, shared-memory tiles, and architecture-specific tuning. The difference is that the library version is tested and tuned. Understanding the algorithm makes it possible to predict when the library will be fast (large arrays, standard types) and when it may not be (tiny arrays, unusual layouts, per-frame allocation overhead).

The same applies to CUB. `cub::BlockReduce` is the production version of Chapter 8's `reduceFull` with edge cases already handled. Without the hand-written version, CUB's template errors and tuning knobs are difficult to read. With it, CUB is a tool that can be deployed confidently.

cuBLAS is the clearest example of why understanding matters. It is column-major because it inherits Fortran BLAS conventions. Feeding row-major data with `CUBLAS_OP_N` silently computes a transposed product. This is not a library bug; it is a mismatch between the caller's layout assumption and the library's definition. The dimension-swap trick in §11.4 works because transposition is understood mathematically.

The professional workflow is: know the algorithm, reach for the library, profile it, and hand-roll only when the profiler proves the library is the bottleneck.

Common Pitfalls

- Passing row-major data to cuBLAS with `CUBLAS_OP_N` and receiving the transposed answer. Transpose the operands or use the dimension-swap trick from §11.4.
- Using `thrust::device_vector` in a per-frame hot loop. Its convenience carries allocation and dispatch overhead; measure before assuming it is free.
- Reaching for CUB without understanding its template syntax. The errors are intimidating, but the pattern of temp storage plus `Sum()` is small once seen.
- Rewriting a library call “because it might be faster”. The decision procedure requires a profiler, not a hunch.

Check Your Understanding

Why does cuBLAS need leading dimensions?

Matrices can be sub-matrices (tiles) of larger buffers. `lda` tells cuBLAS how many elements separate the start of one row or column from the next, so it can walk a sub-matrix correctly instead of assuming full density.

What does CUB's BlockReduce provide beyond Chapter 8's `reduceFull`?

CUB handles non-power-of-two block sizes, architecture-specific tuning, and edge cases the hand-written kernel would need to debug. The Chapter 8 kernel is the explanation; CUB is the tested implementation.

When should `thrust::sort` be replaced with a custom radix sort?

Only when profiling shows `thrust::sort` is a significant fraction of runtime and the data or keys have properties a radix sort can exploit, such as fixed-size keys or a known range. Measure sort time and end-to-end time before and after.

Key Takeaways

- Use tuned, tested libraries first: Thrust (algorithms), CUB (block primitives), cuBLAS (dense linear algebra).
- Thrust mirrors the STL: `device_vector`, `transform`, `reduce`, `sort`, `scans`.
- CUB slots into custom kernels: `cub::BlockReduce`, `cub::BlockScan`, `cub::BlockHistogram`.
- cuBLAS is handle-based and column-major; the transpose trap is the classic bug.
- Replace a library call only when the profiler proves it is the bottleneck.

11.7 Exercises

1. Rewrite the Chapter 8 privatised histogram using `cub::BlockHistogram` inside a custom kernel. What does CUB provide that §8.8 had to implement?
2. Why does cuBLAS need `lda`, `ldb`, and `ldc`? What would break if it assumed full density?
3. Explain the column-major trap with a concrete 2x2 example: what does `cublasSgemm` return if row-major A and B are fed with `CUBLAS_OP_N`?
4. Under what measurable condition would you replace a `thrust::sort` with a custom radix sort? List the two measurements you would take first.

Sources and Further Reading

- NVIDIA, *CUDA C++ Programming Guide*, “Thrust” and library sections: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- NVIDIA, *Thrust Quick Start Guide*: <https://docs.nvidia.com/cuda/thrust/>
- NVIDIA, *CUB documentation*: <https://nvidia.github.io/cccl/>
- NVIDIA, *cuBLAS documentation*: <https://docs.nvidia.com/cuda/cublas/>
- NVIDIA, *CUDA C++ Best Practices Guide*, “Use Optimized Libraries”: <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>

Chapter 12: NVRTC, Runtime Compilation & the Driver API

Code companion: the complete, buildable code for this chapter lives in `code/ch12_nvrtc/` in the repository.

Everything so far has used the **CUDA runtime API** - `cudaMalloc`, `cudaMemcpy`, and `cudaStreamCreate` - and the offline toolchain (`nvcc` to PTX to SASS, Chapter 3). This chapter introduces the **driver API**, which exposes lower-level objects (contexts, modules, and kernels), and **NVRTC** (NVIDIA Runtime Compilation), which compiles CUDA source at run time inside a program. Together they enable JIT compilation, user-supplied kernels, and code generated from runtime parameters.

12.1 The Two APIs

Primitive - runtime API. The high-level `cuda*` functions used in Chapters 3-6. It initialises a context implicitly, manages device memory and streams, and launches kernels by name at compile time. **Primitive - driver API.** The low-level `cu*` functions. The program creates contexts explicitly, loads modules of compiled kernels, extracts kernel handles, and launches them with a raw parameter array.

The runtime API is implemented on top of the driver API. Every `cudaMalloc` has a `cuMemAlloc` equivalent, and every `kernel<<<>>>` launch corresponds to a `cuLaunchKernel`. The runtime is more convenient. The driver is more explicit, and it is the only API that can launch kernels that did not exist when the program was compiled.

12.2 PTX, cubin, and fatbin

The offline pipeline produces PTX and SASS (Chapter 3). The artefacts have names:

Primitive - PTX. The portable virtual ISA. It is architecture-independent within CUDA's versioning and is compiled to SASS by the driver at load time. **Primitive - cubin.** A CUDA binary: SASS for one specific compute capability, produced by `ptxas`. **Primitive - fatbin.** A container that bundles multiple cubins and PTX for different architectures so that one executable runs on many GPUs. When code is compiled with `nvcc -arch=sm_90`, the host binary embeds a fatbin.

`nvcc` produces all of these; `cuobjdump` and `nvdiasm` inspect them. The important property for this chapter is that **PTX is text**: it can be generated, examined, and even written by hand. NVRTC produces PTX at run time; the driver loads it.

12.3 NVRTC: Compiling CUDA Source in a Program

NVRTC compiles a CUDA source string to PTX at run time:

```

#include <nVRTC.h>
#include <cuda.h>           // the driver API
#include <string>
#include <vector>
#include <stdexcept>

// -----
// Compile the given CUDA source to PTX using NVRTC.
// Returns the PTX as a string. Throws std::runtime_error on failure with
// the compiler's log (which is where the kernel's errors appear).
// -----
std::string compileToPtx(const char* source, const char* name)
{
    // 1. Create an NVRTC program from the source text.
    nVRTCProgram prog;
    nVRTCResult res = nVRTCCreateProgram(&prog, source, name, 0, nullptr,
                                         nullptr);
    if (res != NVRTC_SUCCESS) throw std::runtime_error("nVRTCCreateProgram");

    // 2. Compile. Options are passed as strings, exactly like nvcc flags.
    const char* options[] = {"-arch=compute_60", "-std=c++17"};
    res = nVRTCCompileProgram(prog, 2, options);

    // 3. On failure, fetch the compilation log and report it.
    if (res != NVRTC_SUCCESS)
    {
        size_t logSize = 0;
        nVRTCGetProgramLogSize(prog, &logSize);
        std::vector<char> log(logSize);
        nVRTCGetProgramLog(prog, log.data());
        nVRTCDestroyProgram(&prog);
        throw std::runtime_error(std::string("NVRTC compile failed:\n") +
                                log.data());
    }

    // 4. Fetch the PTX text.
    size_t ptxSize = 0;
    nVRTCGetPTXSize(prog, &ptxSize);
    std::vector<char> ptx(ptxSize);
    nVRTCGetPTX(prog, ptx.data());
    nVRTCDestroyProgram(&prog);
    return std::string(ptx.data(), ptxSize);
}

```

The `-arch=compute_60` option makes NVRTC compile for a virtual architecture. The driver later JIT-compiles the PTX to the actual SASS of the installed GPU. Pascal-class `compute_60` PTX runs on every CUDA 12.x GPU through the driver's forward-compatible JIT. A higher virtual architecture such as `compute_90` targets Hopper-class GPUs and may produce better SASS there, at the cost of portability.

Runtime compilation is useful for three reasons:

1. **User-supplied code.** The program accepts kernels as strings, the pattern behind JIT-based DSLs and kernel playground tools.
2. **Runtime-specialised code generation.** A solver can generate a kernel with unrolling and constants specialised to the runtime problem size, which a generic precompiled kernel cannot do.
3. **Deployment simplicity.** PTX can be shipped instead of per-architecture fatbins; the driver JIT-compiles at first use.

The cost is compile time at run time, typically hundreds of milliseconds, plus the complexity of the two-stage load.

12.4 The Driver API: Loading and Launching

Given PTX, the driver API turns it into an executable kernel:

```

// -----
// Load PTX text into the current driver context and launch a kernel
// "addVectors" with the given grid/block shape and raw parameters.
// -----
void launchFromPtx(const std::string& ptx,
                  const float* d_a, const float* d_b, float* d_c, int n)
{
    // 1. Initialise the driver API (idempotent).
    cuInit(0);

    // 2. Create a context on device 0. The driver API has no implicit
context;
    // this is the "explicit" part of the driver API. Production code that
    // uses the runtime API should use the primary/current context instead
of
    // creating a second one (see 12.6).
    CUdevice device;
    CUcontext context;
    cuDeviceGet(&device, 0);
    cuCtxCreate(&context, 0, device);

    // 3. Load the PTX into a MODULE: a collection of compiled kernels.
    CUmodule module;
    CUresult res = cuModuleLoadData(&module, ptx.c_str());
    if (res != CUDA_SUCCESS) throw std::runtime_error("cuModuleLoadData");

    // 4. Get a handle to the kernel by NAME. The kernel must exist in the
    // PTX with that exact name. A plain __global__ function named
    // addVectors is stored as "addVectors" (C++ name mangling applies only
    // to non-C-linkage functions).
    CUfunction kernel;
    res = cuModuleGetFunction(&kernel, module, "addVectors");
    if (res != CUDA_SUCCESS) throw std::runtime_error("cuModuleGetFunction");

    // 5. Package the kernel arguments. The driver API takes a raw array of
    // POINTERS TO the arguments - hence the address-of dance below.
    void* args[] = { &d_a, &d_b, &d_c, &n };

    // 6. Launch. Grid and block dimensions, sharedMemBytes, stream, kernel,
    // and args. The grid must cover all n elements.
    const int threads = 256;
    const int blocks = (n + threads - 1) / threads;
    res = cuLaunchKernel(kernel,
                        blocks, 1, 1,          // grid
                        threads, 1, 1,         // block

```

```

                                0, nullptr,                // no dynamic shared, default
stream
                                args, nullptr);           // arguments, no extra options
    if (res != CUDA_SUCCESS) throw std::runtime_error("cuLaunchKernel");

    cuCtxSynchronize();

    // 7. Destroy the module and context in a long-running process. For a
    //     kernel compiled once and replayed many times, keep both alive.
    cuModuleUnload(module);
    cuCtxDestroy(context);
}

```

The `args` array is a `void**` where each element points to the storage of one kernel argument. For a `float*` parameter `d_a`, the storage is the pointer variable, so the code passes `&d_a`. Passing `d_a` directly is the classic driver-API crash: the driver reads `args[i]`, interprets the pointer value as the argument's bytes, and loads the wrong data.

The driver API requires explicit context management because the runtime's implicit context is hidden. Contexts control resource lifetime and allow interoperability with libraries; the price is the ceremony above.

12.5 The JIT Cache

The first use of a module pays the PTX-to-SASS JIT compile. Subsequent loads of the same PTX in the same process reuse the driver's in-memory cache. Across processes, the driver persists compiled binaries in `~/.nv/ComputeCache`. The cache can be controlled with environment variables:

```

export CUDA_CACHE_MAXSIZE=1073741824    # 1 GB on-disk cache
export CUDA_CACHE_DISABLE=0              # 0 = enabled

```

NVRTC compiles source to PTX in the process. The driver compiles PTX to SASS and caches that result. A long-running server should compile once at startup, keep the module alive for the process lifetime, and never recompile per request.

12.6 Runtime API + Driver API: The Hybrid

The two APIs can coexist. The runtime manages memory and streams; the driver launches JIT-compiled kernels. The bridge is the **current context**: the runtime's implicit context is also the driver's current context, so device pointers obtained from `cudaMalloc` remain valid for `cuLaunchKernel` in the same thread. This hybrid - `cudaMalloc` for memory, NVRTC plus driver for the kernel - is the pattern used by the capstone in Chapter 15.

What the Runtime API Automates

The runtime API is convenient because it makes decisions implicitly: it creates a context lazily, loads modules, manages memory, and hides launch plumbing. The driver API exposes those decisions. Contexts are explicit objects. Modules are containers of compiled kernels loaded from PTX or cubin data. Kernel arguments are not type-checked by the compiler; they are packaged into a raw array of pointers and handed to `cuLaunchKernel`.

This exposure is necessary for code that did not exist at compile time. NVRTC takes CUDA source as a string, compiles it to PTX, and returns the text. The driver loads that PTX, JIT-compiled it to SASS for the installed GPU, and returns a function handle. PTX is architecture-independent, so the same text can become SASS for a T4, an A100, or an H100.

The most common driver-API bug also illustrates the model. `cuLaunchKernel` receives `void** args`, an array where each element is a pointer to the storage of one kernel argument. For a `float*` parameter `d_a`, the storage is the local pointer variable, so the code passes `&d_a`. If `d_a` is passed instead, the driver interprets the pointer value itself as the argument's bytes and typically crashes. The driver API does not know the kernel signature, so the program must describe where every argument lives. The type safety provided by the runtime API is replaced here by a precise convention.

The hybrid pattern is the pragmatic synthesis: runtime API for memory and streams, driver API for the kernel. Because both share the current context, `cudaMalloc` pointers remain valid when passed to `cuLaunchKernel`.

Common Pitfalls

- Passing `d_a` instead of `&d_a` in the `args` array. The driver treats each `args[i]` as a pointer to the argument's storage; passing the pointer value directly makes it read the pointer's bytes as the argument.
- Ignoring the NVRTC compile log. A failed compile reports the problem, but only if the log is fetched and printed.
- Creating a second context when the runtime already has one. Use the current or primary context; a separate context can break pointer validity.
- Forgetting that PTX for `compute_90` does not run on older GPUs. Choose a virtual architecture that matches the deployment range.

Check Your Understanding

Why must `args[i]` be the address of the argument?

`cuLaunchKernel` receives a `void**` in which each element is a pointer to the argument's storage. For a `float*` parameter `d_a`, the storage is the local pointer variable, so the driver needs `&d_a`. Passing `d_a` makes the driver read the pointer value as the argument's bytes, which loads wrong data and typically crashes.

What is the difference between PTX and cubin?

PTX is portable virtual ISA text. It is architecture-independent and can be JIT-compiled by the driver for the installed GPU. A cubin is SASS for one specific compute capability and cannot run on another architecture without recompilation.

Why is the hybrid (runtime memory + driver kernel) pattern useful?

It keeps the convenient runtime API for allocations and copies while using the driver API for the one thing the runtime cannot do: launching a kernel compiled at run time. Both share the same current context, so device pointers remain valid across the boundary.

Key Takeaways

- The runtime API (`cuda*`) is built on the driver API (`cu*`); the driver is the only way to launch kernels that did not exist at compile time.
- PTX is portable text; cubin is SASS for one architecture; fatbin bundles several.
- NVRTC compiles a source string to PTX at run time; errors appear in the compilation log.
- `cuLaunchKernel` takes a raw array of pointers-to-arguments; passing the pointer instead of its address is the classic crash.
- The JIT cache (`CUDA_CACHE_MAXSIZE`) and module reuse make runtime compilation production-viable.

12.7 Exercises

1. List the three artefacts produced by the offline toolchain and where each is compiled: host toolchain, `ptxas`, or the driver at load time.
2. In `launchFromPtx`, why is the argument for a `float*` parameter `&d_a` and not `d_a`? What exactly does the driver read from `args[i]`?
3. A server receives kernel source from users. Argue for or against caching compiled PTX keyed by a hash of the source, and name the two caches involved.
4. When would you choose `-arch=compute_60` over `-arch=compute_90` for NVRTC, and what does each choice cost?

Sources and Further Reading

- NVIDIA, *CUDA Runtime API and Driver API* reference: <https://docs.nvidia.com/cuda/cuda-runtime-api/> and <https://docs.nvidia.com/cuda/cuda-driver-api/>
- NVIDIA, *NVRTC User Guide*: <https://docs.nvidia.com/cuda/nvrtc/>
- NVIDIA, *CUDA C++ Programming Guide*, “Just-in-Time Compilation” and “Compute Capabilities”: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>

Chapter 13: Rust Meets the GPU

Code companion: the complete, buildable code for this chapter lives in `code/ch13_rust_vector_add/` in the repository.

This part of the book changes language but not hardware. The GPU remains the machine of Chapter 2, and the kernels of Chapters 3-12 still run on it. What changes is the host: Rust replaces C++ for allocating device memory, moving data, and launching kernels. This chapter covers the Rust CUDA ecosystem (`rustcuda` and `cudarc`) and a complete Rust host program, with the safety properties Rust provides at each step.

13.1 Rust on the Host

The GPU failure modes discussed in earlier chapters are often host-side failures first: a leak is a missing `cudaFree`, a use-after-free is a dangling device pointer, and a race is usually launched from the host. Rust’s ownership system addresses these directly:

- **Ownership and lifetimes.** A `CudaSlice<T>` owns its device allocation. When it is dropped, the allocation is freed. Double-frees and leaks become type errors rather than runtime incidents.
- **No data races by construction.** The borrow checker prevents two mutable references to the same buffer from existing simultaneously, a guarantee the C++ compiler does not offer.
- **Result-based errors.** CUDA’s `cudaError_t` becomes a typed `Result<T, CudaError>`. Ignoring an error is a compile-time warning through `must_use`, not silent misbehaviour.

The cost is the one Rust charges everywhere: the borrow checker rejects code it cannot prove safe, and the FFI boundary, where `unsafe` lives, must be drawn precisely. This chapter is about drawing that boundary well.

13.2 The Ecosystem: `rustacuda` and `cudarc`

Two host libraries dominate:

- `rustacuda` - an older wrapper over the CUDA driver API. It provides safe modules for contexts, modules, functions, streams, and memory. It is historically important and now largely superseded for new work.
- `cudarc` - the actively maintained wrapper around the driver API (`cudarc::driver`), NVRTC (`cudarc::nVRTC`), and the CUDA libraries (cuBLAS, cuDNN, cuFFT, cuRAND, NCCL). It exposes three layers per wrapper: `safe` (high-level, checked), `result` (thin, returns error codes), and `sys` (raw FFI). This book uses `cudarc`.

CUDA-Oxide (Chapter 14) is different in kind: not a wrapper around CUDA C++, but a compiler that turns Rust kernels into PTX. This chapter uses `cudarc` to drive existing kernels compiled from C++.

13.3 The Kernel, Compiled Ahead of Time

The Chapter 3 vector-add kernel is reused and compiled to PTX with `nvcc`:

```
// kernels/vector_add.cu
// Compiled once, ahead of time, to PTX:
//   nvcc -arch=compute_60 -ptx kernels/vector_add.cu -o vector_add.ptx
// compute_60 PTX runs on any CUDA 12.x GPU (Pascal or newer) via the
// driver's JIT; use compute_87 on Jetson Orin, compute_80 on A100, or
// compute_90 on H100 for native SASS.
extern "C" __global__ void vector_add(const float* a, const float* b,
                                     float* c, int n)
{
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) c[i] = a[i] + b[i];
}
```

CUDA C++ mangles kernel names like any C++ symbol. The driver API loads kernels by name (Chapter 12), so `extern "C"` guarantees the module symbol is literally `vector_add`. The Rust side can then look it up without demangling.

13.4 The Rust Host Program

```
// main.rs - Rust host driving the vector_add kernel via cudarc.
// Requires: CUDA toolkit installed (for the driver and nvrtc), and the
// vector_add.ptx file in the working directory (or embedded; see 13.5).

use cudarc::driver::{CudaDevice, CudaSlice, LaunchAsync, LaunchConfig};
use cudarc::nvrtc::Ptx; // wraps PTX source: Ptx::from_file or Ptx::from_src

fn main() -> Result<(), Box<dyn std::error::Error>> {
    // --- Device handle -----
    // CudaDevice::new(0) opens the first GPU, initialising the driver and
    // creating the CUDA context. It returns Result: no GPU -> Err here,
    // reported as a typed error instead of a crash.
    let dev = CudaDevice::new(0)?;

    // --- Problem size -----
    const N: usize = 1 << 20; // 1,048,576 elements
    let n32: i32 = N as i32; // kernel expects a 32-bit int

    // --- Host data -----
    let a: Vec<f32> = (0..N).map(|i| i as f32).collect();
    let b: Vec<f32> = (0..N).map(|i| 2.0 * i as f32).collect();

    // --- Device allocation -----
    // alloc_zeros allocates device memory and zero-initialises it. The
    // returned CudaSlice<f32> OWNS the allocation: dropping it frees it.
    // No cudaFree call appears in this program.
    let mut d_a: CudaSlice<f32> = dev.alloc_zeros::<f32>(N)?;
    let mut d_b: CudaSlice<f32> = dev.alloc_zeros::<f32>(N)?;
    let mut d_c: CudaSlice<f32> = dev.alloc_zeros::<f32>(N)?;

    // --- Host -> device copies -----
    // htdo_copy_into takes ownership of the host Vec because the copy is
    // queued on the device's stream; no other code can mutate the buffer
    while
        // it is in flight. We pass clones so the originals remain usable for
        // verification below. The &mut destination expresses that the copy
        mutates
            // the device buffer, and Rust requires exclusive access to do so.
            dev.htdo_copy_into(a.clone(), &mut d_a)?;
            dev.htdo_copy_into(b.clone(), &mut d_b)?;

    // --- Load the PTX and fetch the kernel handle
```

```

-----
// load_ptx loads the module and registers the named kernel. The PTX is
// wrapped in a Ptx: Ptx::from_file for a path, Ptx::from_src for an
// embedded string (13.5). Missing files and missing symbols surface as
// Results.
let module = "vector_add";
dev.load_ptx(Ptx::from_file("vector_add.ptx"), module, &["vector_add"])?;
let f = dev.get_func(module, "vector_add")
    .ok_or("kernel 'vector_add' not found in the loaded module"?;

// --- Launch
-----

// The launch is marked unsafe: the configuration (grid/block shape) and
// the argument tuple must match the kernel's real signature and index
// space. cudarc checks argument arity and types at the type level but
// cannot verify the kernel's internal assumptions - hence the SAFETY
// comment.
//
// SAFETY: the grid covers N threads (LaunchConfig::for_num_elems rounds
up
// to whole warps), the kernel guards with `if (i < n)`, and the argument
// tuple (&d_a, &d_b, &mut d_c, n32) matches the extern "C" signature.
unsafe {
    f.launch(LaunchConfig::for_num_elems(N as u32),
              (&d_a, &d_b, &mut d_c, n32))
};

// --- Device -> host copy
-----

// dtoh_sync_copy blocks until the stream's work completes and copies the
// result back. The ? propagates any device error encountered.
let c: Vec<f32> = dev.dtoh_sync_copy(&d_c)?;

// --- Verify
-----

let max_err = c.iter().zip(a.iter().zip(b.iter()))
    .map(|(c, (a, b))| (c - (a + b)).abs())
    .fold(0.0f32, f32::max);
println!("max error = {max_err}");

// d_a, d_b, d_c are dropped here; their destructors free the allocations.
// The device handle's context is cleaned up on drop.
Ok(())
}

```

Safety ledger. The only `unsafe` block is the launch. The type system checks the arity and types of the argument tuple `(&d_a, &d_b, &mut d_c, n32)`, but not the semantics: that the kernel's index arithmetic matches the launch configuration, or that `n32` matches the kernel's `int n`. The `SAFETY` comment states the invariants a reviewer must check, the same contract Chapter 3 expressed as comments in C++.

Everything else, allocation, copies, and module loading, is a safe API: ownership guarantees lifetimes, and `Result` guarantees error handling.

What is not solved. The kernel is still C++ and still unsafe by construction: an out-of-bounds write inside `vector_add` corrupts whatever it corrupts, and Rust cannot observe it. Rust secures the host, not the device. CUDA-Oxide (Chapter 14) addresses the device side.

13.5 Embedding the PTX

A filesystem dependency is fragile in production. The PTX can be embedded in the binary at compile time:

```
#![allow(unused)]
fn main() {
    // build.rs (or a const in the crate) embeds the PTX text.
    const VECTOR_ADD_PTX: &str = include_str!("vector_add.ptx");

    // Load directly from the embedded string instead of the filesystem.
    // Ptx::from_src wraps the string; Ptx::from_file wraps a path (13.4).
    dev.load_ptx(Ptx::from_src(VECTOR_ADD_PTX), module, &["vector_add"])?;
}
```

`include_str!` inlines the file at compile time. The binary is self-contained, and the kernel cannot go missing during deployment. The capstone uses this pattern.

13.6 Comparing with the C++ Host

Concern	C++ (Chapters 3-6)	Rust + cudarc
Allocation lifetime	Manual <code>cudaMalloc/cudaFree</code>	<code>CudaSlice</code> RAI on drop
Copy direction	<code>cudaMemcpy</code> with direction enum	Typed <code>htod_copy_into</code> / <code>dtoh_sync_copy</code>
Error handling	<code>CHECK</code> macro discipline	Typed <code>Result</code> with <code>?</code>
Kernel launch	<code><<<>></code> , unchecked args	<code>unsafe</code> launch with typed args + SAFETY comment
Data races	Compiler silent	Borrow checker rejects at compile time
Device-side safety	No help	No help (until CUDA-Oxide)

Each right-hand entry removes a bug class that the left-hand column required a discipline or a runtime tool to catch. The price, the `unsafe` block and its `SAFETY` comment, is explicit and small.

13.7 Lifetimes in Action

The claims in §13.1 have concrete demonstrations. The borrow checker rejects whole classes of GPU programs at compile time:

```

#![allow(unused)]
fn main() {
// What the borrow checker prevents (none of these compile):

// 1. Two mutable borrows of the same device buffer. The launch tuple below
//    needs two &mut to d_c, which Rust forbids:
//    f.launch(config, (&mut d_c, &mut d_c)) // ERROR: cannot borrow twice
//    In C++ this compiles and produces a race inside the kernel.

// 2. Using a buffer after moving it. The CudaSlice is gone after the move, so
//    any use is a compile error rather than a use-after-free:
//    let stolen = d_a; // d_a is moved into stolen
//    dev.htod_copy_into(&a, &mut d_a); // ERROR: borrow of moved value
//    In C++, the equivalent sequence (copying a raw pointer, then freeing it)
//    is a use-after-free that requires Compute Sanitizer to catch at run
//    time.

// 3. Passing a host Vec where a device slice is expected. The types do not
//    match, so the error is at the call site:
//    dev.htod_copy_into(&host_vec, &mut d_c); // ERROR: type mismatch
}

```

Each rejected program is a bug class that the C++ chapters required a runtime tool (Compute Sanitizer, Chapter 16) or a discipline (the `CHECK` macro, Chapter 3) to catch. Rust moves detection to the compiler, which runs earlier and cannot be forgotten. The borrow checker is the `CHECK` macro promoted to a compile-time guarantee.

The Unsafe Boundary Is an Explicit Contract

The design decision in this chapter is where the unsafe boundary is drawn. Rust's safety guarantees apply only to code within the rules of ownership and borrowing. A CUDA kernel launch sits at the edge of those rules because the host cannot see inside the kernel and therefore cannot prove that the kernel's index arithmetic matches the launch configuration. That unprovable obligation is what the `unsafe` block marks.

Before the launch, the guarantees are compile-time. `CudaDevice::new(0)` returns a `Result`, so a missing GPU must be handled. `alloc_zeros` returns an owned `CudaSlice`, so the allocation is freed when the slice is dropped. `htod_copy_into` takes ownership of the host vector, so the data cannot be mutated while the copy is in flight. The launch requires `&mut`

`d_c` for the output buffer, so two kernels cannot hold mutable references to the same buffer simultaneously.

The launch is where the type system runs out. The tuple `(&d_a, &d_b, &mut d_c, n32)` has the right arity and types, but nothing in the types proves that the grid covers exactly `N` elements, that the kernel's `if (i < n)` guard matches, or that `n32` is the correct interpretation of the kernel's `int n`. The **SAFETY** comment is the contract that fills the gap: a reviewer must be able to verify those facts from the comment and the surrounding context. This is the same contract Chapter 3 expressed as C++ comments. Rust makes everything around the launch enforceable by the compiler, leaving one small, documented, auditable seam instead of a program-wide discipline.

Common Pitfalls

- Forgetting `extern "C"` on the kernel. The driver looks up kernels by unmangled name; without it, `get_func(module, "vector_add")` fails.
- Copying a `CudaSlice` instead of moving it. Device buffers are unique resources; use `&mut` and moves, not copies.
- Putting `unsafe` around the whole program instead of just the launch. The purpose is to isolate the unprovable part, not to annotate everything.
- Ignoring the **SAFETY** comment. If the kernel-side assumptions cannot be stated, the launch is not known to be correct.

Check Your Understanding

Why does `extern "C"` matter?

C++ mangles function names, for example `_Z11vector_add....`. The CUDA driver loads kernels by exact string name, so `extern "C"` guarantees the symbol is literally `vector_add`, letting the Rust host look it up without demangling.

Why is `&mut d_c` required in the launch tuple?

The kernel writes through the `c` pointer. Rust requires exclusive access for mutation, so the launch needs `&mut d_c`; passing `&d_c` would be a compile error because shared references

cannot be used for mutation. The borrow checker also prevents two kernels from holding `&mut` to the same buffer at once.

What does `include_str!` provide over `Ptx::from_file`?

`include_str!` embeds the PTX text into the binary at compile time. The binary is self-contained and cannot lose the kernel file during deployment. `Ptx::from_file` reads the filesystem at run time, which is simpler for experimentation but fragile in production.

Key Takeaways

- Rust secures the host: ownership prevents leaks and double-frees, the borrow checker prevents data races, and Result prevents ignored errors.
- `cudarc` provides RAII device memory (`CudaSlice`), typed copies, and module loading from PTX.
- `extern "C"` keeps kernel symbols unmangled so the driver can find them by name.
- The unsafe launch is explicit; the SAFETY comment states the kernel-side obligations the type system cannot check.
- `include_str!` embeds PTX in the binary, removing the filesystem dependency.

13.8 Exercises

1. Explain why `extern "C"` on the kernel matters for `dev.get_func(module, "vector_add")`. What would happen without it?
2. The launch is wrapped in `unsafe`. List three kernel-side assumptions that the SAFETY comment must document.
3. Trace the lifetimes: why is `&mut d_c` (not `&d_c`) required in the launch tuple, and what does the borrow checker prevent?
4. Compare `include_str!` with a runtime filesystem read. When is the filesystem version the right choice?

Sources and Further Reading

- NVIDIA, *CUDA C++ Programming Guide*: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- The Rust Book, “Ownership” and “Unsafe Rust”: <https://doc.rust-lang.org/book/>
- `rustacuda` crate documentation: <https://docs.rs/rustacuda/>
- `cudarc` crate documentation: <https://docs.rs/cudarc/>

Chapter 14: CUDA-Oxide - Kernels in Pure Rust

Code companion: the complete, buildable code for this chapter lives in `code/ch14_cuda_oxide/` in the repository.

Chapter 13 secured the host with Rust, but the kernel remained C++ compiled by `nvcc` and invoked through an `unsafe` boundary. **CUDA-Oxide** is NVIDIA Labs' experimental answer to the remaining gap: a `rustc` codegen backend that compiles idiomatic Rust kernels directly to PTX. There is no DSL, no foreign-language binding, and no `nvcc`; host and device code can live in the same file. This chapter describes the project as documented by its repository, including its pipeline and the parts that remain experimental.

14.1 What CUDA-Oxide Is

CUDA-Oxide (repository `NVlabs/cuda-oxide`, announced May 2026) is described by its authors as:

"An experimental Rust-to-CUDA compiler that lets you write SIMT GPU kernels in safe(ish), idiomatic Rust. It compiles standard Rust code directly to PTX - no DSLs, no foreign language bindings, just Rust."

Its design goals, from the project documentation:

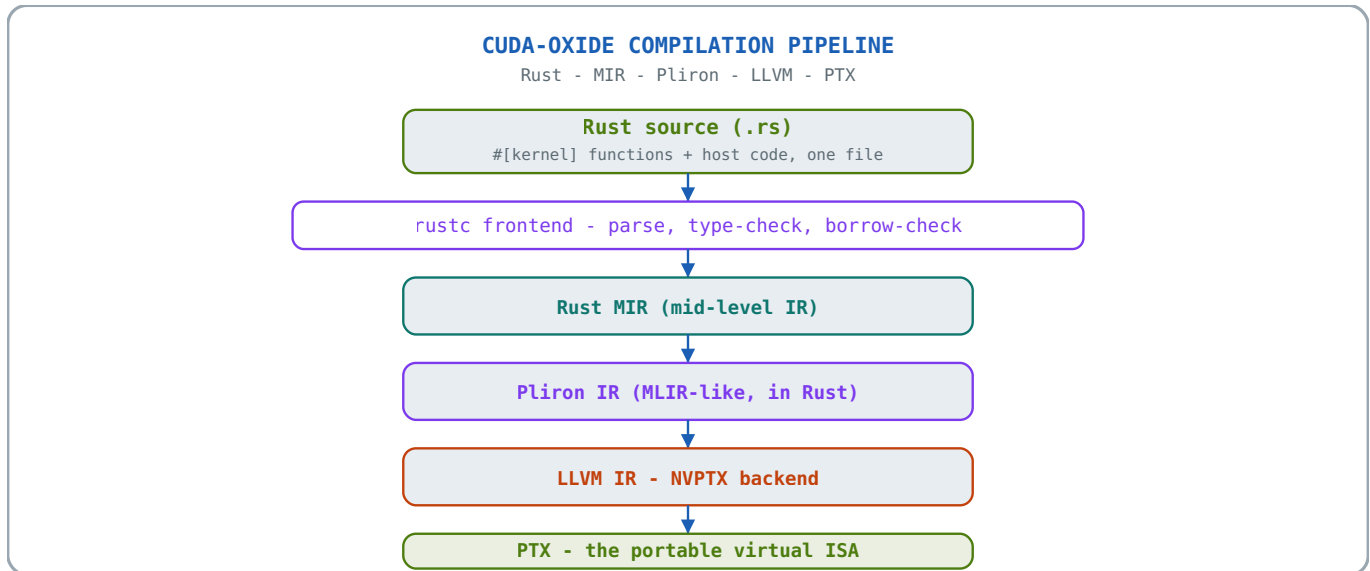
- **Single-source compilation.** Host and device code live in the same file and are built with one command, `cargo oxide build`.
- **A `rustc` codegen backend** that compiles `#[kernel]` functions to PTX.
- **Device-side abstractions:** type-safe indexing, shared memory, scoped atomics, barriers, TMA, and warp or cluster operations.

- **Compile-time kernel policies** for separate tuned specialisations without runtime policy arguments.
- **A host-side runtime** (`cuda-core`, `cuda-async`) for memory management, pinned host transfers, and kernel launching.

The project’s own term “safe(ish)” matters. CUDA-Oxide keeps Rust’s type system and ownership on the device, but SIMT programming includes operations, such as raw launch configuration and memory ordering, that cannot yet be fully proven safe. The project is explicit about the boundary.

14.2 The Compilation Pipeline

CUDA-Oxide does not translate Rust to CUDA C. It reuses `rustc`’s internal representations and replaces only the code generation stage:



Because the front end is real `rustc`, ownership, borrowing, pattern matching, and traits are checked before GPU code is generated. A kernel that violates the borrow checker never becomes PTX. The experimental part is the back end: Pliron is a young framework, and the project warns that lowering to LLVM IR may contain bugs and missing features.

14.3 Installation

CUDA-Oxide is Linux-only at the time of writing (tested on Ubuntu 24.04) and requires:

- **cargo-oxide**, the cargo subcommand that drives the build (`cargo oxide build/run/inspect/...`);
- **Rust nightly** with the `rust-src`, `rustc-dev`, and `llvm-tools` components, pinned in the project's `rust-toolchain.toml`;
- **CUDA Toolkit 12.x or newer**;
- **Clang and libclang** development headers, required by `bindgen` when building the host `cuda-bindings` crate.

```
# Install the cargo subcommand with the pinned nightly toolchain:
cargo +nightly-2026-04-03 install --git https://github.com/NVlabs/cuda-oxide.git cargo-oxide

# On first run, cargo-oxide fetches and builds the codegen backend.
# Verify CUDA is on the path:
export PATH="/usr/local/cuda/bin:$PATH"
nvcc --version
```

The workflow is cargo-shaped:

```
cargo oxide run      host_closure      # build and run an example
cargo oxide inspect vecadd             # build and print the generated PTX
cargo oxide pipeline vecadd            # show the full pipeline (MIR -> Pliron -
> LLVM -> PTX)
cargo oxide sanitize vecadd --tool memcheck # CUDA correctness checks
cargo oxide debug vecadd --tui           # debug with cuda-gdb
```

14.4 A First Kernel: The Generic `map`

The project's documented example is a generic elementwise `map`, the Rust equivalent of Chapter 10's `transformKernel`, with the kernel written in Rust:

```

// Single source file: device AND host code together.
use cuda_device::{kernel, thread, DisjointSlice};
use cuda_host::{cuda_module, load_kernel_module};
use cuda_core::{CudaContext, DeviceBuffer, LaunchConfig};

// -----
// Device side: a generic kernel that applies any function to each element.
// F can be a closure with captures - rustc monomorphises it to a concrete
// type at compile time, exactly like a C++ template instantiation.
// -----
#[cuda_module]
mod kernels {
    use super::*;

    // The #[kernel] attribute tells the backend to compile this function
    // to PTX. It is the Rust equivalent of __global__.
    #[kernel]
    pub fn map<T: Copy, F: Fn(T) -> T + Copy>(f: F, input: &[T],
                                             mut out: DisjointSlice<T>) {
        let idx = thread::index_1d(); // threadIdx/blockIdx, fused
        let i = idx.get();             // the global linear index
        // DisjointSlice guarantees this thread's slot is exclusive:
        // two threads can never get_mut the same element.
        if let Some(out_elem) = out.get_mut(idx) {
            *out_elem = f(input[i]);
        }
    }
}

// -----
// Host side: allocate, load the module, launch.
// -----
fn main() -> Result<(), Box<dyn std::error::Error>> {
    let ctx = CudaContext::new(0)?; // open GPU 0
    let stream = ctx.default_stream();

    let data: Vec<f32> = (0..1024).map(|i| i as f32).collect();
    let input = DeviceBuffer::from_host(&stream, &data)?;
    let mut output = DeviceBuffer::<f32>::zeroed(&stream, 1024)?;

    // Load the module: the codegen backend writes host_closure.ptx next to
    // Cargo.toml; load_kernel_module reads that file and returns a CUDA
    // module. from_module binds it to the typed launch API generated by
    // #[cuda_module].
    let module = load_kernel_module(&ctx, "host_closure")?;

```

```

let typed = kernels::from_module(module)?;

// Launch with a closure. `factor` is captured and passed to the GPU
// automatically (scalarised into a kernel parameter).
let factor = 2.5f32;
// SAFETY: this raw configuration is fully 1-D, matches index_1d(), and
// launches one thread per output element. A launch contract can move
// this proof into the generated safe API.
unsafe {
    typed.map::<f32, _>(
        stream.as_ref(),
        LaunchConfig::for_num_elems(1024),
        move |x: f32| x * factor,
        &input,
        &mut output,
    )
}?:;

let result = output.to_host_vec(&stream)?;
assert!((result[1] - 2.5).abs() < 1e-5);
println!("PASSED: CUDA-Oxide map closure produced expected results");
Ok(())
}

```

The device-side pieces are:

- `#[cuda_module] mod kernels { ... }` makes the backend compile the `#[kernel]` functions in the module to PTX and generate the host loading and launch glue.
- `#[kernel] pub fn map<T: Copy, F: Fn(T) -> T + Copy>(...)` declares a generic kernel. The compiler instantiates one PTX function per `(T, F)` combination, the same monomorphisation used by C++ templates in Chapter 10.
- `thread::index_1d()` is the fused equivalent of `blockIdx.x * blockDim.x + threadIdx.x` from Chapter 3, returned as a typed index.
- `DisjointSlice<T>` is a guaranteed-disjoint view of the output. Its `get_mut(id)` method returns a mutable reference to this thread's exclusive element. Two threads cannot obtain mutable access to the same slot, making the one-thread-per-output pattern a type-level guarantee.
- `if let Some(out_elem) = out.get_mut(id)` expresses the boundary guard: `None` is the out-of-range case.

The host-side pieces are:

- `CudaContext::new(0)` opens the GPU.

- `DeviceBuffer::from_host(&stream, &data)` allocates and copies in one call on the given stream.
- `load_kernel_module` reads the PTX file produced by the backend, and `kernels::from_module` binds it to the generated typed launch API. The `typed.map::<f32, _>(...)` method is generated from the kernel signature, so arguments are type-checked against the kernel parameter list. In the standalone generic-kernel build, the supported path loads the PTX file because a PTX bundle is not yet embedded in the executable. Non-generic kernels can use the embedded `kernels::load`.
- `unsafe { ... }` marks the raw launch because `LaunchConfig` is raw data: nothing in its type proves that the grid shape matches the kernel's indexing assumptions. The `SAFETY` comment states the proof obligation, as in Chapter 13.

14.5 The Safety Progression: `#[launch_contract]`

CUDA-Oxide's plan for the raw `unsafe` launch is the **launch contract**: a `#[launch_contract(...)]` attribute that moves configuration validation into generated code. A kernel annotated with a contract receives a checked `PreparedLaunch` through a safe generated method. Launch dimensions and resources are validated against the declared contract instead of being an unverifiable `unsafe` obligation.

This is the project's roadmap in miniature: each `unsafe` block is a known gap with a planned replacement. The `unsafe` in this chapter is not a licence to ignore safety; it is documented debt that the project is paying down.

14.6 Async: `cuda-async` and `DeviceOperation`

The `cuda-async` crate changes the launch shape for composable asynchronous work. The explicit `stream:` argument disappears and the launch returns a lazy `DeviceOperation` that executes when `.sync()` or `.await` is called:

```

#![allow(unused)]
fn main() {
    use cuda_async::device_operation::DeviceOperation;

    // Assuming module, input, output come from the cuda-async setup:
    let factor = 2.5f32;
    let launch = unsafe {
        // SAFETY: the raw launch is 1-D and matches this kernel's index space.
        module.map_async::<f32, _>(
            LaunchConfig::for_num_elems(1024),
            move |x: f32| x * factor,
            &input,
            &mut output,
        )?
    };
    launch.sync()?;      // or: .await?;
}

```

The lazy operation lets a program build a graph of GPU work without executing it, the same idea as CUDA Graphs (Chapter 6, §6.7), expressed as composable Rust values. `.sync()` blocks; `.await` composes with `async/await` host code. The capstone uses this shape.

14.7 Device-Side Abstractions Beyond `map`

The project documents device-side facilities beyond simple indexing:

- **Shared memory** - typed, scoped allocation within a block;
- **Scoped atomics** - atomic operations with explicit thread scopes, as in Chapter 5, with Rust scoping;
- **Barriers** - block and cluster synchronisation;
- **TMA** (Tensor Memory Accelerator) - Hopper's bulk asynchronous copies;
- **Warp/cluster operations** - shuffle-like primitives such as Chapter 8's `__shfl_down_sync`, with type-safe masks.

These are in active development. The API may change between revisions; this is the nature of alpha software.

14.8 CUDA-Oxide vs the Alternatives

Approach	Kernel language	Device safety	Maturity
CUDA C++ (Chapters 3-12)	C++	None (by hand)	Production
Rust host + C++ kernel (Ch. 13)	C++	Host only	Production
CUDA-Oxide (this chapter)	Rust	Type-checked, safe(ish)	Alpha, Linux, nightly
<code>cudarc</code> nVRTC JIT	C++ string	Host only	Production

CUDA-Oxide is not yet a production tool for most teams. It is an architecture preview: a demonstration that Rust can target the GPU without giving up its guarantees, and a first draft of the safety story SIMT programming needs. Its pipeline (MIR to Plirion to LLVM to PTX) and abstractions (`DisjointSlice`, launch contracts, async operations) point toward the shape of CUDA’s Rust future. The underlying principles, type-safe indexing, explicit safety obligations, and single-source compilation, are the same principles this book has used since Chapter 3.

Safe SIMT and Hardware Assumptions

In CUDA C++, the “one thread per output element” rule is enforced by a comment. Nothing in the language stops a thread from writing `out[i + 1]` or `out[2*i]`; the compiler accepts it, and the bug appears later as corrupt data or an illegal memory access. CUDA-Oxide expresses such conventions in the type system so violations become compile errors.

`DisjointSlice<T>` is the clearest example. Its `get_mut(idx)` returns a mutable reference to the element owned by this thread and guarantees that no other thread can obtain a mutable reference to the same element. The “one thread per output, no races” property that Chapter 3 stated in a comment is now an API property. `thread::index_1d()` fuses the index formula into one typed operation, so the formula cannot be mistyped as a hand-written expression can. Because the kernel is Rust, the borrow checker runs on the device code before any PTX is generated: a kernel that would create two mutable references to the same slot never compiles.

`unsafe` remains because launch geometry is still raw data. `LaunchConfig` describes the number of threads, but nothing in its type proves that this count matches the kernel’s indexing assumptions. The `SAFETY` comment documents the obligation. The `#[launch_contract]` roadmap shows how this obligation can become a checked

precondition: a kernel declares its contract, and the generated launch path validates the configuration against it.

The architectural point is that a system is safest when invalid programs cannot be written, not when it has the most runtime checks. CUDA-Oxide is an early, incomplete version of that idea: it moves some conventions into types, leaves others as documented unsafe obligations, and is explicit about the difference.

Common Pitfalls

- Assuming CUDA-Oxide is production-ready. It is alpha; APIs change between revisions. The companion code tracks CUDA-Oxide 0.2.1.
- Treating `DisjointSlice` as a licence to ignore bounds. `get_mut` returns `None` for out-of-range indices; forgetting the `if let` guard still skips work silently.
- Believing `unsafe` means unchecked. It means “checked by a human through the SAFETY comment”; write the comment before writing the launch.
- Porting CUDA C++ idioms verbatim, such as pointer arithmetic, instead of using the Rust-native abstractions demonstrated in this chapter.

Check Your Understanding

What does `DisjointSlice` prevent that a comment in CUDA C++ does not?

It makes the one-thread-per-output property a type guarantee: two threads cannot obtain `get_mut` for the same element. In CUDA C++, that invariant is only a comment; nothing stops a thread from writing any index.

Why is the launch still unsafe if the kernel is safe?

The kernel’s internal indexing may be safe, but the launch configuration is raw data. Nothing in `LaunchConfig` proves the grid covers the output exactly as the kernel assumes, so the launch remains an unsafe obligation documented by a SAFETY comment.

What does `#[launch_contract]` change about the obligation?

It moves the geometry proof into generated code. Launch dimensions and resources are validated against the kernel’s declared contract, so the unsafe obligation becomes a checked precondition instead of a manual comment.

Key Takeaways

- CUDA-Oxide is NVIDIA Labs’ rustc backend: `#[kernel]` Rust functions compile to PTX, with no `nvcc` or DSL.
- The pipeline Rust -> MIR -> Plirion -> LLVM -> PTX keeps rustc’s front-end guarantees (ownership, borrow checking) before GPU code is generated.
- `DisjointSlice` provides the one-thread-per-output, no-races guarantee at the type level.
- `LaunchConfig` is raw data; launching is unsafe until a `#[launch_contract]` moves the proof into generated code.
- CUDA-Oxide is alpha, Linux-only, and nightly-only: treat it as an architecture preview, not a production dependency.

14.9 Exercises

1. Compare `thread::index_1d()` with the Chapter 3 formula `blockIdx.x * blockDim.x + threadIdx.x`. What does the fused abstraction prevent?
2. Why is `DisjointSlice<T>`’s `get_mut` the type-level version of the Chapter 3 “one thread per output, no races” comment?
3. The raw launch is `unsafe` with a `SAFETY` comment; `#[launch_contract]` moves the proof into generated code. Explain the difference in terms of the obligation, not the syntax.
4. Using the pipeline diagram in §14.2, identify which phases run on the host toolchain and which produce device code. Why does borrow checking happen before any PTX generation?

Sources and Further Reading

- NVIDIA Labs, CUDA-Oxide repository: <https://github.com/NVlabs/cuda-oxide>

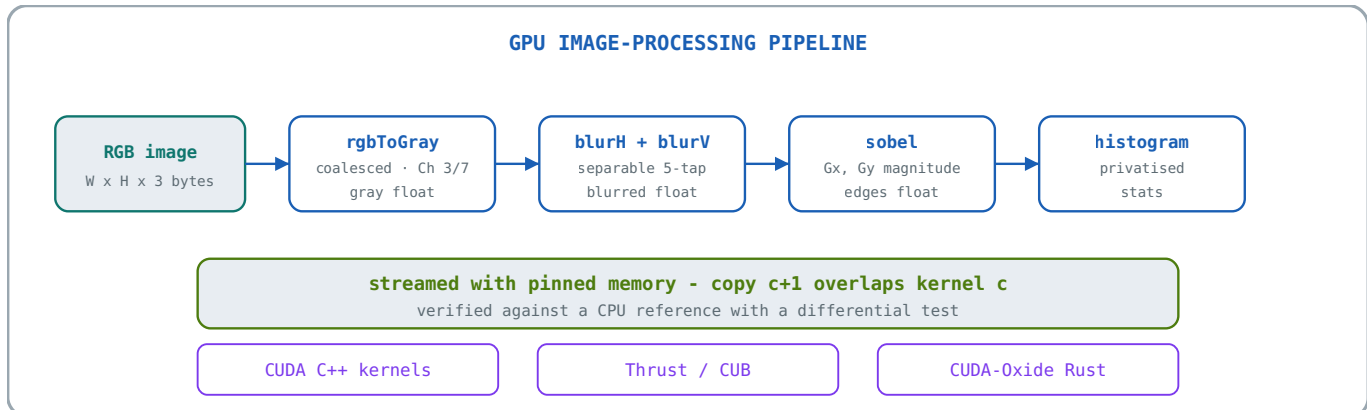
- NVIDIA, *CUDA C++ Programming Guide*, “Compute Capabilities” for PTX/SASS details: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- Rust Reference, “Inline assembly” and “Unsafe” sections: <https://doc.rust-lang.org/reference/>

Chapter 15: Capstone - The GPU Image Processing Pipeline

Code companion: the complete, buildable code for this chapter lives in [code/ch15_capstone/](#) in the repository.

This chapter combines the earlier material into one complete system. The capstone is a GPU image-processing pipeline: **RGB -> greyscale -> Gaussian blur -> Sobel edge detection**, implemented three ways (hand-written CUDA C++, Thrust, and CUDA-Oxide Rust), streamed with pinned memory, verified against a CPU reference, and measured with CUDA events. It is small enough to fit in one chapter and structured enough to exercise the book's methods end to end.

15.1 Pipeline and Data Flow



Design decisions and their rationale:

- **Greyscale stored as float, not unsigned char.** Blur and Sobel accumulate fractional weights. **float** avoids rounding at every stage and matches the arithmetic-intensity discussion of Chapter 1. The final edge map is scaled to **unsigned char** for output.
- **Separable Gaussian.** A 2-D Gaussian of radius 2 is a 5x5 stencil: 25 taps per output pixel. A separable Gaussian performs a horizontal 5-tap pass followed by a vertical 5-

tap pass: 10 taps per pixel. The two-pass form has the same mathematical result and each pass is naturally coalesced.

- **Sobel as two separable kernels.** The Sobel operator is the pair of 3x3 kernels G_x and G_y . Each factors into a derivative pass and a smoothing pass. This implementation computes the two 3x3 convolutions directly and takes the magnitude $\sqrt{G_x^2 + G_y^2}$.
- **Histogram at the end.** The histogram verifies that the pipeline produced sensible data and demonstrates Chapter 8's privatised histogram on a real workload.

15.2 Stage 1: RGB to Greyscale (CUDA C++)

The canonical coalesced kernel uses one thread per output pixel and the Chapter 3 index formula, so consecutive threads read consecutive pixels:

```
// -----
// rgbToGray: 3 bytes/pixel RGB (uchar3) -> 1 float/pixel greyscale.
// Consecutive threads -> consecutive pixels -> coalesced reads and writes.
// The weights are the standard BT.601 luma coefficients; they sum to 1.0,
// so no scaling is needed and a constant-grey input maps to itself.
// -----
__global__ void rgbToGray(const uchar3* rgb, float* gray, int numPixels)
{
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < numPixels)
    {
        const uchar3 px = rgb[i];                // 12-byte read, coalesced
        // uchar3 components are 0..255; multiply in float to avoid
        // integer truncation.
        gray[i] = 0.299f * static_cast<float>(px.x)
                + 0.587f * static_cast<float>(px.y)
                + 0.114f * static_cast<float>(px.z);
    }
}
```

`uchar3` is CUDA's built-in 3-byte vector type and matches the RGB layout exactly. Its alignment is 1, so it can point at raw RGB bytes. A `float3` would not be safe for the same data because it is 16-byte aligned.

15.3 Stage 2: Separable Gaussian Blur

The 5-tap weights for $\sigma = 1$ are `[0.06136, 0.24477, 0.38774, 0.24477, 0.06136]`, a normalised Gaussian. The horizontal pass reads a row segment including a halo of two pixels on each side; the vertical pass does the same along columns.

An initial, plausible implementation clamps only the outermost stencil coordinates:

```
// WRONG at image borders:
__global__ void blurH(const float* in, float* out, int width, int height)
{
    const int x = blockIdx.x * blockDim.x + threadIdx.x;
    const int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x < width && y < height)
    {
        const float* row = in + y * width;
        const int x0 = max(x - 2, 0), x1 = min(x + 2, width - 1);
        out[y * width + x] = 0.06136f * row[x0] + 0.24477f * row[x0]
                           + 0.38774f * row[x]   + 0.24477f * row[x1]
                           + 0.06136f * row[x1];
    }
}
```

This version is correct in the interior but wrong at borders. The inner taps `x-1` and `x+1` reuse the clamped outer values, so the replicated border pixel is weighted twice, for example as `0.06136 + 0.24477` on the left edge. The correct version clamps each tap independently:

```

__global__ void blurH(const float* in, float* out, int width, int height)
{
    const int x = blockIdx.x * blockDim.x + threadIdx.x;
    const int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x < width && y < height)
    {
        const float* row = in + y * width;
        // Weights, left to right: [0.06136, 0.24477, 0.38774, 0.24477,
0.06136]
        const float w[5] = {0.06136f, 0.24477f, 0.38774f, 0.24477f, 0.06136f};
        float acc = 0.0f;
        // Each tap clamps its index to the row bounds independently:
        // interior pixels use the exact stencil; edge pixels are replicated.
        #pragma unroll
        for (int t = -2; t <= 2; ++t)
        {
            const int sx = min(max(x + t, 0), width - 1);
            acc += w[t + 2] * row[sx];
        }
        out[y * width + x] = acc;
    }
}

```

The first version is plausible and wrong; the second is correct by construction. Border handling is the difference. The vertical pass uses the same logic with `x` and `y` exchanged; it is omitted here to avoid repetition.

Separable blur uses two kernels rather than one fused 5x5 kernel. A fused kernel would read a 5x5 neighbourhood per thread, or communicate the intermediate image through shared memory with a block halo. Two global passes are simpler, fully coalesced, and bandwidth-dominated at this image scale.

15.4 Stage 3: Sobel Edge Detection

```
// -----
// sobel: magnitude of the gradient. Gx = derivative across x, smoothed in y;
// Gy = derivative across y, smoothed in x. We compute both 3x3 convolutions
// and the magnitude sqrt(Gx^2 + Gy^2) per pixel.
// -----
__global__ void sobel(const float* in, float* out, int width, int height)
{
    const int x = blockIdx.x * blockDim.x + threadIdx.x;
    const int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x < width && y < height)
    {
        // Clamp the 3x3 neighbourhood at the borders (replicate-edge).
        const int xm = max(x - 1, 0), xp = min(x + 1, width - 1);
        const int ym = max(y - 1, 0), yp = min(y + 1, height - 1);

        const float* r0 = in + ym * width;
        const float* r1 = in + y * width;
        const float* r2 = in + yp * width;

        // Horizontal derivative Gx: [-1 0 1] across each of the three rows,
        // weighted [1 2 1] down the columns.
        const float gx = (r2[xp] + 2.0f * r1[xp] + r0[xp])
                        - (r2[xm] + 2.0f * r1[xm] + r0[xm]);
        // Vertical derivative Gy: [-1 0 1] down the columns,
        // weighted [1 2 1] across the rows.
        const float gy = (r2[xm] + 2.0f * r2[x] + r2[xp])
                        - (r0[xm] + 2.0f * r0[x] + r0[xp]);

        // Magnitude. sqrtf is the device-side square root.
        out[y * width + x] = sqrtf(gx * gx + gy * gy);
    }
}
```

G_x is a derivative in x (the kernel $\begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$) convolved with a smoothing in y (the kernel $\begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$); G_y is the transpose. The kernel reads nine pixels and produces both convolutions.

15.4.1 Scale Back to `unsigned char`

The magnitude $\sqrt{G_x^2 + G_y^2}$ is a `float` that can exceed 255. The histogram counts `unsigned char` bins, so a scaling step is required. Clamping is necessary: converting an out-of-range float to `unsigned char` is undefined behaviour, and the histogram would count byte patterns instead of edges.

```
// Clamp the float edge magnitude to [0, 255] and store as uchar.
__global__ void scaleEdges(const float* in, unsigned char* out, int n)
{
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n)
    {
        const int v = static_cast<int>(in[i] + 0.5f);    // round, don't
truncate
        out[i] = static_cast<unsigned char>(min(max(v, 0), 255));
    }
}
```

15.5 The Streaming Host Pipeline

The frame loop uses the machinery of Chapters 4 and 6: pinned host memory, two streams, and double buffering so that transfers overlap kernels:

```

// -----
// One "frame" = load RGB, run the kernels, store edges. Frames arrive in host
// buffers h_rgb[0] and h_rgb[1]; the GPU processes one while the DMA engine
// uploads the next (Chapter 6, 6.5).
// -----
void processFrames(/* ... device buffers, streams, sizes ... */)
{
    const int  numPixels = width * height;
    const bool last = (frame == frameCount - 1);

    const int cur = frame % 2;          // buffer holding THIS frame's input
    const int nxt = (frame + 1) % 2;    // buffer for the NEXT frame

    // Upload the NEXT frame while THIS one computes (pinned memory), and
    // record an event after it: the NEXT iteration's kernel waits on it.
    if (!last)
    {
        CHECK(cudaMemcpyAsync(d_rgb[nxt], h_rgb[nxt],
                               rgbBytes, cudaMemcpyHostToDevice, sCopy));
        CHECK(cudaEventRecord(copyDone[nxt], sCopy));
    }

    // Make the compute stream wait for THIS frame's copy. Its event was
    // recorded in the previous iteration (or during the prime before the
    // loop). cudaStreamWaitEvent installs the dependency without blocking.
    CHECK(cudaStreamWaitEvent(sCompute, copyDone[cur], 0));

    // The stages, all in the compute stream, in order:
    const dim3 block(256);
    const dim3 grid((numPixels + 255) / 256);
    // d_rgb is a raw byte buffer; rgbToGray reads it as uchar3 (alignment 1,
    // §15.2), so the view is explicit:
    rgbToGray<<<grid, block, 0, sCompute>>>(
        reinterpret_cast<const uchar3*>(d_rgb[cur]), d_gray, numPixels);

    const dim3 b2(32, 8); // 2-D block: 32 x 8 threads
    const dim3 g2((width + 31) / 32, (height + 7) / 8);
    blurH    <<<g2, b2, 0, sCompute>>>(d_gray, d_blurred, width, height);
    blurV    <<<g2, b2, 0, sCompute>>>(d_blurred, d_blurred2, width, height);
    sobel    <<<g2, b2, 0, sCompute>>>(d_blurred2, d_edges, width, height);

    // Scale float edges back to uchar before the histogram: the histogram
    // counts edge intensities, not the bytes of a float.
    scaleEdges<<<grid, block, 0, sCompute>>>(d_edges, d_edges8, numPixels);
}

```

```
// The privatised histogram is a 1-D kernel with 256-thread blocks. It
// uses the same 1-D grid/block as rgbToGray, not the 2-D stencil block.
histogram<<<grid, block, 0, sCompute>>>(d_edges8, d_hist, numPixels);
// (histogram kernel as in Chapter 8, 8.8)

// Copy the edge map back (device -> host, pinned, async). The buffer is
// numPixels BYTES (uchar edges), not 4 bytes per pixel.
CHECK(cudaMemcpyAsync(h_edges[cur], d_edges8,
                      numPixels * sizeof(unsigned char),
                      cudaMemcpyDeviceToHost, sCompute));
}
```

The copy for frame $n + 1$ and the kernels for frame n operate on different buffers, which is the condition for overlap (§6.5). The event and dependency pair (`cudaEventRecord` plus `cudaStreamWaitEvent`) keeps the order correct on every iteration without serialising the pipeline.

The 2-D grid gives each thread a natural `(x, y)` pixel. The 32x8 block shape keeps blocks tile-shaped, with 32 threads in `x` so that warps are row-aligned.

15.6 The Same Pipeline in Thrust

The library version replaces hand-written kernels with `thrust::transform` shaped calls (Chapter 11). Stencil kernels need neighbouring pixels, which `transform` can obtain through index-based functors:

```

#include <thrust/iterator/zip_iterator.h>
#include <thrust/iterator/counting_iterator.h>

// Greyscale: pure elementwise -> a plain transform functor.
struct ToGray {
    __device__ float operator()(const uchar3& px) const {
        return 0.299f * px.x + 0.587f * px.y + 0.114f * px.z;
    }
};

// Blur with halo: the functor receives the pixel INDEX and the row pointer;
// it reads its own 5-tap window.
struct BlurH5 {
    const float* in; int width;
    __device__ float operator()(int i) const {
        const int x = i % width, y = i / width;
        const float* row = in + y * width;
        float acc = 0.0f;
        const float w[5] = {0.06136f, 0.24477f, 0.38774f, 0.24477f, 0.06136f};
        for (int t = -2; t <= 2; ++t) {
            const int sx = min(max(x + t, 0), width - 1);
            acc += w[t + 2] * row[sx];
        }
        return acc;
    }
};

// Host side: chain the stages over device vectors.
thrust::device_vector<uchar3> d_rgb(...);
thrust::device_vector<float> d_gray(n), d_blur(n), d_edges(n);

thrust::transform(d_rgb.begin(), d_rgb.end(), d_gray.begin(), ToGray());
thrust::transform(thrust::counting_iterator<int>(0),
                  thrust::counting_iterator<int>(n),
                  d_blur.begin(),
                  BlurH5{thrust::raw_pointer_cast(d_gray.data()), width});
// ... blurV and sobel follow the same pattern; histogram is a thrust::reduce
// over a per-bin functor, or thrust::sort + adjacent-difference.

```

Thrust removes launch plumbing and boundary guards. The index-based stencil functor reintroduces the stencil logic that the custom kernel had. For elementwise stages such as greyscale and magnitude, Thrust is a clear win. For stencil stages, the custom kernel of §15.3 is comparable in code size and easier to tune. This is the Chapter 11 decision procedure in practice.

15.7 The Same Pipeline in CUDA-Oxide

With CUDA-Oxide (Chapter 14), the greyscale stage becomes a Rust `#[kernel]` function with `DisjointSlice` on the output:

```
#![allow(unused)]
fn main() {
    use cuda_device::{kernel, thread, DisjointSlice};
    use cuda_host::cuda_module;

    #[cuda_module]
    mod kernels {
        use super::*;

        // Greyscale in pure Rust. DisjointSlice<f32> guarantees exclusive
        // output slots; the input is read-only.
        #[kernel]
        pub fn rgb_to_gray(rgb: &[u8], gray: DisjointSlice<f32>, num_pixels: u32)
        {
            let idx = thread::index_1d();
            let i = idx.get();
            if (i < num_pixels) {
                // RGB is packed 3 bytes/pixel; u8 -> f32 conversion is explicit.
                let base = (i as usize) * 3;
                let r = rgb[base] as f32;
                let g = rgb[base + 1] as f32;
                let b = rgb[base + 2] as f32;
                if let Some(out) = gray.get_mut(idx) {
                    *out = 0.299 * r + 0.587 * g + 0.114 * b;
                }
            }
        }
    }
}
```

The kernel is written in the host's language, indexed by the fused `thread::index_1d()` (Chapter 14), and protected by `DisjointSlice`, so the output has no alias and no manual boundary contract. The stencil kernels follow the same shape with per-tap clamping as in the C++ versions. As Chapter 14 noted, the API is alpha; the shape is the point.

15.8 Verification: The Differential Test

A pipeline that produces wrong edges quickly is not useful. The verification strategy is the differential test:

1. **CPU reference.** A plain-loop implementation of the same stages is the correctness oracle.
2. **GPU pipeline.** The same stages run on a test image.
3. **Stage-by-stage comparison.** Greyscale, blurred, and edge maps must agree within a tolerance. Float stages use `1e-3`; the `uchar` edge map allows ± 1 because the SFU `sqrtof` can round differently at a `.5` boundary.
4. **Property checks.** Histogram bins must fall in expected ranges, and an all-black image must produce all-zero edges (a known-answer test).

The differential test converts “the pipeline works” into a repeatable assertion. The second implementation (Thrust) and third (CUDA-Oxide) must pass the same test as the first.

15.9 Measurement: The Report Card

Performance is measured with CUDA events (Chapter 6, §6.4) over many frames, with warm-up excluded:

```
// Per-stage timing with events:
cudaEventRecord(start, sCompute);
rgbToGray<<<...>>>(...);
cudaEventRecord(mid, sCompute);
blurH<<<...>>>(); blurV<<<...>>>(); sobel<<<...>>>(); scaleEdges<<<...>>>();
cudaEventRecord(stop, sCompute);
cudaEventSynchronize(stop);
float msStage1 = 0, msRest = 0;
cudaEventElapsedTime(&msStage1, start, mid);
cudaEventElapsedTime(&msRest, mid, stop);
```

The report card below gives teaching numbers for a 1920x1080 frame on a modern GPU. Measure on the target hardware; the ratios between stages are what the roofline predicts:

Stage	Time	Bandwidth (3.35 TB/s peak)	Roofline verdict
rgbToGray	~6 μ s	~75% of peak	Memory-bound (as predicted)
blurH + blurV	~14 μ s	~70% of peak	Memory-bound, halo cost visible
sobel	~7 μ s	~70% of peak	Memory-bound
histogram	~10 μ s	-	Atomic overhead, privatised
Total compute	~40 μs	-	~25,000 FPS compute-only; transfer-limited overall

A 1920x1080 RGB frame is 6.2 MB to upload, and the `uchar` edge map is 2.1 MB to download, about 8.3 MB of host-device traffic per frame. At a pinned PCIe Gen4 rate of about 20 GB/s, that is roughly 400 μ s of transfer per frame, about ten times the total compute time. The pipeline is transfer-limited: the kernels are not the bottleneck at this image size. This is the case for the streaming machinery of Chapters 4 and 6.

The roofline predicted the memory-bound verdicts before any code ran: each stage moves a few bytes per pixel and performs few FLOPs. The measurement confirms the prediction. The engineering loop is: predict with the model, confirm with the instrument, and optimise only the confirmed bottleneck.

15.10 The Capstone in One Paragraph

The pipeline compresses the book into one system: coalesced kernels with explicit index arithmetic (Chapters 3 and 7), synchronisation-free stages and a privatised histogram (Chapters 5 and 8), pinned memory and streamed double buffering (Chapters 4 and 6), library and language alternatives that must pass the same differential test (Chapters 11, 13, and 14), and a measurement discipline that turns opinions into numbers (Chapter 16). Building this pipeline and explaining every line is the practical version of the book's goal: the same reasoning transfers to new kernels.

The Pipeline as a Testbed

The capstone is more than a program. It is a small, complete system in which every idea from the earlier chapters has a concrete responsibility and a way to be tested.

The roofline model predicts, before code runs, that each stage is memory-bound because each stage moves a few bytes per pixel and does little arithmetic. The report card tests that prediction with CUDA events. The streaming host loop tests whether pinned memory plus two streams plus events hides transfer time behind kernel time. The differential test tests the strongest claim: all three implementations produce the same result as the CPU reference.

The capstone is a testbed because one piece can be changed and the whole suite re-run. Replace the separable blur with a fused 5x5 kernel, and the differential test checks correctness while event timings test whether the roofline prediction still holds. Change the histogram launch to the wrong block shape, and the histogram total exposes the bug. Every stage is a hypothesis; the pipeline is the experiment.

Common Pitfalls

- Reusing a 2-D stencil block for a 1-D kernel. The histogram must be launched with a 1-D 256-thread block. Launching it with the 2-D (32, 8) stencil block only zeroes part of the private bins and over-counts.
- Clamping only the outer stencil taps at image borders. Every tap must be clamped independently, or the edge weights are wrong.
- Using pageable host memory in the streaming loop. Async copies silently become synchronous and the overlap disappears.
- Trusting a pipeline without a differential test. A fast wrong image is not a result.

Check Your Understanding

Why must every stencil tap be clamped independently at borders?

If only the outer taps are clamped, inner taps reuse the clamped values and the border pixel is double-weighted, for example as $0.06136 + 0.24477$. Independent clamping replicates the edge pixel for each tap and preserves the correct weights.

Why is the histogram a meaningful sanity check for the pipeline?

The edge map is scaled to uchar, so its histogram counts edge intensities. If the histogram total does not equal frames times pixels, data was dropped or double-counted somewhere in the chain.

Why can the differential test tolerate $1e-3$ for floats but only ± 1 for uchar?

Float stages use different summation orders, such as FMA contraction, and the device `sqrtf` approximates the last bits, so exact equality is unrealistic. The uchar edge map is quantised; a rounding difference at a `.5` boundary can flip one byte, so ± 1 is allowed. Anything larger is a real error.

Key Takeaways

- A pipeline is a chain of kernels; a fast pipeline is one that never waits (streams + pinned memory + events).
- A separable Gaussian uses 2×5 taps instead of 25; each stencil tap must be clamped independently at borders.
- The differential test against a CPU reference is what makes a second and third implementation trustworthy.
- The roofline predicted every capstone stage was memory-bound before code ran.
- Use median-of-many-runs timings, a fixed environment, and CUDA events for device time.

15.11 Exercises

1. Why is the separable blur “10 taps instead of 25”? Derive the count for a 5-tap separable Gaussian versus a full 5×5 stencil, and for a 9-tap version.
2. The first `blurH` in §15.3 was plausible and wrong. Explain what was wrong and state the property the corrected kernel guarantees at the borders.
3. In the streaming loop, why must `h_rgb` be pinned memory? Trace what happens if it is pageable.
4. The differential test uses a tolerance of `1e-3` for float stages and ± 1 for the uchar edge map. Why not exact equality? Consider Chapter 5, §5.6, and the SFU `sqrtf` in the Sobel kernel.

5. Using the roofline model, predict whether making the blur a single fused 5x5 kernel (25 taps, no intermediate) would be faster or slower than the two-pass version, and explain the trade.

Chapter 15b: Benchmarking CUDA-Oxide on Jetson Orin

Code companion: the complete, buildable code for this chapter lives in [arpanpathak/cuda-oxide-demo](#). The article version is [CUDA_RUST_JETSON_BENCHMARKS.md](#).

Chapter 14 showed how CUDA-Oxide compiles idiomatic Rust kernels to PTX. This chapter measures those kernels on real hardware and asks when the GPU is worth using on an embedded SoC.

Four kernels are benchmarked on a Jetson Orin:

1. **SAXPY** - a memory-bound vector operation.
2. **Dot product** - a block reduction.
3. **Matrix multiply** - naive versus shared-memory tiled.
4. **Jacobi solver for the 2D Laplace equation** - a stencil that solves a partial differential equation.

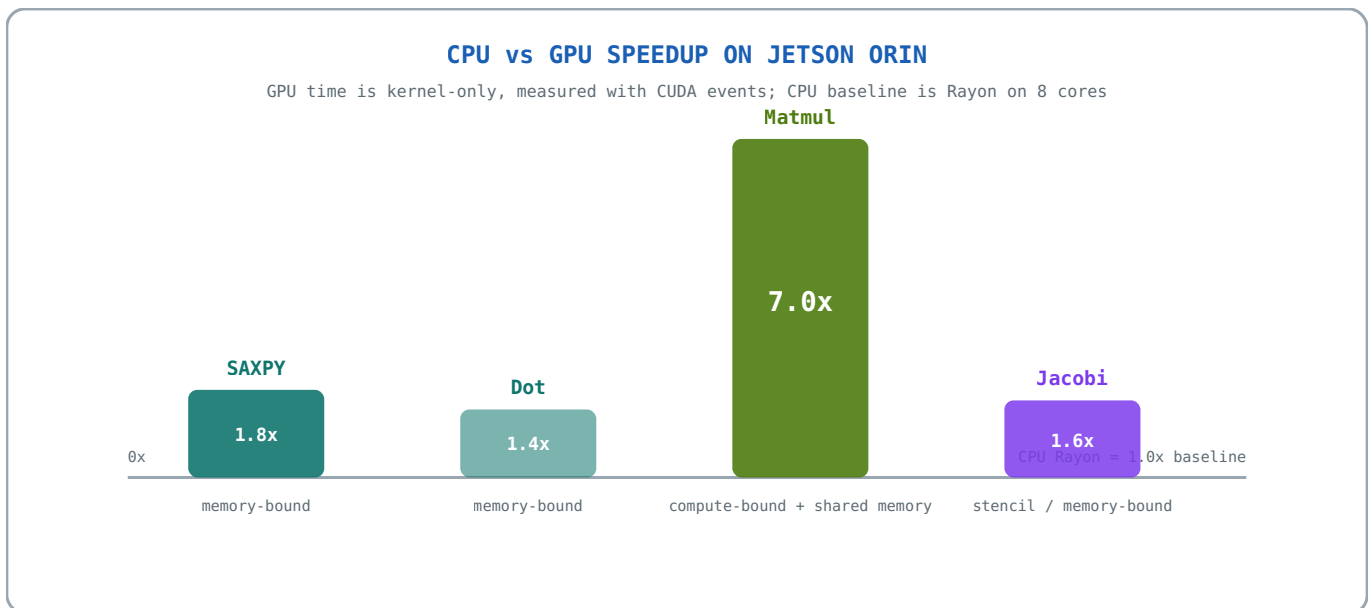
Every GPU result is verified against a CPU reference. The CPU baseline uses Rayon so that all eight ARM cores participate.

15b.1 Hardware Context

The Jetson Orin is not a datacenter GPU. It is an embedded SoC in which the CPU and GPU share:

- the same DRAM bandwidth,
- the same power budget,
- the same thermal envelope.

This changes the performance story. On a discrete GPU, many kernels beat the CPU. On an SoC, the CPU is competitive for memory-bound work because both processors are limited by the same memory system.



The chart shows:

- SAXPY: 1.8x GPU speedup.
- Dot product: roughly parity (0.7x to 1.8x across runs).
- Tiled matmul: 7.0x GPU speedup.
- Jacobi global stencil: 1.6x GPU speedup.

Compute-bound work with data reuse is where the GPU dominates. Memory-bound work is closer to a tie.

15b.2 The Kernels in the Demo

The repository is modular. Each benchmark binary is split into small single-purpose modules:

```
src/bin/06_benchmark/
├── main.rs          # entry point
├── kernels.rs       # #[kernel] device functions
├── cpu.rs           # Rayon CPU references
├── data.rs          # generators + correctness checks
├── measure.rs       # wall-clock / CUDA-event timing
├── bench.rs         # one small method per benchmark
└── report.rs        # Markdown/CSV output

src/bin/07_laplace_jacobi/
├── main.rs          # entry point + parameters
├── kernels.rs       # global + shared-memory stencils
├── cpu.rs           # Rayon CPU Jacobi reference
├── data.rs          # correctness checks
├── measure.rs       # CPU timing helper
├── bench.rs         # orchestration + GPU event timing
└── report.rs        # results printing + file output
```

SAXPY

```
#![allow(unused)]
fn main() {
    #[kernel]
    pub fn saxpy(alpha: f32, input_x: &[f32], input_y: &[f32], mut output:
    DisjointSlice<f32>) {
        let index = thread::index_1d();
        let position = index.get();

        if let Some(slot) = output.get_mut(index) {
            *slot = alpha * input_x[position] + input_y[position];
        }
    }
}
```

Dot Product Block Reduction

```

#![allow(unused)]
fn main() {
    static mut SHARED: SharedArray<f32, 256> = SharedArray::UNINIT;
    // grid-stride accumulation into local_sum ...
    unsafe { SHARED[thread_id] = local_sum; }
    thread::sync_threads();

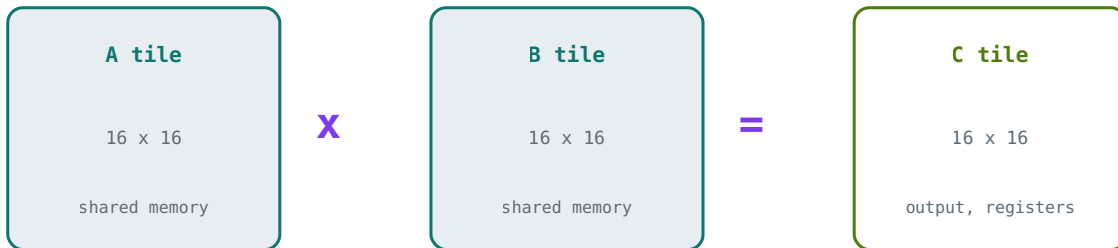
    let mut offset = 128;
    while offset > 0 {
        if thread_id < offset {
            unsafe { SHARED[thread_id] += SHARED[thread_id + offset]; }
        }
        thread::sync_threads();
        offset /= 2;
    }
}

```

Tiled Matrix Multiply

TILED MATMUL: REUSE DATA IN SHARED MEMORY

16x16 tiles of A and B are loaded once, then reused for 16x16 output cells



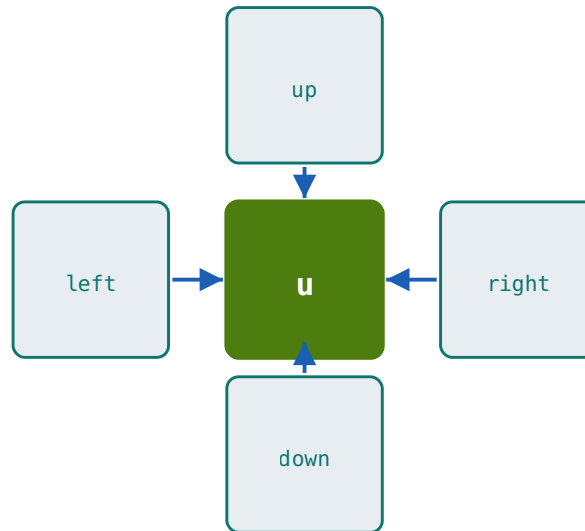
for each K-tile: load A and B into shared memory, sync, multiply-accumulate, sync
global memory traffic drops from $O(K)$ per output cell to $O(1)$ per tile

Each 16x16 block loads a tile of A and a tile of B into shared memory, synchronises, computes the output tile from shared memory, synchronises again, and advances to the next K-tile. This change turns the naive GPU kernel into a roughly 6x faster kernel.

Jacobi Solver

FIVE-POINT JACOBI STENCIL

$$u_new[i,j] = (u[i-1,j] + u[i+1,j] + u[i,j-1] + u[i,j+1]) / 4$$



boundary cells are fixed: top = 100, other edges = 0
each iteration spreads boundary information one cell further inward

The Laplace equation is the steady-state diffusion equation. On a grid, the discrete form becomes:

$$u_new[i,j] = (u[i-1,j] + u[i+1,j] + u[i,j-1] + u[i,j+1]) / 4$$

Each iteration replaces every interior cell with the average of its four neighbours. The top boundary is fixed at 100, the other edges at 0, and the iteration spreads the boundary information inward.

15b.3 Where the Laplace Equation Appears

The Laplace equation $\nabla^2 u = 0$ describes equilibrium in physics:

- **Heat conduction:** steady-state temperature in a solid.
- **Electrostatics:** electric potential in a charge-free region.
- **Fluid dynamics:** pressure in potential flow.
- **Image processing:** inpainting, smoothing, and edge detection.

- **Graphics:** surface fairing and smooth height fields.

The Poisson variant $\nabla^2 u = f$ adds sources. Jacobi iteration is the simplest solver and is the foundation for the multigrid and Krylov methods used in production solvers.

15b.4 Benchmark Methodology

- **CPU:** Rayon-parallel Rust, `std::time::Instant`, best of 5.
- **GPU:** CUDA events, best of 5, kernel time only.
- **Warm-up:** 5 GPU launches before timing so that the Jetson's clocks ramp up.
- **Data:** deterministic pseudo-random `f32` vectors and matrices.
- **Verification:** every GPU result is compared against the CPU reference with a tolerance.

15b.5 Results

Vector Kernels

Benchmark	CPU ms	CPU rate	GPU ms	GPU rate	Speedup
SAXPY, N = 16,777,216	4.989	40.4 GB/s	2.700	74.6 GB/s	1.8x
Dot product, N = 33,554,432	6.926	38.8 GB/s	4.891	54.9 GB/s	1.4x

Matrix Multiply

Implementation	Time	Rate	Speedup vs CPU Rayon
CPU, 1 thread	214.480 ms	10.0 GFLOPS	0.35x
CPU, Rayon (8 cores)	73.531 ms	29.2 GFLOPS	1.0x
GPU, naive	66.490 ms	32.3 GFLOPS	1.1x
GPU, tiled 16x16	10.575 ms	203.1 GFLOPS	7.0x

Jacobi Solver

Implementation	ms/iteration	500 iterations	Speedup
CPU, Rayon	0.2283	114.1 ms	1.0x
GPU, global-memory stencil	0.1460	73.0 ms	1.6x
GPU, shared-memory tiled stencil	0.2258	112.9 ms	1.0x

Both GPU Jacobi variants match the CPU field exactly after 500 iterations.

15b.6 Lessons

1. **Memory-bound kernels are close on SoCs.** When the CPU can already saturate the shared memory bandwidth, the GPU adds little.
2. **Shared-memory tiling is the GPU's main advantage here.** Matmul jumps from 32 to 203 GFLOPS, a 6.3x improvement over the naive GPU kernel and 7x over the CPU.
3. **Shared memory is not always the answer.** The tiled Jacobi stencil is slower than the simple global stencil on the Orin because the unified L2 cache absorbs the halo reads. Measure on the target hardware.
4. **Warm-up matters on embedded GPUs.** Without warm-up launches, the first measured kernel can be 2x slower because of clock ramping.

15b.7 Reproduce

```
git clone https://github.com/arpanpathak/cuda-oxide-demo.git
cd cuda-oxide-demo

cargo oxide run --bin 06_benchmark
cargo oxide run --bin 07_laplace_jacobi
```

Release builds:

```
cargo oxide build -- --release --bin 06_benchmark --bin 07_laplace_jacobi
./target/release/06_benchmark
./target/release/07_laplace_jacobi
```

Reports are written to `benchmarks/benchmark_results.{md,csv}` and `benchmarks/jacobi_results.{md,csv}`.

Sources and Further Reading

- NVIDIA, *Jetson Orin Developer Guide*: <https://developer.nvidia.com/embedded/learn/jetson-orin>
- CUDA-Oxide demo repository used in this chapter: <https://github.com/arpanpathak/cuda-oxide-demo>

Chapter 16: Profiling, Debugging & Performance Engineering

“We should forget about small efficiencies, say about 97% of the time: premature optimization is the root of all evil.” — Donald E. Knuth, “Structured Programming with go to Statements” (1974)

Every earlier chapter claimed that one implementation is faster than another. This chapter is about proving such claims. It covers the four instruments of GPU engineering: **Nsight Systems** and **Nsight Compute** (profilers), **Compute Sanitizer** (debugger), and the **benchmarking discipline**. It also covers differential and property testing, which keep optimisations grounded. After this chapter you can take any kernel from this book and answer two questions reproducibly: is it correct, and is it fast?

16.1 The Two Profilers

NVIDIA ships two profilers with distinct jobs:

Primitive - Nsight Systems (nsys). A system-level profiler. It shows a timeline of the whole program: when kernels ran, when transfers ran, when the CPU was idle, and how streams overlapped. It answers where the time goes and, in particular, whether the GPU was idle waiting for the host. **Primitive - Nsight Compute (ncu)**. A kernel-level profiler. It reports per-kernel hardware counters: achieved occupancy, memory throughput, shared-memory bank conflicts, warp stall reasons, and FLOP counts. It answers why a kernel is slow.

The workflow is **nsys** first, **ncu** second. If the GPU is idle 40% of the time, kernel tuning does not help; the fix is streams and overlap (Chapter 6). Only when the timeline shows the GPU busy should a kernel be examined with **ncu**.

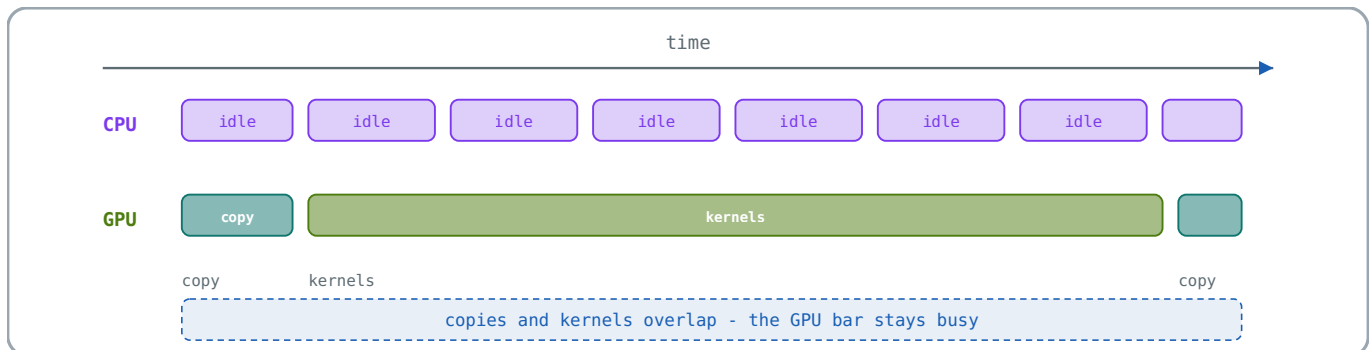
```
# System-level timeline (10 seconds of the application):
nsys profile --duration=10 ./pipeline

# Kernel-level analysis of the sobel kernel (the profiler replays the kernel
# under instrumentation):
ncu --kernel-name regex:sobel --set full ./pipeline
```

ncu replays the kernel because collecting full counters changes timing. It runs the kernel multiple times under instrumentation and aggregates counters, so the numbers describe the kernel rather than the profiler’s overhead. This is also why **ncu** cannot profile every metric group at once; use **--set** presets.

16.2 Reading the Timeline (nsys)

A healthy streamed pipeline (Chapter 15) shows:



The GPU bar is continuously busy: copies and kernels overlap, and the only gaps are pipeline priming. Unhealthy timelines and their diagnoses:

Timeline symptom	Diagnosis	Fix
GPU idle between kernel and next copy	Default-stream serialisation	Name streams, use <code>cudaStreamNonBlocking</code> (Ch. 6)
Small kernel gaps every frame	Host launch overhead	CUDA Graphs (Ch. 6, §6.7)
Long grey “CPU time” blocks	Host-side stall (I/O, alloc)	Pre-allocate, pin memory (Ch. 4)
Copy and kernel never overlap	Pageable memory	<code>cudaMallocHost</code> (Ch. 4)

Each symptom has a one-line fix and each fix maps to a chapter already read.

16.3 The Kernel Report (ncu)

For a single kernel, `ncu --set full` reports the metrics this book has trained the reader to interpret:

- **Achieved occupancy** (§2.9): warps resident versus the theoretical maximum. Low occupancy plus memory stalls means too few warps are available to hide latency.
- **Memory throughput**: percentage of peak DRAM bandwidth used. Near 100% on a memory-bound kernel means coalescing is working. Far below it means the checklist of Chapter 7 (§7.9) applies.
- **Shared-memory bank conflicts**: cycles lost to bank conflicts (§2.8, §7.5). Zero is achievable with padding.
- **Warp stall reasons**: why warps wait. `long_scoreboard` means waiting on a global load; `short_scoreboard` means shared memory; `barrier` means waiting at `__syncthreads`; `drain` means stores are not flushed. Each stall reason points at a different chapter of this book.

The discipline is: record the metric, form a hypothesis, change one thing, and re-measure. Two simultaneous changes make a measurement uninterpretable.

16.4 Compute Sanitizer

Primitive - Compute Sanitizer (`compute-sanitizer`). A runtime tool that instruments a kernel to detect memory and synchronisation errors that would otherwise be silent: out-of-bounds accesses, misaligned accesses, data races, and invalid `__syncthreads` usage.

```
# Memory checking: out-of-bounds, uninitialised, and misaligned accesses.
compute-sanitizer --tool memcheck ./pipeline

# Race checking: data races between threads (Chapter 5's bug class).
compute-sanitizer --tool racecheck ./pipeline

# Initialisation checking: reads of uninitialised memory.
compute-sanitizer --tool initcheck ./pipeline

# Synchronisation checking: divergent __syncthreads (Chapter 5, 5.3).
compute-sanitizer --tool synccheck ./pipeline
```

A CPU out-of-bounds write usually crashes at the instruction. A GPU out-of-bounds write can corrupt adjacent memory in the same allocation: the kernel reports success and the corruption appears three stages later as a wrong image. `memcheck` identifies the write at the moment it happens, with the thread and instruction.

Every race, bank conflict, and divergence class discussed in Chapters 5 and 7 has a detector. Run the detector before trusting reasoning.

16.5 cuda-gdb

For bugs that resist automatic tools, `cuda-gdb` provides an interactive debugger for device code: breakpoints inside kernels, inspection of `threadIdx` and `blockIdx`, register and shared-memory watches, and warp-by-warp stepping.

```
cuda-gdb ./pipeline
(cuda-gdb) break sobel
(cuda-gdb) run
(cuda-gdb) set cuda break_on_launch application # stop at every kernel
(cuda-gdb) info cuda kernels                  # list active kernels
(cuda-gdb) thread 5                           # select a specific thread
(cuda-gdb) print x                            # inspect kernel variables
```

Reach for `cuda-gdb` after Compute Sanitizer has cleared memory and trace errors. A remaining logical bug, such as wrong index arithmetic or wrong stencil weights, can be inspected by breaking on a kernel and checking a specific thread's values by hand.

16.6 clock64(): Timing Inside the Kernel

Profiler replay can change the answer for kernels whose performance depends on cache state. `clock64()` reads a per-SM cycle counter from inside the kernel:

```
// Time a code region from INSIDE the kernel. Returns SM cycles.
// Useful when profiler replay perturbs the measurement; otherwise prefer ncu.
__device__ long long profileRegion()
{
    const long long t0 = clock64();
    // ... the region being timed ...
    const long long t1 = clock64();
    return t1 - t0;           // SM cycles (see device clock rate)
}
```

`clock64()` measures this thread's view: the scheduler may preempt the warp mid-region, and the SM clock can vary with power state. Use it for relative comparisons of code paths within one kernel run, not as a cross-run benchmark. For cross-run numbers, use CUDA events (Chapter 6) and the discipline of §16.7.

16.7 The Benchmarking Discipline

A number from one run is not a measurement. The reproducible protocol used for every claim in this book is:

1. **Warm up.** Run the kernel several times before measuring so that caches, page tables, and JIT state are steady.
2. **Repeat and report the median,** not the mean. The median is robust to outliers such as OS preemption and clock boost. Report the spread, for example P10/P90, alongside.
3. **Use events, not host timers,** for device work (Chapter 6, §6.4).
4. **Fix the environment.** Record the GPU (`nvidia-smi -L`), the CUDA version (`nvcc --version`), the driver, and the compiler flags. `-arch`, `-O3`, and `--use_fast_math` all change results.
5. **Verify the output** before trusting the timing. A fast wrong kernel is not a result.

```
// The protocol in miniature. ncu or nsys can validate further, but this
// structure is the minimum reproducible measurement.
float benchmarkKernel(int iters)
{
    // warm-up:
    kernel<<<grid, block>>>(...);  cudaDeviceSynchronize();

    std::vector<float> times;
    for (int r = 0; r < iters; ++r)
    {
        cudaEventRecord(start);  kernel<<<grid, block>>>(...);
        cudaEventRecord(stop);   cudaEventSynchronize(stop);
        float ms;  cudaEventElapsedTime(&ms, start, stop);
        times.push_back(ms);
    }
    std::sort(times.begin(), times.end());
    return times[times.size() / 2];  // median
}
```

16.8 Verification: Differential and Property Testing

Performance engineering without correctness is not useful. Two techniques from the capstone generalise:

- **Differential testing** compares the GPU result with a trusted CPU reference (Chapter 15, §15.8). Run it in CI on every change. A refactor that changes the last bit of a reduction (Chapter 5, §5.6) is caught rather than shipped.
- **Property testing** asserts invariants that hold for any input: a histogram’s counts sum to the input length, a transpose’s output is the input’s transpose, and an edge map of a constant image is all zeros. Property tests find bugs that differential tests miss because both implementations can be wrong in the same way.

Once the differential and property suites exist, an optimisation is just a change run through the suite. This is what lets the optimisation progression of Chapters 7-9 proceed without fear.

16.9 The Engineering Loop

The chapter reduces to a loop:

1. **Measure** (`nsys` timeline; is the GPU busy?).
2. **Profile** (`ncu`; what is the kernel's bottleneck?).
3. **Hypothesise** (name the chapter that addresses the bottleneck).
4. **Change one thing**.
5. **Re-measure** (median of many runs, fixed environment).
6. **Verify** (differential and property tests still pass).

Skipping step 2 or 6 is guessing. Following all six steps is engineering. Everything in this book, the roofline of Chapter 1, the coalescing of Chapter 7, and the pipelines of Chapter 15, is an argument about what step 3 should say. The loop is how the argument is checked.

The Profiler as a Hypothesis Machine

Nsight Compute reports many metrics, and it is tempting to read them as grades: occupancy 80%, memory throughput 90%, therefore good. The professional reading is different: each metric is a hypothesis about why the kernel is not as fast as it could be, and metrics only make sense when combined into an explanation.

A kernel with low achieved occupancy and high memory stall time suggests that too few warps are resident to hide global-load latency. The fix is not “increase occupancy”; it is “increase the pool of ready warps without spilling registers or starving shared memory”, which may mean reducing registers, changing block size, or restructuring the kernel. A kernel with very high memory throughput on a stage the roofline predicted to be memory-bound confirms that memory is the wall; the move is to reduce the amount of memory traffic by tiling, vectorisation, or an algorithmic change, not to tune the access pattern further. A stall reason such as `long_scoreboard` points to global loads; `barrier` points to `__syncthreads`. Each points to a different chapter.

The same mindset applies to Compute Sanitizer. A clean `memcheck` or `racecheck` run is evidence that one class of bug was not detected on the inputs that ran. Races are timing-dependent and memory errors depend on the addresses touched. The tools find classes of bugs; differential and property tests verify behaviour. Together they make an optimisation trustworthy.

The engineering loop exists because a single number does not give the answer. It says what to change next. The kernel is the hypothesis, the profiler is the instrument, and the median-of-many-runs measurement is the experiment.

Common Pitfalls

- Profiling a debug build. Optimised builds have different register usage, inlining, and performance; always profile release builds.
- Trusting one run. Kernels are subject to clock boost, thermal state, and OS noise; report the median of many runs.
- Changing two variables between measurements. The result is uninterpretable; change one thing, re-measure, and repeat.
- Skipping warm-up. The first launch pays JIT, page-table, and cache warm-up costs that are not part of steady-state performance.
- Believing a fast wrong kernel is a result. Verify output before trusting timings.

Check Your Understanding

Why does ncu replay the kernel under instrumentation?

Full counter collection changes timing and state. Replaying the kernel multiple times under instrumentation lets the profiler aggregate hardware counters without the distortion of a single heavily instrumented run.

Why report the median instead of the mean?

The median is robust to outliers such as OS preemption, clock boost, and thermal throttling. The mean is dragged by rare large outliers and does not represent the typical run.

What does racecheck catch that a passing test does not?

A race is timing-dependent and may not manifest on the inputs tested. **racecheck** instruments memory accesses and detects unsynchronised read/write pairs even when the race happens to produce the right answer on the test cases.

Key Takeaways

- `nsys` answers where the time goes; `ncu` answers why a kernel is slow.
- Compute Sanitizer finds what kernels hide: memcheck, racecheck, initcheck, synccheck.
- `cuda-gdb` debugs kernels interactively, thread by thread.
- The benchmark protocol: warm up, repeat, report the median, use events, fix the environment, verify the output.
- The loop measure, profile, hypothesise, change one thing, re-measure, verify underlies every performance claim in this book.

16.10 Exercises

1. A kernel shows 100% memory throughput in `ncu`, but the application is slow. Which profiler do you run next, and what do you look for?
2. `racecheck` reports a race in a kernel that passes all tests. Explain why this is not a contradiction and what class of bug it represents (Chapter 5, §5.1).
3. Why does the benchmark report the median rather than the mean? Give a concrete source of outliers that the median neutralises.
4. A colleague optimises a kernel and reports a 30% speedup measured with `std::chrono` around a single launch. List three things wrong with that measurement.

Sources and Further Reading

- NVIDIA, *Nsight Systems User Guide*: <https://docs.nvidia.com/nsight-systems/>
- NVIDIA, *Nsight Compute User Guide*: <https://docs.nvidia.com/nsight-compute/>
- NVIDIA, *Compute Sanitizer User Guide*: <https://docs.nvidia.com/compute-sanitizer/>
- NVIDIA, *CUDA C++ Best Practices Guide*, “Profiling” section: <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>

Chapter 17: GPU Systems Programming & Memory-Mapped I/O

Chapters 3-16 treated the GPU as an API: `cudaMalloc`, `cudaMemcpy`, and `kernel<<<...>>>`. This chapter examines the systems side: what happens on the wire when the CPU tells the GPU to do something, why device memory differs from host memory, how data moves without the CPU, and what *memory-mapped I/O*, *BAR*, *DMA*, and *IOMMU* mean for a GPU. This knowledge is not required to write correct CUDA, but it is required to explain why a kernel or transfer is slow rather than guessing.

17.1 The GPU Is a PCIe Device

A discrete GPU, and an integrated GPU on a Jetson module, connects to the CPU through a **PCIe** (Peripheral Component Interconnect Express) link. PCIe is a packet-based serial bus that replaced the parallel PCI bus. Every PCIe device, whether GPU, NVMe SSD, network card, or USB controller, appears to the CPU as a device with:

- a **configuration space**, a 4 KB region the CPU reads at boot to discover the device, its vendor, BARs, and interrupts;
- one or more **BARs** (Base Address Registers) that tell the CPU where the device's control registers and device memory are mapped in the CPU's physical address space;
- **MMIO regions** for control registers, doorbells, and mailboxes;
- **DMA capabilities**, so the device can read and write system memory directly.

GPUs are PCIe devices with unusually large amounts of device memory and high DMA bandwidth.

Primitive - PCIe. A packet-switched, point-to-point serial bus. Each lane is a differential pair; x16 means 16 lanes. PCIe Gen4 x16 delivers roughly 32 GB/s raw in each direction, which is why host-device copies top out around 20-25 GB/s in practice (Chapter 4).

Primitive - BAR (Base Address Register). A PCIe configuration-space register that defines where a device's registers and memory are mapped into the CPU's physical address space. The CPU can read and write those addresses with ordinary load and store instructions.

CUDA has `cudaMemcpy` because the CPU cannot ordinarily see GPU memory as RAM. GPU memory lives behind the device's BAR or a DMA mapping, and its cost model differs from host RAM: a CPU load from a GPU MMIO region can be extremely slow, while a DMA engine can move gigabytes at near-bus speed without the CPU touching every byte. The CUDA API manages these two worlds.

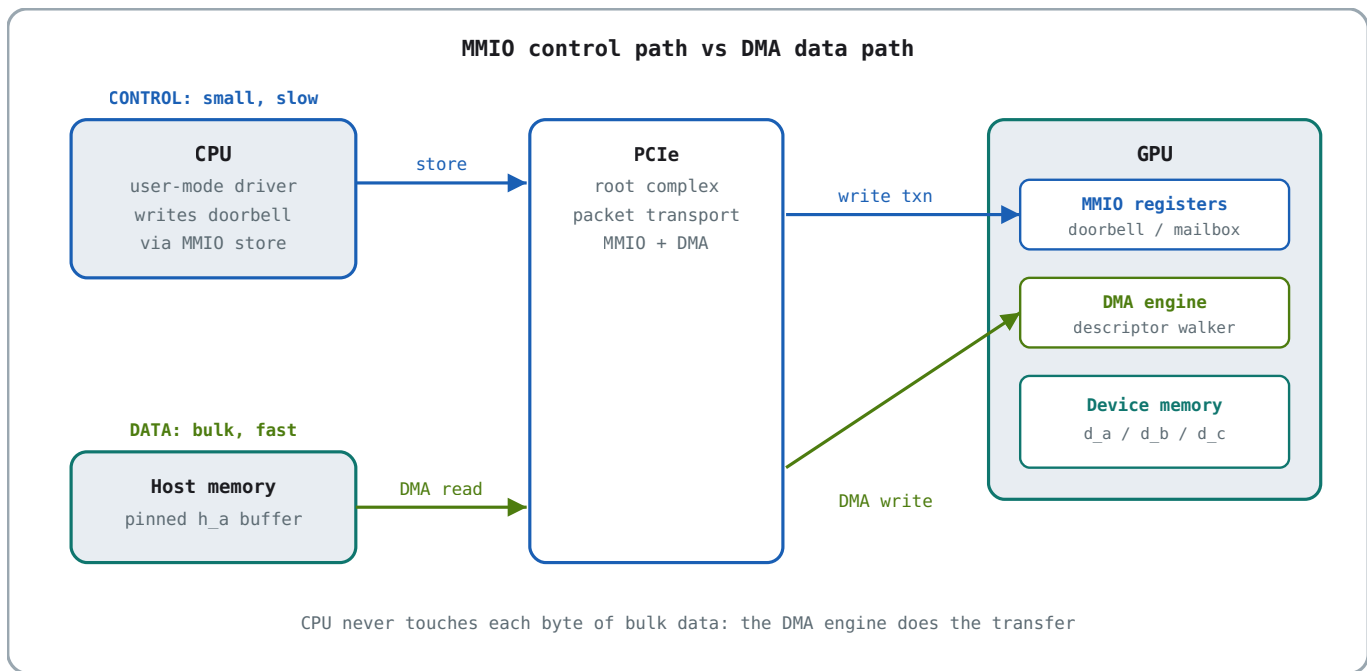
17.2 Memory-Mapped I/O: The Control Path

Memory-mapped I/O (MMIO) exposes a device's control registers as if they were memory addresses. The CPU writes a command by storing a value to an address; the PCIe controller converts that store into a PCIe write transaction that lands in the device's register file.

For a GPU, the MMIO region contains items such as:

- the **doorbell**, a register the CPU writes to tell the GPU that new work has been queued;
- **command buffers or push buffers**, rings of commands the CPU writes into host memory and then announces with the doorbell;
- **mailbox registers** for small status exchanges;
- **performance, temperature, and power registers**, exposed through NVML and `nvidia-smi` but read through the same MMIO path.

The store is **posted**: the CPU does not wait for the GPU to process the command. This is why kernel launches are asynchronous (Chapter 6): the host writes the launch command and rings the doorbell, and the GPU processes the work when it reaches it.



Primitive - MMIO. A device's control registers are mapped into the CPU's physical address space; CPU loads and stores to those addresses become PCIe transactions to the device. MMIO is the control path: small, slow, and used for commands and status, not bulk data.

Every MMIO read or write is a round trip across the bus into the device's register file. A CPU store to MMIO can take hundreds of nanoseconds and cannot be speculatively repeated. Bulk data therefore moves by DMA, not MMIO. MMIO tells the GPU where the data is; DMA moves the data.

17.3 BARs and PCIe Configuration Space

Linux enumerates every PCIe device at boot and reads its configuration space. The relevant fields for a GPU are:

- **Vendor ID and Device ID**, for example NVIDIA's vendor ID is `0x10de`;
- **Class Code**, `0x03` for display controllers;
- **BARs**, up to six 32-bit or 64-bit base address registers;
- **MSI-X** interrupt configuration.

GPU BARs typically map:

- **BAR0**: MMIO register space (control registers, doorbells);
- **BAR1/BAR2**: a window into the GPU framebuffer (device memory). On some systems this window supports legacy framebuffer consoles;
- **BAR3+**: device-specific regions such as UEFI GOP, ATS, or resizable BAR.

On Linux, `lspci` exposes this:

```
lspci | grep -i nvidia
# 01:00.0 3D controller: NVIDIA Corporation GA102 [GeForce RTX 3080] (rev a1)

lspci -v -s 01:00.0
# ... Region 0: Memory at f0000000 (64-bit, prefetchable)
#      Region 2: Memory at ... (64-bit, prefetchable)
```

Resizable BAR. Modern GPUs support Resizable BAR (called Smart Access Memory on AMD platforms). The BAR window into device memory can be enlarged so that the CPU can map a large fraction, or all, of GPU memory into the CPU address space. This is the physical substrate that makes unified memory and zero-copy efficient on modern systems: the CPU can reach device memory through the BAR with ordinary loads and stores, subject to PCIe latency.

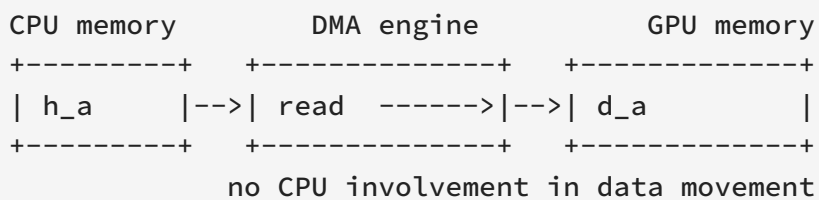
Primitive - device memory window. The GPU framebuffer is not directly addressable by the CPU by default. A BAR creates a CPU address window that the CPU can map and access, but each access crosses PCIe and obeys bus latency and ordering rules. `cudaMalloc` returns a device pointer, not a CPU pointer; `cudaHostAlloc` returns a host pointer that the GPU's DMA engine can reach.

17.4 DMA: The Data Path

If MMIO is the control path, **DMA** (Direct Memory Access) is the data path. A DMA engine copies data between system memory and device memory without the CPU touching each

byte. The sequence for `cudaMemcpy(d_a, h_a, nBytes, cudaMemcpyHostToDevice)` is roughly:

1. The user-mode driver pins the host buffer. For pageable memory it may first copy into a pinned staging buffer (Chapter 4).
2. The CPU programs a **DMA descriptor** containing the source physical address, destination device address, and length.
3. The CPU rings the DMA engine's doorbell.
4. The DMA engine walks the descriptor, reads from system memory, and writes to device memory over PCIe. The CPU is free to do other work.
5. An interrupt or doorbell response notifies the driver that the copy is complete.



Primitive - DMA (Direct Memory Access). A hardware engine that copies data between memory domains, such as host memory and device memory, without CPU per-byte involvement. The CPU sets up a descriptor and the engine performs the bulk transfer.

Pageable memory needs a staging copy because the DMA engine requires physical addresses that remain valid while the transfer is in flight. Ordinary `malloc` pages can be swapped or moved by the OS. Pinning the pages with `cudaMallocHost` guarantees the physical pages stay put, so DMA can access them directly. This is the systems-level reason for Chapter 4's rule: pin what you stream.

17.5 IOMMU/SMMU: The DMA Firewall

An **IOMMU** (Intel/AMD terminology) or **SMMU** (ARM terminology) sits between devices and system memory. It does for DMA what the CPU's MMU does for CPU loads: it translates device virtual addresses to physical addresses and enforces permissions.

Without an IOMMU, a buggy or malicious device could DMA to any physical address, including kernel memory. With an IOMMU:

- the device receives virtual addresses mapped by the IOMMU to physical pages;
- the CPU can revoke mappings, which matters for hot-unplug and isolation;
- the device cannot touch memory that was not explicitly granted;
- scatter-gather becomes natural: non-contiguous physical pages can be presented to the device as a contiguous list.

The trade-off is translation overhead and, historically, lower bandwidth on some platforms. High-performance GPU users sometimes disable the IOMMU or use a bypass mode after measuring. `cudaHostAlloc` with pinned memory remains the fastest path because the driver can pre-map pinned pages in the IOMMU once and reuse the mapping.

Primitive - IOMMU/SMMU. A hardware unit that translates and validates DMA addresses, giving devices virtual addresses and preventing access to arbitrary physical memory.

For CUDA programmers this means `cudaMemcpy` is not “memcpy on the GPU”; it is a sequence of mapping, descriptor programming, doorbell, DMA, and unmap. When `cudaMemcpyAsync` takes longer than expected, part of the cost can be page-table and IOMMU work rather than the wire transfer.

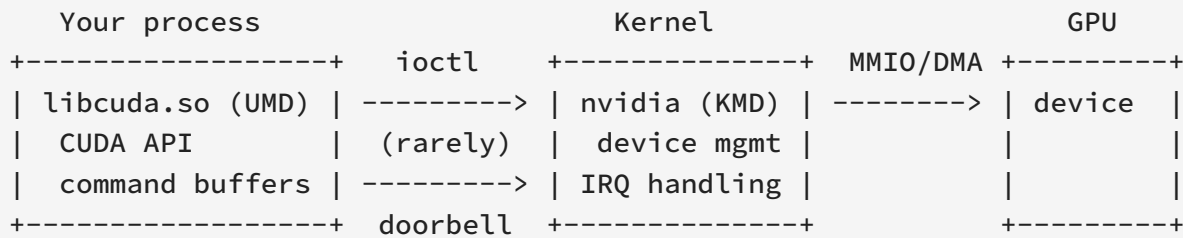
17.6 User-Mode Driver vs Kernel-Mode Driver

CUDA has two driver layers:

- **Kernel-Mode Driver (KMD)** runs in the kernel, as `nvidia` or `nvgpu` on Jetson. It owns the device, MMIO mappings, interrupts, power, and memory mappings. Only the kernel can program the device directly.
- **User-Mode Driver (UMD)** runs in the process as `libcuda.so`. It implements the CUDA API, launches, memory management, and context state. For performance, the UMD avoids kernel round trips by writing commands into user-mapped command buffers and ringing the doorbell directly. This is why modern GPU launches can be cheap: many do not require a kernel call.

The split explains why:

- a CUDA context is per-process rather than per-thread;
- most CUDA API calls do not enter the kernel; they operate on command buffers in user space;
- a GPU crash takes down the context rather than the whole system; the KMD resets the GPU and returns an error.



Primitive - UMD (User-Mode Driver). The library linked into the process. It implements the CUDA API, manages per-process state, and submits work to the GPU with as few kernel transitions as possible. **Primitive - KMD (Kernel-Mode Driver).** The kernel module that owns the device, handles interrupts and power, and is the only component allowed to program the hardware directly.

17.7 Observing the System on Linux

Linux exposes this machinery without writing a driver:

```
# PCIe topology
lspci -tv

# GPU MMIO regions / BARs
lspci -v -s $(lspci | grep -i nvidia | awk '{print $1}' | head -1)

# Kernel driver in use
lspci -k -s $(lspci | grep -i nvidia | awk '{print $1}' | head -1)

# Interrupts / IOMMU groups
ls /sys/kernel/iommu_groups/
cat /proc/interrupts | grep -i nvidia

# Device memory size as the kernel sees it
cat /sys/bus/pci/devices/*/resource 2>/dev/null | head
```

On a Jetson the GPU is part of the SoC, but the same concepts appear through `/sys/class/misc/nvhost-*`, debugfs, and the `nvgpu` driver interface.

Inspect the BAR sizes, the driver name, and whether the device is in an IOMMU group. A GPU behind an IOMMU with a slow translation path explains copies that run below the link specification.

17.8 From MMIO to CUDA APIs

Every CUDA API maps onto the systems concepts above:

CUDA API	Systems concept
<code>cudaMalloc</code>	Allocates device memory managed by the KMD; returns a device pointer, not a CPU pointer
<code>cudaMemcpy</code>	Pins or maps host memory, programs a DMA descriptor, rings a doorbell
<code>cudaMemcpyAsync</code>	Same, but queued in a stream so DMA can overlap kernels
<code>cudaMallocHost</code>	Allocates pinned host memory so DMA can access it directly
<code>cudaHostAllocMapped</code> / <code>cudaHostGetDevicePointer</code>	Maps host memory into the device address space (zero-copy)
<code>cudaMallocManaged</code>	Uses page fault and migration machinery, plus IOMMU or BAR mapping, to present one virtual address space
<code>cudaDeviceEnablePeerAccess</code>	Programs the GPU's P2P DMA path (Chapter 18)

CUDA is a systems API presented as a math API. Every call is a transaction with a driver, a DMA engine, and a memory map. When performance is surprising, the explanation is usually in this model: MMIO for control, DMA for data, IOMMU for safety, and pinning for speed.

CPU Access to Device Memory

The common question “why can’t I dereference a device pointer on the host?” has a precise systems answer:

1. The device pointer is a GPU virtual address, meaningful only in the GPU’s address space.
2. GPU memory sits behind a BAR. The CPU could map it, but mapping all of device memory into the CPU address space costs page-table and IOMMU entries, every CPU access crosses PCIe and is slow, and CPU accesses must be ordered with GPU accesses.
3. The driver therefore returns a handle, the device pointer, and provides explicit copy APIs. The copy API performs the transfer efficiently with DMA, and the driver handles ordering.

Unified memory differs because the driver presents one virtual address that both the CPU and GPU can dereference. The driver uses page faults, migrations, and on modern systems a BAR window or ATS (Address Translation Services). Every fault or migration is a systems operation with hidden latency, which is why Chapter 4 recommends explicit prefetching.

Common Pitfalls

1. **Treating device pointers as host pointers.** Dereferencing a device pointer on the CPU is undefined behaviour and usually a segmentation fault.
2. **Not pinning streaming buffers.** Pageable memory forces a staging copy and silently destroys async-copy performance.
3. **Believing MMIO is ordinary memory.** MMIO is not cacheable RAM; a CPU store to a doorbell is a command, not data storage. MMIO is not for bulk data.
4. **Ignoring IOMMU overhead.** DMA through an IOMMU can be measurably slower on some platforms. Measure with and without, then decide.

5. **Assuming `cudaMemcpy` is synchronous by hardware design.** It is synchronous in the API sense, but underneath it is a DMA operation. The cost is descriptor setup plus transfer, which is why async copies on pinned memory overlap well.

Check Your Understanding

What is the difference between MMIO and DMA?

MMIO is the control path: the CPU writes commands and status to device registers through PCIe transactions, such as a doorbell. DMA is the data path: a hardware engine moves bulk data between memory domains without CPU per-byte involvement. MMIO tells the device what to do; DMA moves the data it operates on.

Why does pageable host memory need a staging copy for DMA?

DMA requires physical addresses that remain valid for the duration of the transfer. Pageable pages can be swapped or moved by the OS, so the runtime copies the data into a pinned staging buffer whose physical pages are locked. Pinned memory (`cudaMallocHost`) skips that staging copy.

What does the IOMMU protect against?

It prevents a device from DMAing to arbitrary physical memory. The IOMMU translates device virtual addresses to physical addresses and enforces permissions, so a device can touch only memory the driver explicitly mapped for it.

Exercises

1. Run `lspci -v` on your machine and identify the GPU's BARs. What is the size of the BAR that maps device memory? On Jetson, inspect the SoC memory map instead.
2. Explain, using the MMIO/DMA model, why `cudaMemcpyAsync` can overlap with a kernel while synchronous `cudaMemcpy` cannot.
3. Draw the full path of a `cudaMemcpy` from a pinned host buffer to device memory, naming every component: UMD, KMD, IOMMU, DMA engine, PCIe, and device memory.

4. Why is exposing device memory as a plain CPU-mapped BAR and letting applications dereference device pointers directly a bad idea? Give two reasons from this chapter.

Sources and Further Reading

- NVIDIA, *CUDA C++ Programming Guide*, “Hardware Implementation” and “Compute Capabilities”: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- PCI-SIG, *PCI Express Base Specification* and `lspci/pciutils` documentation for device enumeration.
- Linux kernel documentation, “DMA-API” and “IOMMU” sections: <https://www.kernel.org/doc/html/latest/core-api/dma-api.html>

Chapter 18: NVLink & NVSwitch - The Multi-GPU Interconnect

The previous chapters were single-GPU. This chapter introduces the hardware that supports multi-GPU programming: **NVLink**, NVIDIA's high-bandwidth point-to-point interconnect, and **NVSwitch**, a switch that connects many NVLink ports. The chapter covers topology, memory semantics, peer-to-peer copies in CUDA, and when NVLink matters compared with PCIe.

18.1 Scaling Beyond One GPU

Training a large model or processing a large dataset eventually exceeds one GPU's memory and compute. The classic scaling strategies are:

1. **Model parallelism** - split the model across GPUs; each GPU holds a piece.
2. **Data parallelism** - each GPU holds a full model copy but processes a different batch. Gradients must be reduced across GPUs every step.
3. **Pipeline parallelism** - different model stages live on different GPUs.

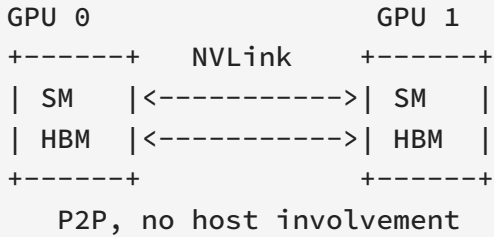
All three move data between GPUs. Inter-GPU communication speed determines whether a multi-GPU system scales or idles waiting for communication. NVLink exists to make that communication fast.

18.2 NVLink: Point-to-Point, High-Bandwidth

NVLink is a high-bandwidth, low-latency, point-to-point interconnect between two GPUs, and on some platforms between a GPU and CPU. It is not PCIe. Its key properties:

- **Higher bandwidth than PCIe.** An NVLink connection is bidirectional. NVLink 3.0 (Ampere) provides 25 GB/s per direction per link; an A100 has 12 links, for 600 GB/s of total bidirectional bandwidth. PCIe Gen4 x16 provides about 32 GB/s total. NVLink can be an order of magnitude faster than PCIe for peer-to-peer traffic.

- **Lower latency for small messages.** NVLink uses a protocol designed for GPU-to-GPU traffic rather than a PCIe root-complex round trip.
- **Direct GPU-to-GPU DMA.** One GPU's DMA engine can read and write another GPU's memory without bouncing through host memory.



Primitive - NVLink. NVIDIA's proprietary high-bandwidth, point-to-point GPU interconnect. It carries data and, on supported architectures, coherent memory traffic directly between GPUs.

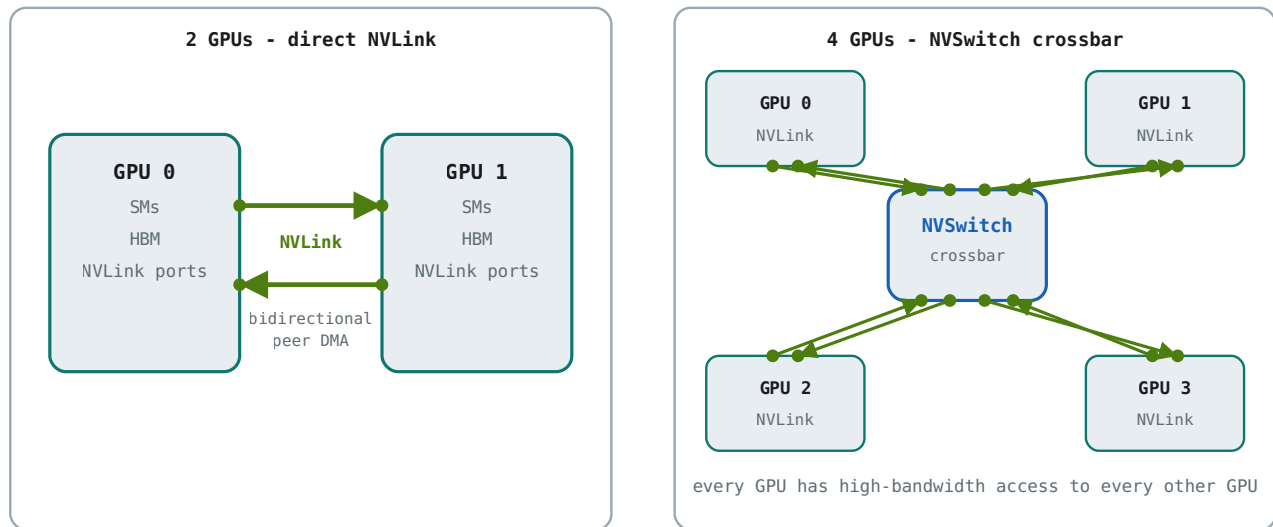
Each GPU has a fixed number of NVLink links. An A100 has 12 links, an H100 has 18, and consumer cards have fewer or none; many consumer cards use PCIe only. Links can be used as direct GPU-to-GPU connections in two-GPU systems or as connections through an NVSwitch in larger systems.

18.3 NVSwitch and Topologies

A full mesh of N GPUs requires $N(N - 1)/2$ links per GPU. For eight GPUs that is 28 links per GPU, which is impractical. **NVSwitch** provides a crossbar inside the chassis: each GPU connects to the switch, and the switch forwards traffic between any pair.

- **2 GPUs:** direct NVLink; no switch is needed.
- **4-GPU HGX baseboards:** four GPUs connected to two NVSwitches.
- **8-GPU HGX baseboards:** eight GPUs connected to NVSwitches in a topology that gives every GPU high-bandwidth access to every other GPU.

NVLink topologies: direct link vs NVSwitch



This is why `nvidia-smi topo -m` can report `NV#` for peer paths: the topology decides whether a copy between two GPUs uses a direct link, a switch hop, or a PCIe root-complex path.

Primitive - NVSwitch. A crossbar switch that connects many GPUs' NVLink ports, giving each GPU high-bandwidth access to every other GPU without a full mesh of direct links.

18.4 NVLink Memory Semantics

NVLink is not just fast PCIe. It changes the memory model between GPUs:

- **Peer-to-peer (P2P) access.** A kernel on GPU 0 can read and write GPU 1's memory when peer access is enabled and the topology allows it. The access travels over NVLink rather than host memory.
- **Peer atomics.** Atomic operations can target another GPU's memory. This enables lock-free multi-GPU algorithms, but peer-atomic throughput over NVLink is lower than local HBM atomics. Use them sparingly.

- **Coherent and address-translation features.** NVLink-C2C (chip-to-chip) on Grace-Hopper carries coherent CPU-GPU traffic with hardware-managed cache coherence. The CPU and GPU can share one unified memory domain.
- **Unified memory over NVLink.** With `cudaMallocManaged`, pages can migrate between GPUs over NVLink. This is convenient, but page migration is a system operation and can be slower than explicit P2P copies.

The practical rule is: explicit P2P copies such as `cudaMemcpyPeerAsync` are predictable and fast; unified memory is convenient but must be measured.

18.5 Peer-to-Peer in CUDA

The CUDA API for P2P is small:

```
// 1. Query whether P2P is possible and enable it.
int canAccess = 0;
cudaDeviceCanAccessPeer(&canAccess, device0, device1);
if (canAccess) {
    cudaSetDevice(device0);
    cudaDeviceEnablePeerAccess(device1, 0);
}

// 2. Copy directly between device memories.
cudaMemcpyPeerAsync(d_buf1, device1,
                    d_buf0, device0,
                    bytes, stream);

// 3. Or, once peer access is enabled, a kernel on GPU 0 can read GPU 1's
//     pointer directly (subject to topology and architecture support).

// 4. Disable when done.
cudaDeviceDisablePeerAccess(device1);
```

Primitive - peer access. The CUDA mechanism that lets one device access another device's memory. It requires hardware support (NVLink or PCIe P2P), a compatible topology, and explicit enabling with `cudaDeviceEnablePeerAccess`.

Check the topology first. `cudaDeviceCanAccessPeer` returns true only when the platform supports it. Two GPUs can sometimes use P2P over PCIe when they share a root complex; NVLink-connected GPUs always support it. `nvidia-smi topo -m` shows which GPUs are connected:

```
nvidia-smi topo -m
#           GPU0  GPU1  GPU2  GPU3  ...
# GPU0      X    NV#   NV#   NV#
# ...
```

18.6 When NVLink Matters

Multi-GPU execution time is approximately:

```
total_time = compute_time + communication_time
speedup = single_gpu_time / (compute_time/N + communication_time)
```

If communication time is significant, adding GPUs does not scale. NVLink helps by shrinking communication time.

When NVLink matters:

- Frequent all-reduces, such as gradient synchronisation in data-parallel training.
- Pipeline parallelism in which activations pass between GPUs.
- Fine-grained P2P reads in multi-GPU databases or graph analytics.

When PCIe is sufficient:

- One-time dataset uploads from host to GPU.
- Coarse task parallelism in which GPUs rarely communicate.
- Communication that is small relative to compute, as in embarrassingly parallel batches.

Apply the measurement discipline of Chapter 16. Use `cudaMemcpyPeer` with CUDA events or `nccl-tests` (Chapter 19) to measure the achievable P2P bandwidth on the actual hardware.

18.7 Topology Discovery

```
# Matrix of GPU-to-GPU links (NV# = NVLink, PIX/PXB = PCIe paths)
nvidia-smi topo -m

# Detailed NVLink status (link count, active links, errors)
nvidia-smi nvlink -s
```

The NVML API exposes the same information programmatically through functions such as `nvmlDeviceGetTopologyCommonAncestor`, which is useful in tools that must adapt to the hardware.

NVLink and the Data Path

The surface answer is higher bandwidth. The deeper answer concerns the data path.

PCIe P2P between two GPUs often passes through the root complex: GPU 0, PCIe switch or root complex, GPU 1. This adds latency and shares the host's PCIe bandwidth. NVLink is a direct GPU-to-GPU link, or a path through a switch crossbar. There is no host-memory hop, no root-complex arbitration, and the protocol is designed for GPU memory semantics, including atomics and, on some architectures, coherence.

The result is lower latency per message and more concurrent independent communication streams. NCCL (Chapter 19) prefers NVLink topologies because collective algorithms require many simultaneous point-to-point transfers.

NVLink-C2C on Grace-Hopper connects CPU and GPU with coherent memory semantics. The CPU and GPU can share a memory pool with hardware coherence, removing copies from the programming model. This differs from the discrete PCIe model but uses the same P2P, atomics, and topology concepts.

Common Pitfalls

1. **Assuming all GPUs can do P2P.** Check `cudaDeviceCanAccessPeer`; many consumer platforms do not support PCIe P2P or require special settings.

2. **Using unified memory instead of explicit P2P on hot paths.** Page migration over NVLink is not free. Measure before replacing `cudaMemcpyPeerAsync`.
3. **Ignoring topology.** Two GPUs on different PCIe switches can have a much slower path than two GPUs sharing a root complex. Read `nvidia-smi topo -m`.
4. **Using peer atomics on hot paths.** NVLink atomics are slower than local HBM atomics; use them for rare synchronisation rather than per-element updates.
5. **Forgetting to disable peer access** before changing device context or shutting down. The runtime can otherwise leave stale mappings.

Check Your Understanding

Why is NVLink faster than PCIe for GPU-to-GPU traffic?

NVLink is a direct high-bandwidth GPU interconnect with low latency and no host or root-complex round trip. PCIe P2P often routes through the root complex and shares host PCIe bandwidth. NVLink is purpose-built for GPU memory semantics and carries more concurrent traffic.

What does NVSwitch add over direct NVLink?

It lets more than two GPUs communicate at high bandwidth without a full mesh of direct links. Each GPU connects to the switch and the switch forwards traffic between any pair, enabling all-to-all patterns in four- and eight-GPU systems.

What does `cudaDeviceEnablePeerAccess` do?

It enables one device to access another device's memory directly, subject to hardware support and topology. After enabling, `cudaMemcpyPeer` and, in some cases, kernels can read and write peer memory without going through host memory.

Exercises

1. Run `nvidia-smi topo -m` on a multi-GPU machine, or research a DGX topology diagram, and identify which GPU pairs are NVLink-connected.
2. Write a small program that measures `cudaMemcpyPeer` bandwidth between two GPUs with CUDA events and compare it with host-device copy bandwidth.

3. Explain why a full mesh of direct NVLink links is impractical for eight GPUs and how NVSwitch solves the problem.
4. In data-parallel training, gradients are all-reduced every step. Would you prefer NVLink or PCIe for that workload? Justify with the model in §18.6.

Sources and Further Reading

- NVIDIA, *NVLink & NVSwitch* product documentation: <https://www.nvidia.com/en-us/data-center/nvlink/>
- NVIDIA, *CUDA C++ Programming Guide*, “Peer-to-Peer Access” and “Unified Memory” sections: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- NVIDIA, *NVIDIA Multi-GPU Communication Library (NCCL)* documentation: <https://docs.nvidia.com/deeplearning/nccl/user-guide/>

Chapter 19: NCCL & Multi-GPU Collective Communication

Chapter 18 described the hardware, NVLink and NVSwitch. This chapter describes the software that turns that hardware into scalable multi-GPU programs: **NCCL** (NVIDIA Collective Communications Library). NCCL implements the **collective operations** - all-reduce, broadcast, all-gather, and reduce-scatter - used by every data-parallel training loop. It is the library behind PyTorch's `DistributedDataParallel` and TensorFlow's `MirroredStrategy`.

19.1 The Multi-GPU Programming Problem

In data-parallel training, each GPU holds a copy of the model and processes a different batch. At the end of a step, each GPU has computed a local gradient. The next step requires the same averaged gradient on every GPU, so the GPUs must combine their gradients and distribute the result. That operation is an **all-reduce**.

A naive implementation would have one GPU collect all gradients, sum them, and broadcast the result. That does not scale: the collector becomes the bottleneck and most links stay idle. Collective algorithms use all links simultaneously in structured patterns.

Primitive - collective operation. An operation that involves every rank in a group and produces a result that depends on data from all ranks. Examples: all-reduce, broadcast, reduce, all-gather, and reduce-scatter.

19.2 Collective Operations

Operation	Input	Output
Broadcast	one rank has data	all ranks have the data
Reduce	all ranks have data	one rank has the combination
All-reduce	all ranks have data	all ranks have the combination
All-gather	each rank has a piece	every rank has all pieces
Reduce-scatter	each rank has data	each rank has one combined piece

The mathematical operation is usually sum, but collectives generalise to min, max, product, and other associative operations. NCCL supports `ncclSum`, `ncclProd`, `ncclMin`, and `ncclMax`.

19.3 NCCL Architecture

NCCL:

- discovers the **topology** (which GPUs are NVLink-connected, which use PCIe, and which are on different hosts);
- builds a **communication plan** (ring, tree, or hybrid);
- creates **channels**, independent communication paths that can run concurrently;
- uses CUDA **streams**, **peer access**, and on modern systems **NVLink atomics and multicast** to move data.

NCCL operations are launched on a CUDA stream like kernels. They are asynchronous and participate in stream ordering:

```
// Each rank does:
ncclCommInitRank(&comm, nranks, ncclUniqueId, rank);
// ... work ...
ncclAllReduce(sendbuff, recvbuff, count,
              ncclFloat, ncclSum,
              comm, stream);
// NCCL is asynchronous; synchronize the stream when results are needed.
```

Primitive - rank. A process or GPU participating in a collective group. Ranks are numbered 0..n-1 and each rank has its own `ncclComm`. **Primitive - communicator**

(`nccclComm`). The NCCL object that represents a rank's membership in a group. Every collective call takes a communicator.

19.4 Ring All-Reduce

The classic NCCL algorithm is the **ring all-reduce**. N GPUs form a ring:

```
GPU0 -> GPU1 -> GPU2 -> ... -> GPU(N-1) -> GPU0
```

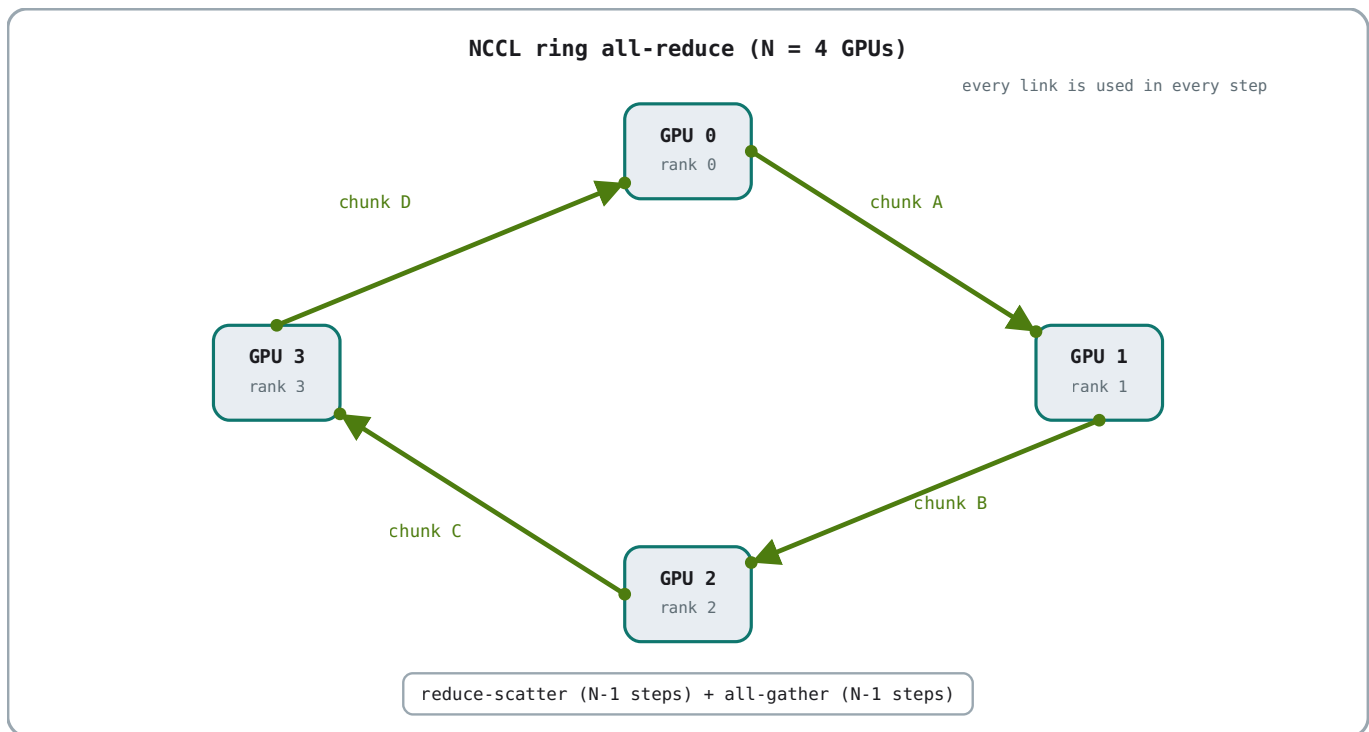
The data is split into N chunks. The algorithm has two phases:

1. **Reduce-scatter**. Each GPU sends a chunk to its neighbour, receives a chunk from the other neighbour, adds its own contribution, and passes the partial result on. After $N - 1$ steps, each GPU holds the complete reduced value for one chunk.
2. **All-gather**. Each GPU sends its reduced chunk around the ring again. After $N - 1$ steps, every GPU has every reduced chunk.

Every link is used in every step. For N GPUs and a message of size M :

- data moved per GPU is approximately $2M(N - 1)/N$;
- bandwidth utilisation is optimal for large messages;
- latency grows with N because there are $N - 1$ steps.

Rings therefore suit large messages rather than tiny ones.



19.5 Tree All-Reduce

For small messages or many ranks, a **tree** algorithm has lower latency than a ring because the number of steps is $O(\log N)$ rather than $O(N)$.

```
GPU0
|-- GPU1
|   |-- GPU3
|   |-- GPU4
|-- GPU2
|   |-- GPU5
|   |-- GPU6
```

- **Reduce phase:** leaves send data upward; each parent sums its children and its own data.
- **Broadcast phase:** the root sends the total down the tree.

Trees place more traffic on the root's links but have lower latency. NCCL chooses ring or tree, or a hybrid, based on message size, rank count, and topology. On NVSwitch systems, **NVLS** (NVLink SHARP) lets the switch perform reduction in flight, so all-reduce can approach the cost of a single send.

Primitive - ring all-reduce. A bandwidth-optimal all-reduce for large messages: reduce-scatter around a ring, then all-gather around the ring. **Primitive - tree all-reduce.** A latency-optimal all-reduce for small messages: a tree reduce followed by a tree broadcast.

19.6 A Complete NCCL Example

A minimal all-reduce program requires a multi-GPU machine with NCCL installed and one process per GPU:

```

// all_reduce.cu - run with one process per GPU, e.g. mpirun -np 4
#include <stdio>
#include <stdlib>
#include <vector>
#include <cuda_runtime.h>
#include <nccl.h>

#define CHECK_CUDA(call) do { cudaError_t e = (call); \
    if (e != cudaSuccess) { fprintf(stderr, "CUDA %s\n", \
    cudaGetErrorString(e)); exit(1); } } while (0)
#define CHECK_NCCL(call) do { ncclResult_t r = (call); \
    if (r != ncclSuccess) { fprintf(stderr, "NCCL %s\n", \
    ncclGetErrorString(r)); exit(1); } } while (0)

int main(int argc, char** argv)
{
    const int nranks = 4;           // number of GPUs/processes
    const int rank   = atoi(argv[1]); // this process's rank (0..n-1)
    const int count   = 1 << 20;     // floats per rank

    CHECK_CUDA(cudaSetDevice(rank)); // rank i uses GPU i in this simple
    setup

    ncclUniqueId id;
    if (rank == 0) ncclGetUniqueId(&id);
    // In real deployments use MPI to broadcast id to all ranks.

    ncclComm_t comm;
    CHECK_NCCL(ncclCommInitRank(&comm, nranks, id, rank));

    float *sendbuf, *recvbuf;
    CHECK_CUDA(cudaMalloc(&sendbuf, count * sizeof(float)));
    CHECK_CUDA(cudaMalloc(&recvbuf, count * sizeof(float)));
    CHECK_CUDA(cudaMemset(recvbuf, 0, count * sizeof(float)));

    // Each rank's data: rank + 1 (so the all-reduce sum is known).
    std::vector<float> h(count, static_cast<float>(rank + 1));
    CHECK_CUDA(cudaMemcpy(sendbuf, h.data(), count * sizeof(float),
                          cudaMemcpyHostToDevice));

    cudaStream_t stream;
    CHECK_CUDA(cudaStreamCreate(&stream));

    // All-reduce: every rank ends with sum = 1+2+3+4 = 10 per element.
    CHECK_NCCL(ncclAllReduce(sendbuf, recvbuf, count,

```

```

        ncclFloat, ncclSum,
        comm, stream));
CHECK_CUDA(cudaStreamSynchronize(stream));

std::vector<float> out(count);
CHECK_CUDA(cudaMemcpy(out.data(), recvbuf, count * sizeof(float),
                      cudaMemcpyDeviceToHost));
printf("rank %d: out[0] = %f (expected 10)\n", rank, out[0]);

CHECK_NCCL(ncclCommDestroy(comm));
CHECK_CUDA(cudaFree(sendbuf));
CHECK_CUDA(cudaFree(recvbuf));
return 0;
}

```

Build and run (simplified; real multi-node runs use MPI or a launcher):

```

nvcc -arch=compute_60 all_reduce.cu -o all_reduce -lncccl
# start one process per GPU with the unique ID propagated by MPI

```

The hard parts of NCCL programming are not the arithmetic. They are creating the communicator, distributing the unique ID, choosing a topology-aware algorithm, and ensuring every rank calls collectives in the same order on compatible streams.

19.7 NCCL in PyTorch

Most applications call NCCL through a framework. PyTorch's `DistributedDataParallel` uses NCCL as its backend:

```

import torch.distributed as dist

dist.init_process_group(backend="nccl", world_size=4, rank=rank)
model = torch.nn.parallel.DistributedDataParallel(model)
# forward/backward
loss.backward()          # DDP hooks an all-reduce of gradients via NCCL

```

The framework handles communicator setup and launches NCCL collectives on the correct streams. Understanding ring and tree behaviour still matters: `NCCL_P2P_LEVEL`, `NCCL_ALGO`, and message size affect whether ring or tree is selected.

19.8 Debugging and Tuning NCCL

```
# Verbose logs: topology, chosen algorithms, channel count
NCCL_DEBUG=INFO ./train.py

# Detailed trace for a single collective
NCCL_DEBUG=TRACE ./train.py

# Force specific algorithms / transports
NCCL_ALGO=Ring ./train.py
NCCL_P2P_LEVEL=NV ./train.py

# Measure raw collective bandwidth/latency (from nccl-tests)
./build/all_reduce_perf -b 8 -e 128M -f 2 -g 4
```

Common issues:

- **Communicator setup hangs** - the unique ID was not distributed correctly or ranks disagree on the world size.
- **Slow all-reduce on small messages** - the selected algorithm may be ring when tree would be better; try `NCCL_ALGO=Tree`.
- **P2P disabled or blocked** - check `NCCL_P2P_LEVEL` and topology.
- **Stream mismatch** - the NCCL call must be on the same stream as the kernels whose results it consumes, or ordered with events.

Ring versus Centralised All-Reduce

A centralised all-reduce makes the collector read $N - 1$ messages, combine them, and write $N - 1$ results. Traffic through one GPU is $O(NM)$ and all other links idle. A ring all-reduce spreads the work: every GPU sends and receives $N - 1$ chunks of size M/N , so total data per GPU is $O(M)$ and all links are busy in every step. For large M , the ring is bandwidth-optimal. For small M , the $N - 1$ serial steps make latency dominate, so a tree with $O(\log N)$ steps wins. NCCL chooses the algorithm by measuring the hardware, applying the same measure-don't-guess discipline as Chapter 16.

On NVSwitch systems, NVLink SHARP lets the switch perform arithmetic while forwarding data. Each GPU sends its data once and receives the reduced result once; the switch does

the combining. This is the multi-GPU analogue of computation in the memory system, and it is why NVLink/NVSwitch plus NCCL is the backbone of large-scale training.

Common Pitfalls

1. **Calling NCCL collectives in different order on different ranks.** Collective operations must be matched across ranks; mismatched order deadlocks or corrupts data.
2. **Forgetting stream ordering.** NCCL calls are asynchronous. Reading results without synchronising or ordering events can race.
3. **Using the default communicator ID everywhere.** In multi-process runs, the unique ID must be generated once and broadcast through MPI, a file, or an environment variable; every rank must use the same ID.
4. **Ignoring topology.** A ring over PCIe-only GPUs is much slower than a ring over NVLink. Check `nvidia-smi topo -m` and `NCCL_P2P_LEVEL`.
5. **Assuming NCCL is only for training.** NCCL supports any all-to-all GPU communication: distributed inference, multi-GPU sorts, graph processing, and scientific computing.

Check Your Understanding

What is the difference between ring and tree all-reduce?

Ring all-reduce is bandwidth-optimal for large messages: each GPU sends and receives $N - 1$ chunks and every link stays busy, but latency grows with N . Tree all-reduce has $O(\log N)$ steps and is better for small messages, but the root and upper links carry more traffic. NCCL chooses between them based on message size, rank count, and topology.

Why must the `ncclUniqueId` be shared among ranks?

The unique ID is the bootstrap token that lets all ranks agree they are joining the same communicator group. Rank 0 generates it, and it must be distributed through MPI, a file, or the environment before `ncclCommInitRank`.

Why is all-reduce the core operation of data-parallel training?

Every GPU computes a local gradient for the same model parameters. The next step needs the same averaged gradient on every GPU, so the gradients must be summed or averaged across all GPUs and the result made available to all. That is exactly an all-reduce.

Exercises

1. Trace ring all-reduce for $N = 4$ and a four-chunk message: list what each GPU sends and receives in each of the six steps (three reduce-scatter and three all-gather).
2. Using `nccl-tests`, measure `all_reduce_perf` for 8 bytes versus 128 MB and explain which algorithm NCCL chose and why.
3. Modify the example program to use `ncclBroadcast` instead of all-reduce: rank 0 sends its buffer and all ranks receive it. Verify with a known value.
4. Explain how NVLink SHARP (NVLS) makes all-reduce cheaper than the ring algorithm and why the switch is a natural place to do arithmetic.

Sources and Further Reading

- NVIDIA, *NCCL User Guide*: <https://docs.nvidia.com/deeplearning/nccl/user-guide/>
- NVIDIA, *NCCL GitHub repository*: <https://github.com/NVIDIA/nccl>
- NVIDIA, *CUDA C++ Programming Guide*, “Peer-to-Peer” and “Streams” sections: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- Patarasuk and Yuan, “Bandwidth Optimal All-reduce Algorithms for Clusters of Workstations,” *Journal of Parallel and Distributed Computing*, 2009. Source of the ring all-reduce analysis.

Epilogue - The Road Ahead

You have worked from the mathematics of parallelism to a streamed, profiled, and verified image pipeline implemented three ways. This epilogue briefly describes where the field is heading.

Hardware

Each GPU generation moves the ridge point of Chapter 1: more FLOPs, more bandwidth, and increasingly specialised arithmetic. Tensor cores, introduced with Volta and central to AI workloads since, execute dense matrix multiplication in hardware instead of looping over CUDA cores. Hopper added the Tensor Memory Accelerator (TMA) for bulk asynchronous copies and thread-block clusters. Blackwell continues the trend. The memory hierarchy, the warp, and the SM from Chapter 2 still define the machine; the arithmetic units are becoming more specialised.

The skills in this book are foundational. Tensor-core programming is still thread-block programming with a different instruction set. TMA is still coalescing expressed in bulk. When the next specialised unit appears, it will fit the same model.

Software

Three currents are visible:

- **The SIMT model is cross-vendor.** NVIDIA's ecosystem (CUDA, cuBLAS, Nsight) remains the reference, but SYCL (Khronos's single-source C++ model), HIP (AMD's CUDA-compatible API), and wgpu/WebGPU (browser and native Rust) expose the same underlying execution model. Grids, warps, coalescing, and shared memory translate directly to these APIs.
- **Rust is becoming viable for GPU work.** `cudarc` provides a production-grade Rust host (Chapter 13). CUDA-Oxide (Chapter 14) is an early attempt to bring Rust's guarantees to the kernel itself. Both are young. They point in the same direction:

many GPU failure modes originate in the host language, and languages that remove those failure modes are likely to become more common for GPU hosts.

- **Libraries continue to absorb implementation work.** Every year more of the hard work moves into tuned libraries (Chapter 11). The engineers who use those libraries effectively are not the ones who memorised the APIs; they are the ones who can read a profiler, explain why a kernel is memory-bound, and decide when a custom kernel is worth writing.

The Discipline

The most durable idea in this book is the loop of Chapter 16: measure, profile, hypothesise, change one thing, re-measure, and verify. Hardware changes, languages change, and libraries change. The loop does not.

The Invitation

This book is published on GitHub Pages and is open to pull requests. If a kernel is unclear, a claim is unmeasured, or a chapter misses a concept that confused you, open an issue.

Write kernels that are fast, correct, and understood.

- *Arpan Pathak*

Appendix A - CUDA API Reference

This appendix is the book's vocabulary list: every type, built-in variable, function and flag used in the main text, with its meaning and where it is discussed. It is intended as a lookup table, not as a tutorial - the tutorials are the chapters.

A.1 Execution Configuration

Syntax	Meaning	Chapter
<code>kernel<<<gridDim, blockDim>>>(args...)</code>	Launch <code>kernel</code> with a grid of <code>gridDim</code> blocks of <code>blockDim</code> threads each	3
<code>kernel<<<gridDim, blockDim, sharedBytes, stream>>></code>	As above, with dynamic shared memory (bytes) and an explicit stream	6, 7
<code>dim3</code>	Three- <code>unsigned</code> vector type; fields <code>.x</code> , <code>.y</code> , <code>.z</code>	3
<code>threadIdx</code>	The thread's position within its block (a <code>dim3</code>)	3
<code>blockIdx</code>	The block's position within the grid (a <code>dim3</code>)	3
<code>blockDim</code>	Threads per block, as launched (a <code>dim3</code>)	3
<code>gridDim</code>	Blocks per grid, as launched (a <code>dim3</code>)	3
<code>__launch_bounds__(maxThreads, minBlocks)</code>	Compiler directive: register budget for occupancy	9

The universal 1-D global index: `blockIdx.x * blockDim.x + threadIdx.x`. The 2-D composition is in Chapter 3, §3.5.

A.2 Function Qualifiers

Qualifier	Runs on	Called from	Chapter
<code>__global__</code>	Device	Host	3
<code>__device__</code>	Device	Device only	3
<code>__host__</code> (default)	Host	Host	3
<code>__host__ __device__</code>	Both	Both	10

`extern "C" __global__` keeps the kernel symbol unmangled for driver-API / NVRTC lookup (Chapters 12, 13).

A.3 Vector Types

Type	Bytes	Alignment	Notes
<code>uchar3</code>	3	1	RGB pixels; no padding (Chapter 15)
<code>float2, float4</code>	8, 16	4, 16	Vectorised loads (Chapter 7, §7.6)
<code>int2, int4, double2, uint4</code>	8/16/16/16	as size	Same vectorisation rules
<code>size_t</code>	platform	-	Byte sizes; use instead of <code>int</code> (Chapter 3)

Vectorised accesses require alignment to the vector size.

A.4 Memory Management

Function	Behaviour	Chapter
<code>cudaMalloc(void** p, size_t n)</code>	Allocate <code>n</code> bytes in device global memory	3
<code>cudaFree(void* p)</code>	Free a device allocation	3
<code>cudaMemcpy(dst, src, n, kind)</code>	Synchronous copy; <code>kind = HostToDevice, DeviceToHost, DeviceToDevice, HostToHost</code>	3
<code>cudaMemcpyAsync(dst, src, n, kind, stream)</code>	Asynchronous copy, queued on <code>stream</code> ; requires pinned host memory	4, 6
<code>cudaMallocHost(void** p, size_t n)</code>	Allocate pinned (page-locked) host memory	4
<code>cudaHostAlloc(void** p, size_t n, flags)</code>	Pinned host memory; <code>cudaHostAllocMapped</code> adds zero-copy mapping	4
<code>cudaHostGetDevicePointer(void** dp, void* hp, 0)</code>	Device pointer for zero-copy mapped host memory	4
<code>cudaFreeHost(void* p)</code>	Free pinned host memory	4
<code>cudaMallocManaged(void** p, size_t n)</code>	Unified memory (host + device address space)	4
<code>cudaMemPrefetchAsync(p, n, device, stream)</code>	Migrate unified-memory pages now	4
<code>cudaMemcpyToSymbol(sym, src, n)</code>	Copy into <code>__constant__</code> memory	7

Memory kinds and when to use each: Chapter 4, §4.7.

A.5 Synchronisation and Memory Ordering

Primitive	Meaning	Chapter
<code>__syncthreads()</code>	Block-wide barrier; must be uniformly reachable	5
<code>__threadfence()</code>	Order my device-scope global accesses	5
<code>__threadfence_block()</code>	Order my block-scope accesses	5
<code>__threadfence_system()</code>	Order host+device accesses	5
<code>volatile</code>	Disable register caching of a location	5
<code>atomicAdd/Sub/Exch/CAS/Min/Max/And/Or/Xor</code>	Hardware read-modify-write; return old value	5
<code>cuda::atomic<T, scope></code>	C++20-style atomic with memory orders (CUDA 12)	10

The atomics table with semantics: Chapter 5, §5.5.

A.6 Streams and Events

Function	Behaviour	Chapter
<code>cudaStreamCreate(&s)</code>	Create a stream	6
<code>cudaStreamDestroy(s)</code>	Destroy a stream	6
<code>cudaStreamCreateWithFlags(&s, flag)</code>	<code>cudaStreamNonBlocking</code> disables default-stream sync	6
<code>cudaStreamCreateWithPriority(&s, flag, prio)</code>	Priority stream; range from <code>cudaDeviceGetStreamPriorityRange</code>	6
<code>cudaStreamSynchronize(s)</code>	Wait for all work in <code>s</code>	6
<code>cudaEventCreate(&e) / cudaEventDestroy(e)</code>	Create/destroy an event	6
<code>cudaEventRecord(e, s)</code>	Mark the stream position	6
<code>cudaEventSynchronize(e)</code>	Wait until the device reaches <code>e</code>	6
<code>cudaEventElapsedTime(&ms, e0, e1)</code>	Time between two events	6, 16
<code>cudaStreamWaitEvent(s, e)</code>	Make <code>s</code> wait for <code>e</code> (cross-stream dependency)	6
<code>cudaGraphCreate/Instantiate/Launch/Destroy</code>	Capture and replay device work	6
<code>cudaDeviceSynchronize()</code>	Wait for all device work	3, 6

The synchronisation cheat sheet: Chapter 6, §6.8.

A.7 Error Handling

Primitive	Behaviour	Chapter
<code>cudaError_t</code>	Enum; <code>cudaSuccess == 0</code> , everything else is an error	3
<code>cudaGetLastError()</code>	Return and clear the last asynchronous error	3
<code>cudaGetErrorString(e)</code>	Human-readable error text	3
<code>cudaDeviceSynchronize()</code>	Also surfaces async kernel errors	3, 6
<code>cublasStatus_t</code> , <code>nVRTCResult</code> , <code>CUresult</code>	Library error types (cuBLAS, NVRTC, driver API)	11, 12

Two error modes (synchronous vs asynchronous) and the **CHECK** discipline: Chapter 3, §3.8.

A.8 Device Math Library (selected)

`fminf`, `fmaxf`, `sqrtof`, `fabsf`, `sinf`, `cosf`, `expf`, `logf`, `powf`, `fmaf` (fused multiply-add). The `__host__ __device__` versions work on both sides (Chapter 10, §10.4). Fast approximations live in the `__` prefixed forms (`__expf`, `__sinf`); enable globally with `--use_fast_math` - but measure before trusting (Chapter 16, §16.7).

A.9 Warp-Level Primitives

Primitive	Meaning	Chapter
<code>__shfl_down_sync(mask, val, delta)</code>	Move <code>val</code> from lane <code>lane+delta</code> to <code>lane</code>	8
<code>__shfl_sync</code> , <code>__shfl_up_sync</code> , <code>__shfl_xor_sync</code>	Other shuffle directions	8
<code>0xfffffffffu</code>	The 32-lane mask for <code>_sync</code> primitives	8
<code>__activemask()</code>	Mask of currently active lanes (use with care)	8

All `_sync` primitives require all named lanes to execute them.

A.10 Driver API and NVRTC (Chapter 12)

Primitive	Meaning
<code>cuInit</code> , <code>cuDeviceGet</code> , <code>cuCtxCreate</code> , <code>cuCtxDestroy</code>	Context lifecycle
<code>cuModuleLoadData</code> , <code>cuModuleGetFunction</code> , <code>cuModuleUnload</code>	Load PTX/cubin, fetch kernel by name
<code>cuLaunchKernel(f, gx,gy,gz, bx,by,bz, smem, stream, args, extra)</code>	Raw launch with <code>void*</code> argument array
<code>nVRTCCreateProgram</code> , <code>nVRTCCompileProgram</code> , <code>nVRTCGetPTX</code> , <code>nVRTCGetProgramLog</code>	Runtime compilation of CUDA source
PTX / cubin / fatbin	Portable ISA / SASS binary / multi-arch container

A.11 Tools and Environment (Chapter 16)

Tool	Purpose
<code>nvcc</code>	Offline compiler (<code>-arch=compute_60</code> portable PTX, or <code>-arch=sm_87</code> on Jetson Orin, <code>-ptx</code>)
<code>nsys profile</code>	System-level timeline
<code>ncu --set full</code>	Kernel-level counters
<code>compute-sanitizer --tool memcheck/racecheck/initcheck/synccheck</code>	Runtime error detection
<code>cuda-gdb</code>	Interactive device debugger
<code>clock64()</code>	In-kernel cycle counter
<code>CUDA_CACHE_MAXSIZE</code>	JIT cache size (Chapter 12)

Appendix B - Mathematical Notation Reference

This appendix gathers the mathematics that appears throughout the text. It is not a mathematics course; it is a dictionary. Every symbol below appears at least once in the main chapters.

B.1 Sets and Scalars

Symbol	Meaning	First use
$\mathbb{R}$	The real numbers; $\mathbb{R}^n$ is n -dimensional real space	Ch. 1
n, N, k, i, j	Indices and sizes (integers)	Ch. 1-15
p	The number of processing units (threads, cores)	Ch. 1
f	The serial fraction of a workload (Amdahl)	Ch. 1
s	The serial fraction of parallel time (Gustafson)	Ch. 1

B.2 Performance Quantities

Symbol	Meaning	Definition	First use
T_1	Serial execution time	-	Ch. 1
T_p	Execution time on p units	-	Ch. 1
$S(p)$	Speedup	T_1/T_p	Ch. 1
$E(p)$	Efficiency	$S(p)/p$	Ch. 1
P_{peak}	Peak FLOP rate (FLOP/s)	-	Ch. 1
B	Peak memory bandwidth (bytes/s)	-	Ch. 1
I	Arithmetic intensity	FLOPs ÷ bytes	Ch. 1
I_{ridge}	Ridge point	P_{peak}/B	Ch. 1

The roofline inequality: $P \leq \min(P_{\text{peak}}, I \cdot B)$, with the two regimes *compute-bound* ($I > I_{\text{ridge}}$) and *memory-bound* ($I < I_{\text{ridge}}$). Chapter 1, §1.8.

B.3 Sums and Sequences

Symbol	Meaning	First use
$\sum_{i=0}^{n-1} a_i$	The sum of the sequence $a_0, \dots, a_{n-1}$	Ch. 8
a_i	The i -th element of a sequence	Ch. 8
$\log_2 n$	The base-2 logarithm of n (the tree height)	Ch. 8
$\lfloor x \rfloor$	The floor of x (largest integer $\leq x$)	Ch. 2

Tree reduction performs $n/2$ additions per level over $\log_2 n$ levels; the bank index is $\lfloor \text{addr}/4 \rfloor \bmod 32$. Chapter 2, §2.8; Chapter 8.

B.4 Matrices and Vectors

Symbol	Meaning	First use
A, B, C	Matrices (bold or capital letters)	Ch. 9
$A[i][j]$ or a_{ij}	The element at row i , column j	Ch. 9
N	Matrix dimension ($N \times N$)	Ch. 9
$C = A \times B$	Matrix product: $c_{ij} = \sum_k a_{ik} b_{kj}$	Ch. 9
G_x, G_y	Sobel derivative kernels	Ch. 15

The FLOP count of $N \times N$ multiplication: $2N^3$. The arithmetic intensity: $N/6$ FLOP/byte. Chapter 9, §9.1.

B.5 Statistics and Error

Symbol	Meaning	First use
σ	Standard deviation (Gaussian blur radius)	Ch. 15
$\max x - y $	Maximum absolute difference (verification)	Ch. 3
1×10^{-3}	The float-stage verification tolerance	Ch. 15

The Gaussian weights of the 5-tap blur, $\sigma = 1$:
 $[0.06136, 0.24477, 0.38774, 0.24477, 0.06136]$. Chapter 15, §15.3.

B.6 Greek Letters Used

Letter	Role in this book
α	SAXPY scaling factor ($y = \alpha x + y$)
β	cuBLAS GEMM scaling factor ($C = \alpha AB + \beta C$)
σ	Gaussian standard deviation
Σ	Summation operator

B.7 Conventions

- **Row-major** storage is assumed everywhere unless stated otherwise: element (i, j) of a W -wide matrix lives at linear index $i \cdot W + j$. Consecutive threads map to consecutive j - the coalescing convention of Chapter 7.
- **Zero-based** indexing, matching the hardware (`threadIdx.x` starts at 0).
- **FLOPs** counts a fused multiply-add as two operations (the convention behind Chapter 2's peak rates).
- **Powers of two** dominate block sizes and tile sizes; when a formula requires one, the text says so (Chapter 8, §8.3).

Appendix C - Recommended Reading & Tools

C.1 Books

- **Programming Massively Parallel Processors: A Hands-on Approach** - David B. Kirk and Wen-mei W. Hwu. The standard academic text on GPU programming; the source of many of the patterns this book presents from first principles (reduction, tiling, coalescing).
- **CUDA C++ Best Practices Guide** - NVIDIA. The official optimisation handbook; the §7.9 checklist in this book is a compressed version of its structure.
- **Parallel Programming with C++** - Richard Vuduc et al. (course notes). The mathematics of Chapter 1 in more depth (Amdahl, Gustafson, roofline).
- **The Rust Book** - Steve Klabnik and Carol Nichols. The language reference for Part V, free online.
- **Performance Analysis of the CUDA Programming Model** - for the warp-level and SIMT details behind Chapter 2.

C.2 Official Documentation (Primary Sources)

- **CUDA C++ Programming Guide** - the authoritative language reference.
- **PTX ISA Reference** - the virtual ISA of Chapters 3 and 12, in detail.
- **CUDA Toolkit Documentation** - API references for the runtime, driver, cuBLAS, Thrust, CUB, cuFFT, cuRAND, NVRTC.
- **Nsight Compute / Nsight Systems User Guides** - the profilers of Chapter 16.
- **Compute Sanitizer User Guide** - the debugger of Chapter 16.
- **CUDA-Oxide (NVLabs/cuda-oxide)** - the README and examples are the primary source for Chapter 14; the project moves quickly, so read the repository, not just this book.
- **NCCL Documentation** - the official `nccl` user guide and API reference for Chapter 19 (collectives, algorithms, environment variables).

- **NVLink & NVSwitch** - NVIDIA's interconnect architecture papers and DGX system guides for Chapter 18.
- **Linux kernel PCI/IOMMU documentation** - [Documentation/PCI/](#) and [Documentation/IOMMU.txt](#) for the systems layer of Chapter 17.

C.3 Tools

Tool	Purpose	Chapter
<code>nvcc</code>	CUDA compiler driver	3
<code>nsys</code>	System-level profiling	16
<code>ncu</code>	Kernel-level profiling	16
<code>compute-sanitizer</code>	Memory/race/init/sync checking	16
<code>cuda-gdb</code>	Device debugging	16
<code>cuobjdump, nvdiasm</code>	Inspect cubins, SASS	12
<code>deviceQuery</code> (CUDA sample)	Hardware capabilities	2
<code>cudaOccupancyMaxActiveBlocksPerMultiprocessor</code>	Occupancy computation	2, 16
<code>cargo-oxide</code>	CUDA-Oxide build driver	14
<code>cudarc</code> (crates.io)	Rust host wrapper	13
<code>lspci</code>	PCIe devices, BARs, drivers	17
<code>nvidia-smi topo -m</code>	GPU-to-GPU topology (NVLink vs PCIe)	18
<code>nvidia-smi nvlink -s</code>	NVLink link status	18
<code>nccl-tests</code>	Collective bandwidth/latency benchmarks	19

C.4 Online Resources

- **NVIDIA Developer Blog** - architecture deep dives (tensor cores, TMA, CUDA Graphs).
- **NVIDIA GPU Computing Samples** (`cuda-samples` repository) - reference kernels for every pattern in this book.
- **NVIDIA's official CUDA-Oxide repository and Discord** - the community around the Rust compiler of Chapter 14.
- **crates.io/cudarc** - the Rust host library of Chapter 13, with examples.

C.5 Cloud GPU Services

If you do not own an NVIDIA GPU, the cloud gives you enough free compute to finish this book. The quickest starts: **Google Colab** (free T4 in a browser, zero setup) and **Kaggle Notebooks** (roughly 30 free GPU hours per week). For serious sessions, the big clouds offer new-account credits (Google Cloud roughly USD 300, Microsoft Azure roughly USD 200); for cheap on-demand GPUs, try Lambda, RunPod or Vast.ai; for serverless Python with recurring credits, Modal. **NVIDIA LaunchPad** offers free, time-boxed hands-on labs on real NVIDIA hardware.

The repository README keeps a fuller comparison table with links; credit amounts change frequently, so always check the provider's current terms.

C.6 How to Continue

1. **Re-implement the capstone** (Chapter 15) from memory, without looking. The gaps in your memory are the gaps in your understanding.
2. **Profile your own machine.** Run `deviceQuery`, compute your ridge point (Chapter 1), and take one kernel from this book to 90% of measured peak bandwidth using Chapter 7's checklist.
3. **Read the CUDA-Oxide repository** and track its releases. The alpha project of Chapter 14 is the fastest-moving part of this book.
4. **Port the pipeline** to a cross-vendor API (SYCL, HIP, or wgpu) and observe which ideas transfer unchanged. The answer - almost all of them - is the epilogue's point made measurable.